Алгоритмы и структуры данных

1 семестр · Полное руководство по подготовке к коллоквиуму

Авторский сборник ответов у доски, строгих математических доказательств, инвариантов, эталонных C++ реализаций и ловушек экзаменаторов по всем 17 темам

Университет ИТМО · Факультет информационных технологий и программирования

Санкт-Петербург · 2026 г.

Содержание руководства

Подготовка к устному коллоквиуму по Алгоритмам и Структурам Данных (1 семестр) 5
Полный сборник ответов, математических доказательств, C++ реализаций и ловушек экзаменаторов 5
Тема 1. Оценка сложности (время, память, Big O, функция сложности) 6
1. Модель вычислений RAM (Random Access Machine) 6
2. Функция сложности и сценарии анализа (Worst, Average, Best) 6
3. Пространственная сложность (Space Complexity) 7
4. Асимптотические обозначения ($O, \Omega, \Theta, o, \omega$) 7
5. Любимые ловушки экзаменаторов по этой теме 8
Тема 2. Мастер-теорема (Master Theorem) 10
1. Откуда берётся уравнение: дерево рекурсии 10
2. Три случая: битва «Листья против Корня» 10
3. Главные ловушки для 1 и 2 потока (где срезают на «отлично») 12
4. Что делать, если подзадачи разного размера? (Метод Акра — Баззи) 13
Тема 3. Сортировка вставками (Insertion Sort) 15
1. Визуальная интуиция и Инвариант цикла 15
2. Эталонный код на C++ и две критические детали 16
3. Точная связь с инверсиями (Математика сложности) 16
4. Сортировка бинарными вставками (Binary Insertion Sort) 17
5. Почему Insertion Sort незаменим на практике? 17
Тема 4. Сортировка слиянием (Merge Sort) 19
1. Процедура слияния (`Merge`) и точный подсчёт сравнений 19
2. Математический анализ: почему гарантированно $\Theta(n \log n)$? 20
3. Пространственная сложность и правильная работа с памятью 20
4. Две модификации, за которые ставят «отлично» 21
5. Почему MergeSort — абсолютный король для связных списков? 22
Тема 5. Быстрая сортировка (Quick Sort) 23
1. Сравнение схем разбиения: Ломуто против Хоара 23
2. Строгое доказательство среднего времени $\Theta(n \log n)$ 24
3. Гарантия стека памяти $O(\log n)$ (Tail Call Optimization) 25
4. Трёхпутевое разбиение (3-Way Partition / Флаг Нидерландов) 25
5. Доказательство неустойчивости 26
1. Код разбиения Ломуто наравне со схемой Хоара 26
2. Устойчивость QuickSort через массив индексов 27
Тема 6. Порядковые статистики и QuickSelect 29
1. Постановка задачи 29
2. Алгоритм Хоара: QuickSelect 29
3. Математика сложности QuickSelect 30
4. Детерминированный выбор за $O(n)$: Алгоритм BFPRT (Median of Medians) 30
Цветная схема матрицы (разбор на пальцах) 30
4. Строгое доказательство линейности $O(n)$ с точными константами 31
5. Строгое доказательство нелинейности для групп из 3 элементов 32
5. Что используется на практике: `IntroSelect` (`std::nth_element`) 34
Тема 7. Сортировка подсчётом (Counting Sort) 35
1. Как обойти теорему о $\Omega(n \log n)$? 35
2. Пошаговый алгоритм на пальцах 35
3. Наглядный числовой пример трассировки 36
4. Эталонный код на C++ 36

5. Главные ловушки экзаменаторов по Counting Sort 37
Тема 8. Цифровая сортировка (Radix Sort) 39
1. LSD Radix Sort (От младших разрядов к старшим) 39
2. Полный глубокий разбор MSD Radix Sort (Most Significant Digit) 40
Поразрядная сортировка по старшим разрядам (MSD Radix Sort) 41
1. Обработка строк переменной длины (Sentinel `-1`) 41
2. Пошаговая механика работы алгоритма 41
3. Подробная трассировка на живом примере 42
4. Эталонный C++ код с оптимизацией Insertion Sort 43
5. Глубокий теоретический анализ (Ловушки экзаменаторов) 44
3. Математика сложности и выбор оптимального основания b 45
4. Эталонный C++ код для 32-битных чисел (Байтовая сортировка) 46
5. Любимые ловушки экзаменаторов 46
Тема 9. Пирамида (Двоичная куча) 48
1. Адресация в массиве без указателей 48
2. Две базовые операции: `siftUp` и `siftDown` 49
3. Два вида построения кучи (Build Heap) 50
4. Строгое доказательство сложности $\Theta(n)$ для алгоритма Флойда 50
5. Важные вопросы на понимание 52
Тема 10. Пирамидальная сортировка (HeapSort) 53
1. Концепция: как превратить Selection Sort в ракету 53
2. Две фазы алгоритма на пальцах 53
3. Эталонный код на C++ (строго in-place) 54
4. Математический анализ сложности 54
5. Доказательство неустойчивости (Контрпример) 55
6. Главный практический парадокс: почему QuickSort быстрее HeapSort? 55
Тема 11. Очередь с приоритетом на пирамиде 57
1. Что такое «Очередь» вообще? (АТД vs Структура данных) 57
2. Что такое Очередь с приоритетом (Priority Queue)? 57
3. Баттл структур данных: почему именно Куча (Пирамида)? 58
4. Изменение приоритета: `increaseKey` и `decreaseKey` 58
5. Главная проблема и супер-решение: Позиционный массив (Indirect Heap) 59
6. Зачем это нужно? Алгоритм Дейкстры 60
Тема 12. Динамическое программирование: основы и архитектура 61
1. Откуда такое странное название? 61
2. Два обязательных признака: когда ДП РАБОТАЕТ, а когда НЕТ? 61
3. Четыре шага проектирования ЛЮБОГО ДП (Чек-лист Беллмана) 62
4. Разбор на числах Фибоначчи: эволюция от ужаса до идеала 62
5. Сравнение Top-Down и Bottom-Up (Любимый вопрос на защите) 64
6. Граф состояний (DAG): глубокий взгляд для 1 и 2 потока 64
7. Бонус для 1 потока: Фибоначчи за $O(\log n)$ (Матричная экспоненциация) 64
Тема 13. ДП: Задача о кузнечике 66
1. Постановка задачи с монетами/стоимостями 66
2. Базовая реализация и восстановление пути 66
3. Оптимизация памяти: Сжатие до $O(k)$ 67
4. Оптимизация времени: от $O(\mathrm{nk})$ к $O(n)$ через Монотонный Дек 67
5. Комбинаторный кузнечик: $O(k^3 \log n)$ матрицами 69
Тема 14. Задача о рюкзаке (Knapsack Problem) 70
1. Постановка задачи и почему Жадный алгоритм не работает 70
2. Вариант 1: Классический 0-1 рюкзак (Каждый предмет берётся ≤ 1 раза) 71
3. Сжатие памяти до $O(W)$: Механизм обратного прохода 72

4. Восстановление ответа в 0-1 рюкзаке (Какие предметы взяли?) ... 73
5. Вариант 2: Неограниченный рюкзак (Unbounded Knapsack) ... 73
6. Вариант 3: Рюкзак по стоимости (Когда $W = 10^9$, но цены маленькие) ... 73
7. Вариант 4: Метод «Встреча посередине» (Meet-in-the-Middle) ... 74
8. Почему задача о рюкзаке NP-полная и что такое «псевдополиномиальное время»? ... 75
Тема 15. Наибольшая возрастающая подпоследовательность (НВП) ... 76
1. Подпоследовательность против Подмассива ... 76
2. Базовое квадратичное решение за $\Theta(n^2)$ ... 76
3. Быстрый алгоритм за $\Theta(n \log n)$ (Массив `tails` и бинпоиск) ... 77
4. Главная ловушка: Хранит ли массив `tails` саму НВП? ... 79
5. Вариация: Неубывающая подпоследовательность (LNDS) ... 80
6. Прорыв для 1 и 2 потока: Связь с Таблицами Юнга и RSK ... 80
Тема 16. Динамика на отрезках и по профилю ... 81
БЛОК 1. Динамика на отрезках (Interval DP) ... 82
1. В чём главная идея «отрезков»? ... 82
2. Классическая задача: Оптимальное перемножение цепочки матриц ... 82
3. Решение через ДП на отрезках ... 82
БЛОК 2. Динамика по профилю (Profile DP) ... 84
1. Задача о замощении сетки доминошками 2×1 ... 84
2. Что такое «Профиль»? (Образ кирпичной стены) ... 84
3. Базовый постолбцовый профиль: $\mathrm{dp}[i][\mathrm{mask}]$ ... 84
4. Изломанный профиль (Broken Profile): Шедевр за $O(N \cdot M \cdot 2^N)$ ... 85
5. Для 1 потока: Матричная экспоненциация $O(8^N \log M)$ ... 86
Тема 17. Редакционное расстояние (Алгоритм Вагнера — Фишера) ... 87
1. Три элементарные операции Левенштейна ... 87
2. Переход к ДП: Алгоритм Вагнера — Фишера (1974) ... 87
3. Живая трассировка на таблице: «МАРТ» → «КАРТА» ... 88
4. Сжатие памяти до $O(\min(n, m))$ ... 89
5. Высший пилотаж для 1 и 2 потока: Алгоритм Хиршберга (Память $O(n + m)$) ... 89
6. Почему расстояние Левенштейна — это настоящая Метрика? ... 90
7. Модификация: Расстояние Дамерау — Левенштейна ... 90

Подготовка к устному коллоквиуму по Алгоритмам и Структурам Данных (1 семестр)

Полный сборник ответов, математических доказательств, C++ реализаций и ловушек экзаменаторов

> ⚡ **30-секундный ответ у доски**
>
> **Предисловие и регламент устного коллоквиума:**
>
> На устном экзамене по АиСД проверяется не просто знание кода, а глубокое понимание сути:
> 1. **30-секундный ответ у доски:** Определение, модель RAM, асимптотика худшего/среднего/лучшего случаев и память.
> 2. **Инварианты и математика:** Доказательство корректности и строгий вывод времени (через ряды, индукцию или мастер-теорему).
> 3. **Тонкости и границы:** Устойчивость (stability), поведение на дубликатах, in-place свойства и локальность кэша.
> 4. **Код на C++:** Чистая реализация без багов на границах диапазонов.
> 5. **Ловушки экзаменаторов:** Быстрый и уверенный ответ на провокационные контрпримеры.

Тема 1. Оценка сложности (время, память, Big O, функция сложности)

30-секундный ответ на случай, если тебя вызвали к доске

> ⚡ **30-секундный ответ у доски**
>
> «Оценка сложности алгоритма — это зависимость объёма требуемых ресурсов (времени процессора и памяти) от размера входных данных n в абстрактной модели вычислений (RAM). Мы оцениваем худший, средний и лучший случаи. Для формализации масштабируемости при $n \to \infty$ используются асимптотические классы O, Ω, Θ, устраняющие зависимость от тактовой частоты процессора и компилятора».

1. Модель вычислений RAM (Random Access Machine)

Прежде чем считать операции, нужно договориться, **на каком «компьютере»** мы их считаем. В теории алгоритмов используется идеализированная машина — **RAM**:

1. **Память:** Неограниченный массив ячеек (машинных слов).
2. **Адресация:** Доступ к любой ячейке по её номеру (адресу) занимает ровно 1 **такт ($O(1)$)**.
3. **Элементарные операции:** Арифметика ($+, -, \times, /$), логические битовые операции (`&`, `|`, `^`) и сравнения выполняются за 1 **такт ($O(1)$)**.

> ⚠️ **Ловушка экзаменатора**
>
> **Каверзный вопрос экзаменатора: какого размера числа могут храниться в одной ячейке?** Если размер входа равен n, нам нужно уметь адресовать ячейки от 0 до $n-1$. Чтобы записать адрес n, требуется минимум $\log_2 n$ бит. Поэтому в RAM-модели длина машинного слова принимается равной $w = \Theta(\log n)$ **бит**.
>
> Отсюда возникает разница между двумя типами сложности:
> - **Словарная сложность (Unit Cost):** Операция над машинным словом (w бит) стоит $O(1)$. (Используется в курсе по умолчанию).
> - **Битовая сложность (Logarithmic Cost):** Учитывает каждый бит. Сложение двух L-битных чисел требует $\Theta(L)$ операций, а умножение — $\Theta(L^2)$ или $O(L \log L)$.

2. Функция сложности и сценарии анализа (Worst, Average, Best)

Пусть x — конкретный входной набор данных, а $|x| = n$ — его размер (например, длина массива). Функция трудоёмкости $t(x)$ — точное число элементарных операций на входе x.

Поскольку для одного и того же размера n массивы могут быть разными (уже отсортирован, отсортирован наоборот, случайный), мы выделяем 3 функции:

1. Худший случай (Worst-case complexity)

$$T_{\text{worst}}(n) = \max_{|x|=n} t(x)$$

- Максимальное время работы среди **всех** возможных входов размера n.
- **Это главная метрика в computer science**, так как она даёт железную математическую гарантию: «дольше алгоритм работать не будет ни при каких обстоятельствах».

2. Лучший случай (Best-case complexity)

$$T_{\text{best}}(n) = \min_{|x| = n} t(x)$$

- Минимальное время среди всех входов размера n.
- Показывает, насколько алгоритм **адаптивен** к идеальным данным (например, Insertion Sort на уже отсортированном массиве работает за $\Theta(n)$).

3. Средний случай (Average-case complexity)

$$T_{\text{avg}}(n) = \sum_{|x| = n} P(x) \cdot t(x) = \mathbb{E}[t(x)]$$

- Математическое ожидание времени работы при заданном распределении вероятностей входов $P(x)$ (обычно предполагается, что все $n!$ перестановок равновероятны: $P(x) = \frac{1}{n!}$).
- Пример: QuickSort в среднем $\Theta(n \log n)$, хотя в худшем — $\Theta(n^2)$.

---

3. Пространственная сложность (Space Complexity)

Память алгоритма делится на две принципиальные категории:

1. **Входные данные:** Память под сам входной массив (размера n) **НЕ** включается во вспомогательную память алгоритма.
2. **Вспомогательная память (Auxiliary Space):** Память, которую алгоритм запросил сам в процессе работы.
   - **Куча (динамическая память):** Выделенные массивы, буферы слияния (как в `MergeSort` — $O(n)$).
   - **Стек вызовов (Call Stack):** Фреймы рекурсивных функций.

> ⚠️ **Ловушка экзаменатора**
>
> **Ловушка экзаменатора: путаница между числом вызовов и глубиной стека** Память стека определяется **максимальной глубиной дерева рекурсии**, а НЕ общим числом вызовов!
> - В `MergeSort` суммарно происходит $2n - 1$ вызов функции. Но глубина стека в любой момент времени — не более $\lceil \log_2 n \rceil + 1$. Память стека — $\boldsymbol{O(\log n)}$.

Если вспомогательная память алгоритма составляет $O(1)$ (не зависит от n), алгоритм называется **in-place** («на месте»).

---

4. Асимптотические обозначения ($O, \Omega, \Theta, o, \omega$)

Зачем они нужны? Нам не важна точная формула вида $T(n) = 3.5n^2 + 14n + 25$, потому что константы 3.5 и 14 зависят от того, Intel у тебя или AMD, а слагаемое $14n$ при $n = 10^6$ становится ничтожным по сравнению с n^2.

Пусть $f(n), g(n) \geq 0$.

1. O («О-большое») — Верхняя граница

> ⚡ **30-секундный ответ у доски**
>
> «$f(n)$ растёт **не быстрее**, чем $g(n)$ с точностью до константы».

$$f(n) \in O(g(n)) \iff \exists c > 0, ; \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 : \quad 0 \leq f(n) \leq c \cdot g(n)$$

Зачем нужны c и n_0?

- n_0 отсекает поведение на маленьких n (нас интересует предел при $n \to \infty$).
- c позволяет нивелировать скорость процессора.

2. Ω («Омега-большое») — Нижняя граница

> ⚡ **30-секундный ответ у доски**
>
> «$f(n)$ растёт **не медленнее**, чем $g(n)$ с точностью до константы».

$$f(n) \in \Omega(g(n)) \iff \exists c > 0, ; \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 : \quad 0 \leq c \cdot g(n) \leq f(n)$$

Связь: $f(n) \in \Omega(g(n)) \iff g(n) \in O(f(n))$.

3. Θ («Тета») — Точная граница

> ⚡ **30-секундный ответ у доски**
>
> «$f(n)$ и $g(n)$ имеют **одинаковый порядок роста**».

$$f(n) \in \Theta(g(n)) \iff \exists c_1, c_2 > 0, ; \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 : \quad c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Критерий: $f(n) \in \Theta(g(n)) \iff (f(n) \in O(g(n))) \wedge (f(n) \in \Omega(g(n)))$.

---

4. Строгие границы: o («о-малое») и ω («омега-малое»)

Если O и Ω — это аналоги $\leq$ и $\geq$, то o и ω — это строгие аналоги $<$ и $>$.

Обозначение	Смысл через предел $\lim_{n\to\infty} \frac{f(n)}{g(n)}$	Аналогия
$f(n) \in O(g(n))$	$\leq c < \infty$	$f \leq g$
$f(n) \in \Omega(g(n))$	> 0 (может быть ∞)	$f \geq g$
$f(n) \in \Theta(g(n))$	$= c \in (0, \infty)$	$f = g$
$\boldsymbol{f(n) \in o(g(n))}$	$\boldsymbol{= 0}$	$\boldsymbol{f < g}$
$\boldsymbol{f(n) \in \omega(g(n))}$	$\boldsymbol{= \infty}$	$\boldsymbol{f > g}$

Пример: $n \in o(n^2)$, но $2n^2 \notin o(n^2)$ (потому что предел отношения равен $2 \neq 0$, хотя $2n^2 \in O(n^2)$).

---

5. Любимые ловушки экзаменаторов по этой теме

⚠️ Ловушка 1: Знак равенства

Студенты пишут $T(n) = O(n^2)$.

- **Вопрос экзаменатора:** «Что означает этот знак равенства? Симметричен ли он?»
- **Ответ:** Знак равенства здесь — исторически сложившееся математическое злоупотребление. Строго говоря, $O(n^2)$ — это **множество функций**, а правильная запись: $T(n) \in O(n^2)$. Запись несимметрична: написать $O(n) = O(n^2)$ ещё допустимо (множество $O(n)$ вложено в $O(n^2)$), но $O(n^2) = O(n)$ — грубая ошибка.

⚠️ Ловушка 2: Степени с двойкой

- **Вопрос:** «Верно ли, что $2^{n+1} \in O(2^n)$? А $2^{2n} \in O(2^n)$?»
- **Ответ:**
  1. $2^{n+1} = 2 \cdot 2^n$. Здесь константа $c = 2$. **Верно.**
  2. $2^{2n} = (2^2)^n = 4^n$. Отношение $\frac{4^n}{2^n} = 2^n \to \infty$. Константы не существует. **Неверно.** Константу в показателе степени отбрасывать нельзя!

⚠️ Ловушка 3: Несовместимость лучшего и худшего

- **Вопрос:** «Может ли алгоритм иметь худшее время $O(n)$, а в лучшем случае работать за $\Omega(n^2)$?»

- **Ответ:** Нет, не может. Худшее время работы алгоритма по определению не меньше лучшего на любых входах: $T_{\text{best}}(n) \le T_{\text{worst}}(n)$. Если бы это было так, то $c_1 n^2 \le T_{\text{best}}(n) \le T_{\text{worst}}(n) \le c_2 n$, что при больших n невозможно (n^2 не может быть меньше n).

Вопрос на закрепление (проверь себя):
Как соотносятся по скорости роста три функции:

$$f(n) = n^{\log_2 3}, \quad g(n) = 3^{\log_2 n}, \quad h(n) = n\sqrt{n}?$$

Кто из них больше, кто меньше, или кто-то равен через Θ? Попробуй ответить, и перейдём к **Теме 2 (Мастер-теорема)**!

Тема 2. Мастер-теорема (Master Theorem)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Мастер-теорема решает рекуррентные уравнения вида $T(n) = aT(\frac{n}{b}) + f(n)$. Она основана на анализе дерева рекурсии, где суммарная работа определяется соотношением двух величин: работы в листьях дерева ($n^{\log_b a}$) и работы в корне ($f(n)$). В зависимости от того, кто из них полиномиально доминирует (или они равны), мы получаем один из трёх случаев: побеждают листья ($\Theta(n^{\log_b a})$), побеждает корень ($\Theta(f(n))$) или работа размазана по уровням поровну ($\Theta(n^{\log_b a} \log n)$)».

---

1. Откуда берётся уравнение: дерево рекурсии

Любой алгоритм «разделяй и властвуй» (Divide & Conquer) на входе размера n:

1. Делит задачу на a **подзадач** ($a \geq 1$).
2. Каждая подзадача имеет размер $\frac{n}{b}$ (где $b > 1$ — коэффициент сжатия).
3. Тратит время $f(n)$ на то, чтобы разделить задачу и склеить ответы.

Получаем каноническое уравнение:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Что происходит на уровнях дерева рекурсии?

Нарисуем это дерево в голове:

- **Уровень 0 (корень):** 1 узел размера n. Работа = $f(n)$.
- **Уровень 1:** a узлов размера $\frac{n}{b}$. Суммарная работа = $a \cdot f(\frac{n}{b})$.
- **Уровень 2:** a^2 узлов размера $\frac{n}{b^2}$. Суммарная работа = $a^2 \cdot f(\frac{n}{b^2})$.
- …
- **Уровень j:** a^j узлов размера $\frac{n}{b^j}$.
- **Самый нижний уровень (листья):** Размер задачи падает до 1, когда $\frac{n}{b^h} = 1 \Longrightarrow \boldsymbol{h = \log_b n}$ (высота дерева).

Сколько листьев на самом дне?

На глубине h число листьев равно:

$$a^h = a^{\log_b n} = \boldsymbol{n^{\log_b a}}$$

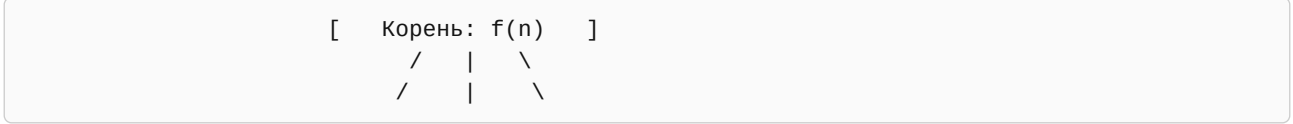
(Это знаменитое логарифмическое тождество: $a^{\log_b n} = n^{\log_b a}$).

Величина $c_{\text{crit}} = \log_b a$ называется **критическим показателем**. Каждый лист — это тривиальная задача размера 1, которая решается за $\Theta(1)$. Значит, **вся работа в листьях стоит $\boldsymbol{\Theta(n^{\log_b a})}$.**

---

2. Три случая: битва «Листья против Корня»

Всё поведение функции $T(n)$ зависит от того, какая сила мощнее: **работа в листьях** ($n^{\log_b a}$) или **работа в корне** ($f(n)$).



```
                    .     .     .
            [ Листья: Theta(n^{log_b a}) ]
```

Случай 1: Доминируют листья (Листья побеждают)

> ⚡ **30-секундный ответ у доски**
>
> «Работа на каждом следующем уровне дерева убывает так быстро (геометрическая прогрессия), что листья забирают асимптотически почти всё время».

Условие: $f(n)$ растёт **полиномиально медленнее**, чем $n^{\log_b a}$. То есть существует $\varepsilon > 0$, такое что:

$$f(n) = O\left(n^{\log_b a - \varepsilon}\right)$$

Итог:

$$\boldsymbol{T(n) = \Theta\left(n^{\log_b a}\right)}$$

Пример (Умножение Карацубы): $T(n) = 3T\left(\frac{n}{2}\right) + O(n)$.
- $a = 3, b = 2$.
- Критический показатель: $\log_2 3 \approx 1.585$.
- Листья стоят: $n^{\log_2 3} \approx n^{1.585}$.
- Корень: $f(n) = n = n^1$.
- Проверяем зазор: $n^1 = O(n^{1.585-\varepsilon})$ при $\varepsilon \approx 0.585 > 0$. Листья сильнее!
- **Ответ:** $T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585})$.

Случай 2: Ничья (Равномерное распределение)

> ⚡ **30-секундный ответ у доски**
>
> «Работа на всех уровнях дерева асимптотически одинакова. Чтобы получить общее время, нужно работу одного уровня умножить на высоту дерева ($\log_b n$)».

Базовое условие: $f(n)$ растёт с той же скоростью, что и $n^{\log_b a}$:

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

Итог:

$$\boldsymbol{T(n) = \Theta\left(n^{\log_b a} \log n\right)}$$

Обобщённый случай 2 (очень любят на 1, 2 потоке!): Если $f(n)$ отличается на логарифмический множитель:

$$f(n) = \Theta\left(n^{\log_b a} \log^k n\right), \quad k \geq 0$$

Тогда каждый уровень даёт этот логарифм, и мы получаем дополнительный логарифм в ответе:

$$\boldsymbol{T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)}$$

Пример (MergeSort): $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$.
- $a = 2, b = 2 \Longrightarrow \log_2 2 = 1$. Листья: n^1.
- Корень: $f(n) = \Theta(n) = \Theta(n^1 \log^0 n)$ ($k = 0$).
- Силы равны! Умножаем на высоту дерева $\log n$.
- **Ответ:** $T(n) = \Theta(n \log n)$.

Случай 3: Доминирует корень (Корень побеждает)

> ⚡ **30-секундный ответ у доски**
>
> «Работа на каждом следующем уровне возрастает так быстро, что корень дерева забирает асимптотически почти всё время».

Условие: $f(n)$ растёт **полиномиально быстрее**, чем $n^{\log_b a}$. То есть существует $\varepsilon > 0$, такое что:

$$f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$$

И ОБЯЗАТЕЛЬНО выполнено условие регулярности (Regularity Condition):

$$\exists c < 1, ; \exists n_0 \quad \forall n \geq n_0 : \quad a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$$

(Смысл условия: дети на следующем уровне суммарно делают строго меньше работы, чем родитель, с коэффициентом $c < 1$).

Итог:

$$\boldsymbol{T(n) = \Theta(f(n))}$$

Пример: $T(n) = 2T\left(\frac{n}{2}\right) + n^2$.

- $a = 2, b = 2 \Longrightarrow \log_2 2 = 1$. Листья: n^1.
- Корень: $f(n) = n^2$.
- Зазор: $n^2 = \Omega(n^{1+1})$ ($\varepsilon = 1 > 0$).
- Проверяем регулярность: $af\left(\frac{n}{b}\right) = 2 \cdot \left(\frac{n}{2}\right)^2 = 2 \cdot \frac{n^2}{4} = \frac{1}{2}n^2 \leq c \cdot n^2$ при $c = \frac{1}{2} < 1$. Выполнено!
- **Ответ:** $T(n) = \Theta(n^2)$.

---

3. Главные ловушки для 1 и 2 потока (где срезают на «отлично»)

⚠️ Ловушка 1: Что такое «полиномиальный зазор» и почему $T(n) = 2T\left(\frac{n}{2}\right) + n\log n$ не решается базовой теоремой?

Посмотрим на уравнение:

$$T(n) = 2T\left(\frac{n}{2}\right) + n\log n$$

- $a = 2, b = 2 \Longrightarrow n^{\log_2 2} = n^1$.
- Функция $f(n) = n\log n$.
- Студент думает: «$n\log n$ больше, чем n, значит, это случай 3! Ответ $\Theta(n\log n)$».
- **ЭКЗАМЕНАТОР СТАВИТ МИНУС!** Почему? Потому что отношение $\frac{f(n)}{n^{\log_b a}} = \frac{n\log n}{n} = \log n$. Логарифм $\log n$ растёт медленнее **любой полиномиальной степени** n^{ε} (даже медленнее $n^{0.0001}$). Полиномиального зазора **нет**! Случай 3 не применим!
- **Как решить правильно:** Использовать **обобщённый случай 2** при $k = 1$:

$$f(n) = \Theta(n^1 \log^1 n) \Longrightarrow T(n) = \Theta(n\log^{1+1} n) = \boldsymbol{\Theta(n\log^2 n)}.$$

⚠️ Ловушка 2: «Чёрная дыра» между Случаем 1 и Случаем 2

А что, если в уравнении стоит деление на логарифм:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

- $n^{\log_2 2} = n$.
- Функция $f(n) = \frac{n}{\log n}$ меньше, чем n.

- Но она меньше не на n^ε, а всего лишь на логарифм!
- В случай 1 мы не попадаем (нет зазора n^ε).
- В случай 2 мы не попадаем (степень логарифма $k = -1$, формула работает только для $k \geq 0$).
- **Вердикт:** Мастер-теорема здесь **бессильна**! (Это уравнение решается непосредственным суммированием по уровням дерева рекурсии: $T(n) = \Theta(n \log\log n)$).

⚠️ Ловушка 3: Нарушение условия регулярности

Экзаменатор даёт уравнение:

$$T(n) = T\left(\frac{n}{2}\right) + n(2 - \cos n)$$

- $a = 1, b = 2 \Longrightarrow \log_2 1 = 0 \Longrightarrow n^0 = 1$.
- Функция $f(n) = n(2 - \cos n) = \Omega(n^1)$ вроде бы полиномиально больше 1.
- Но из-за осцилляций косинуса condition $af\left(\frac{n}{b}\right) \leq cf(n)$ не выполняется для постоянного $c < 1$ (функция то сжимается, то подскакивает). Мастер-теорема не применима.

---

4. Что делать, если подзадачи разного размера? (Метод Акра — Баззи)

На 1 и 2 потоке часто дают уравнение, где подзадачи делятся неравномерно, например:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(2\frac{n}{3}\right) + n$$

Мастер-теорема не может работать с двумя разными знаменателями 3 и 1.5. Здесь используется **теорема Акра — Баззи (Akra — Bazzi)**:

Для рекуррентности $T(n) = \sum_{i=1}^{k} a_i T(b_i n) + g(n)$ мы:
1. Находим вещественное число p из характеристического уравнения:

$$\sum_{i=1}^{k} a_i b_i^p = 1$$

Для нашего примера:

$$1 \cdot \left(\frac{1}{3}\right)^p + 1 \cdot \left(\frac{2}{3}\right)^p = 1 \Longrightarrow \boldsymbol{p = 1}.$$

2. Подставляем p в интегральную формулу:

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du\right)\right)$$

Подставляем $p = 1, g(u) = u$:

$$T(n) = \Theta\left(n^1 \left(1 + \int_1^n \frac{u}{u^2} du\right)\right) = \Theta(n(1 + \ln n)) = \boldsymbol{\Theta(n \log n)}.$$

Твой алгоритм действий у доски (шпаргалка):

1. Выпиши a, b, найди критический показатель $c_{\text{crit}} = \log_b a$.
2. Сравни функцию $f(n)$ с $n^{c_{\text{crit}}}$:
   - Если $f(n)$ полиномиально меньше $\Longrightarrow$ **Случай 1** $\Longrightarrow \Theta\left(n^{\log_b a}\right)$.
   - Если $f(n) = n^{c_{\text{crit}}} \log^k n \Longrightarrow$ **Случай 2** $\Longrightarrow \Theta\left(n^{\log_b a} \log^{k+1} n\right)$.
   - Если $f(n)$ полиномиально больше и регулярна $\Longrightarrow$ **Случай 3** $\Longrightarrow \Theta(f(n))$.

Всё ли кристально понятно по трём случаям и ловушкам? Можем идти к **Теме 3 (Сортировка вставками)** или разобрать какой-то конкретный хитрый пример!

Тема 3. Сортировка вставками (Insertion Sort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Сортировка вставками — это инкрементальный устойчивый (stable) алгоритм сортировки сравнениями, работающий на месте (in-place, доп. память $O(1)$). На каждом шаге i мы берём элемент $A[i]$ и вставляем его в уже отсортированный префикс $A[0...i-1]$, сдвигая бóльшие элементы вправо. Главное свойство — **адаптивность**: время работы алгоритма линейно зависит от числа инверсий в массиве: $T(n) = \Theta(n + I(A))$. В лучшем случае — $\Theta(n)$, в худшем и среднем — $\Theta(n^2)$».

1. Визуальная интуиция и Инвариант цикла

Аналогия с картами

Представь, что ты держишь в левой руке карты. Они уже упорядочены по возрастанию. Ты берёшь со стола очередную карту (key) правой рукой:

- Сравниваешь её с картами в левой руке справа налево (от самых больших к меньшим).
- Сдвигаешь карты, которые больше key, вправо.
- Как только встретил карту, которая $\leq \text{key}$, вставляешь новую карту на освободившееся место.

```
Отсортированный префикс      Текущий     Непросмотренная часть
[ 2    5    7    9 ]    <-  [  6  ]      [ 1    8    3 ]
  0    1    2    3             i           i+1 ... n-1
```

Числа `7` и `9` сдвинутся вправо на одну позицию, и `6` встанет на индекс `2`.

Формальный инвариант внешнего цикла

На экзамене за фразу «ну мы просто вставляем число на своё место» поставят тройку. Формулировать нужно через инвариант:

> **Инвариант цикла:** В начале каждой итерации внешнего цикла по i ($i = 1...n-1$) подмассив $A[0...i-1]$ состоит из тех же самых элементов, которые изначально были на позициях от 0 до $i-1$, но находится **в строго отсортированном порядке**.

Доказательство по индукции:

- **База ($i = 1$):** Подмассив $A[0...0]$ состоит из одного элемента. Массив из одного элемента всегда тривиально отсортирован. Инвариант верен.
- **Шаг (от i к $i+1$):** По предположению индукции $A[0...i-1]$ отсортирован. Внутренний цикл сдвигает элементы $A[j] > \text{key}$ вправо на одну позицию, освобождая позицию для key сразу после первого элемента $\leq \text{key}$. Таким образом, $A[0...i]$ теперь содержит те же элементы плюс key и упорядочен.
- **Завершение ($i = n$):** Цикл завершается, когда $i = n$. Подставляем $i = n$ в инвариант: подмассив $A[0...n-1]$ отсортирован. Весь массив упорядочен!

2. Эталонный код на C++ и две критические детали

```cpp
void insertionSort(std::vector<int>& a) {
    int n = a.size();
    for (int i = 1; i < n; ++i) {
        int key = a[i];
        int j = i - 1;

        // КРИТИЧЕСКИЕ ДЕТАЛИ ЗДЕСЬ:
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
}
```

⚠️ Деталь 1: Почему строго `a[j] > key`, а не `>=`?

Это вопрос на знание **устойчивости (Stability)**:

- Если мы напишем `a[j] > key`, то при встрече дубликата ($A[j] == \text{key}$) условие станет ложным, цикл остановится, и новый дубликат встанет **строго после** старого. Их относительный порядок сохранится! **Алгоритм устойчив**.
- Если случайно написать `a[j] >= key`, мы сдвинем старый равный элемент вправо и вставим новый дубликат левее него. Относительный порядок нарушится — сортировка станет **неустойчивой**.

⚠️ Деталь 2: Порядок условий в `while (j >= 0 && a[j] > key)`

В C++ работает механизм короткого замыкания (`short-circuit evaluation`): Если написать наоборот `while (a[j] > key && j >= 0)`, то при $j = -1$ процессор сначала попытается вычислить `a[-1]`, что приведёт к выходу за границы массива (UB / Segfault). Условие $j \geq 0$ обязано стоять **первым**!

---

3. Точная связь с инверсиями (Математика сложности)

:::meaning **Определение инверсии:** Пара индексов (j, k) образует **инверсию**, если $j < k$, но при этом $A[j] > A[k]$ (то есть элементы стоят не по порядку). :::

Обозначим общее число инверсий в массиве как $I(A)$:

$$I(A) = \bigl|\{(j,k) \mid j < k \wedge A[j] > A[k]\}\bigr|$$

Фундаментальная лемма:

> Каждый элементарный сдвиг `a[j + 1] = a[j]` во внутреннем цикле **уменьшает число инверсий в массиве ровно на** 1.

Почему? Мы меняем местами key и его соседа $A[j]$. Так как $A[j] > \text{key}$, пара $(A[j], \text{key})$ была инверсией и теперь исчезла. Взаимный порядок остальных элементов не изменился.

Отсюда получаем точное число операций:

- **Общее число сдвигов (перемещений):** в точности равно $\boldsymbol{I(A)}$.
- **Общее число сравнений:** равно $I(A) + (n-1)$ — число выходов по $(j < 0)$.
- **Время работы:**

$$\boldsymbol{T(n) = \Theta(n + I(A))}$$

Анализ трёх случаев:

1. **Лучший случай (массив уже отсортирован):**
   - Инверсий нет: $I(A) = 0$.

- Внутренний цикл делает 1 сравнение и 0 сдвигов на каждом шаге.
- $T(n) = \Theta(n)$ — алгоритм мгновенно определяет, что всё отсортировано.

2. **Худший случай (массив отсортирован по убыванию):**
   - Каждая пара образует инверсию. Их число равно числу сочетаний:

$$I(A) = \binom{n}{2} = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$$

- $T(n) = \Theta(n^2)$.

3. **Средний случай (случайная перестановка):**
   - Для любой пары элементов (x, y) с равной вероятностью $\frac{1}{2}$ либо $x > y$, либо $x < y$.
   - Математическое ожидание числа инверсий:

$$\mathbb{E}[I(A)] = \frac{1}{2} \cdot \frac{n(n-1)}{2} = \frac{n(n-1)}{4}$$

- $T(n) = \Theta(n^2)$ (с константой в 2 раза меньше худшего случая).

---

4. Сортировка бинарными вставками (Binary Insertion Sort)

Идея очевидна: префикс $A[0...i-1]$ уже отсортирован! Зачем искать позицию для вставки линейно за $O(i)$, если можно найти её **бинарным поиском** за $O(\log i)$?

- Поиск позиции: `std::upper_bound` за $O(\log i)$.
- Суммарное число сравнений для всего массива:

$$\sum_{i=1}^{n-1} \lceil \log_2 i \rceil = \Theta(n \log n).$$

> ⚠️ **Ловушка экзаменатора**
>
> **Любимый вопрос экзаменатора:** «Если число сравнений стало $O(n \log n)$, значит ли это, что Binary Insertion Sort работает за $O(n \log n)$?»
>
> **Ответ: НЕТ!** Бинарный поиск позволяет быстро **найти** индекс вставки. Но чтобы освободить ячейку для key, нам всё равно нужно **физически сдвинуть** все элементы правее него. Число сдвигов по-прежнему равно числу инверсий $I(A) = \Theta(n^2)$. Поэтому общее время работы остаётся $\mathbf{\Theta(n^2)}$!

(Примечание для 1, 2 потока: если использовать связный список, то сдвиги станут $O(1)$, но в списке нельзя делать бинарный поиск за $O(\log n)$! В итоге обмануть физику массива не удаётся).

---

5. Почему Insertion Sort незаменим на практике?

Казалось бы, зачем учить алгоритм за $O(n^2)$, если есть QuickSort и MergeSort за $O(n \log n)$? В реальных библиотеках (включая STL C++ и Java) Insertion Sort используется постоянно:

1. **Крошечная скрытая константа:** У него нет оверхеда на рекурсию, нет выделения дополнительной памяти.
2. **Идеален на малых n ($n \leq 16...32$):** Из-за константы на массивах до 32 элементов Insertion Sort обгоняет QuickSort! Поэтому в `std::sort` (Introsort) и `pdqsort`, когда рекурсия дробит массив до отрезков ≤ 16, алгоритм переключается на Insertion Sort.
3. **Почти отсортированные данные:** Если в массив из миллиона элементов дописать в конец 5 новых чисел, QuickSort будет перелопачивать всё заново, а Insertion Sort вставит их за линейное время $O(n)$.
4. **Основа Timsort:** Стандартная сортировка Python и Java (`Arrays.sort`) разбивает массив на монотонные куски и досортировывает их с помощью Binary Insertion Sort.

Чек-лист свойств для таблицы на коллоквиуме:

- **Время (лучшее / среднее / худшее):** $O(n)$ / $\Theta(n^2)$ / $\Theta(n^2)$.
- **Память:** $O(1)$ (in-place).
- **Устойчивость:** Да (Stable).
- **Связь с инверсиями:** $T(n) = \Theta(n + I(A))$.

Всё ли уложилось по полочкам? Если да — погнали в **Тему 4: Сортировка слиянием (Merge Sort)**!

Тема 4. Сортировка слиянием (Merge Sort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Сортировка слиянием — это устойчивый (stable) алгоритм парадигмы «разделяй и властвуй». Мы делим массив пополам, рекурсивно сортируем обе половины, а затем сливаем два упорядоченных подмассива в один за линейное время процедурой `Merge` с двумя указателями. Время работы **строго** $\Theta(n \log n)$ во всех случаях (лучший, средний, худший). Вспомогательная память — $O(n)$ под буфер слияния $+O(\log n)$ стек рекурсии. Это идеальный алгоритм для связных списков (где память падает до $O(1)$) и внешней сортировки больших файлов».

1. Процедура слияния (`Merge`) и точный подсчёт сравнений

Вся магия алгоритма сосредоточена в одной процедуре — объединении двух уже отсортированных массивов в один.

Визуализация: метод двух указателей

Пусть у нас есть два отсортированных отрезка: левый $A[l \dots \text{mid}]$ и правый $A[\text{mid}+1 \dots r]$. Мы ставим два указателя: i на начало левого, j на начало правого.

```
Левый подмассив:   [ 2    5    8 ]          Правый:   [ 3    6    9 ]
                     ^                                  ^
                     i                                  j
Буфер:             [   ]
```

Сравниваем $A[i]$ и $A[j]$:

- Меньший элемент записываем в буфер, а его указатель сдвигаем вправо.
- Если один из массивов закончился, просто дописываем остаток второго массива в буфер.
- В конце копируем буфер обратно в исходный массив.

```cpp
void merge(std::vector<int>& a, int l, int mid, int r, std::vector<int>& buf) {
    int i = l;        // бежит по левой половине [l ... mid]
    int j = mid + 1;  // бежит по правой половине [mid+1 ... r]
    int k = l;        // позиция для записи в буфер

    while (i <= mid && j <= r) {
        // КРИТИЧЕСКИ ВАЖНО: именно знак <= обеспечивает устойчивость (stability)!
        if (a[i] <= a[j]) {
            buf[k++] = a[i++];
        } else {
            buf[k++] = a[j++];
        }
    }

    // Дописываем "хвосты", если один из отрезков исчерпался
    while (i <= mid) buf[k++] = a[i++];
    while (j <= r)   buf[k++] = a[j++];

    // Копируем результат обратно в массив a
```

```
    for (int p = l; p <= r; ++p) {
        a[p] = buf[p];
    }
}
```

Точное число сравнений в процедуре `Merge`

Пусть мы сливаем два отрезка суммарной длины $m = r - l + 1$ (каждый размера $\frac{m}{2}$). Каждое сравнение $A[i] \leq A[j]$ гарантированно отправляет один элемент в буфер.

- **Худший случай ($m - 1$ сравнений):** элементы чередуются, например, $[1, 3, 5]$ и $[2, 4, 6]$. Чтобы положить $m - 1$ элемент, нужно сделать ровно $m - 1$ сравнений. Последний элемент допишется без сравнения.
- **Лучший случай ($\frac{m}{2}$ сравнений):** все элементы одного подмассива меньше всех элементов другого, например, $[1, 2, 3]$ и $[4, 5, 6]$. Первые три сравнения отправят левую часть в буфер, левый указатель выйдет за границу, и правая часть скопируется вообще без сравнений.

> Итого: процедура `Merge` для массива длины m делает от $\frac{m}{2}$ до $m - 1$ сравнений.

2. Математический анализ: почему гарантированно $\Theta(n \log n)$?

Алгоритм делит задачу размера n на 2 подзадачи размера $\frac{n}{2}$, а склейка стоит $\Theta(n)$:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Через Мастер-теорему:

- $a = 2, b = 2 \Longrightarrow c_{\text{crit}} = \log_2 2 = 1$.
- Функция склейки: $f(n) = \Theta(n^1) = \Theta\left(n^{\log_b a} \log^0 n\right)$.
- Это в точности **Случай 2** при $k = 0$:

$$T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right) = \boldsymbol{\Theta(n \log n)}.$$

Через дерево рекурсии:

- Высота дерева: ровно $\lceil \log_2 n \rceil$ уровней.
- На каждом уровне суммарная длина всех сливаемых кусочков равна в точности n.
- Значит, **на каждом уровне выполняется ровно $\Theta(n)$ работы**.
- Умножаем работу одного яруса на высоту: $n \times \log_2 n = \boldsymbol{\Theta(n \log n)}$.
- Никакой входной массив не может испортить эту структуру: деление всегда строго пополам!

3. Пространственная сложность и правильная работа с памятью

Многие студенты совершают грубую ошибку в коде:

```
// ПЛОХОЙ КОД:
void mergeSort(std::vector<int>& a, int l, int r) {
    ...
    std::vector<int> buf(r - l + 1); // ВЫДЕЛЕНИЕ ПАМЯТИ НА КАЖДОМ ВЫЗОВЕ!
    ...
}
```

Если делать `vector` внутри каждого рекурсивного вызова:

1. Вызовы системного аллокатора (`new` / `malloc`) безумно замедлят алгоритм.
2. В памяти будет одновременно жить куча временных массивов.

Золотой стандарт:
Один общий буфер размера n выделяется **один раз в самом начале** (в функции-обёртке) и передаётся по ссылке во все рекурсивные вызовы.

- **Вспомогательная память в куче:** строго $O(n)$ (один буфер).
- **Стек рекурсии:** глубина дерева $\lceil \log_2 n \rceil + 1 \Longrightarrow O(\log n)$.
- **Итоговая доп. память:** $\boldsymbol{O(n)}$.

---

4. Две модификации, за которые ставят «отлично»

Модификация 1: Подсчёт инверсий за $O(n \log n)$

Это классическая задача, которую обожают давать на коллоквиуме.

Как посчитать, сколько пар (i, j) образуют инверсию ($i < j$, но $A[i] > A[j]$), быстрее, чем за $O(n^2)$? **Встроить счётчик прямо в MergeSort!**

Посмотрим на шаг слияния:

```
Левая часть:   [ 3    7    9 ]      Правая часть:   [ 2    4    8 ]
                 ^                                   ^
                 i                                   j
```

Сейчас $A[j] = 2$ меньше, чем $A[i] = 3$. Но левая часть **уже отсортирована**! Это значит, что если $A[j]$ меньше $A[i]$, то он **гарантированно меньше всех остальных элементов левой части** (и 7, и 9)! Сколько таких элементов? Ровно от текущего индекса i до конца левого отрезка mid:

$$\text{инверсий к } A[j] = \mathbf{mid} - \boldsymbol{i} + \mathbf{1}.$$

В этой строчке кода мы просто прибавляем:

```
if (a[i] <= a[j]) {
    buf[k++] = a[i++];
} else {
    buf[k++] = a[j++];
    inversions_count += (mid - i + 1); // <-- ОДНА СТРОЧКА!
}
```

Алгоритм находит общее число инверсий за те же самые $\Theta(n \log n)$ попутно с сортировкой!

Модификация 2: Итеративный (Bottom-Up) MergeSort

Можно ли сделать MergeSort вообще без рекурсии и без стека вызовов? Да! Мы идём снизу вверх:

1. Сначала сливаем соседние отрезки длины $\mathrm{width} = 1$ в пары длины 2.
2. Затем сливаем пары длины $\mathrm{width} = 2$ в четвёрки длины 4.
3. Затем $\mathrm{width} = 4, 8, 16, \ldots$ пока $\mathrm{width} < n$.

```
void bottomUpMergeSort(std::vector<int>& a) {
    int n = a.size();
    std::vector<int> buf(n);

    for (int width = 1; width < n; width *= 2) {
        for (int l = 0; l < n; l += 2 * width) {
            int mid = std::min(l + width - 1, n - 1);
            int r = std::min(l + 2 * width - 1, n - 1);
            if (mid < r) {
                merge(a, l, mid, r, buf);
            }
        }
```

```
    }
}
```

- Нет рекурсии $\Longrightarrow$ глубина стека $O(1)$.
- Число проходов внешнего цикла: ровно $\lceil \log_2 n \rceil$.
- Идеально для последовательного доступа к данным на диске или ленте (External Sort).

---

5. Почему MergeSort — абсолютный король для связных списков?

Это коронный вопрос на экзамене:

> **Вопрос:** «Почему для массивов чаще используют QuickSort, а для односвязных списков (`std::list`) — исключительно MergeSort?»

Ответ из трёх пунктов:

1. **Память падает с $O(n)$ до строгого $O(1)$!** В связном списке нам не нужен вспомогательный буфер. При слиянии двух отсортированных списков мы не копируем данные, а просто **переставляем указатели** `next` за $O(1)$ памяти!
2. **Отсутствие произвольного доступа не мешает:** Чтобы найти середину списка, мы используем алгоритм «быстрого и медленного указателя» (Floyd's Tortoise and Hare) за один линейный проход.
3. **QuickSort на связных списках плох:** В QuickSort схема Хоара требует встречного движения указателей с конца к началу (чего односвязный список делать не умеет, там нет `prev`).

Чек-лист свойств MergeSort:

Характеристика	Значение
Время (лучшее / среднее / худшее)	$\Theta(n \log n)$ / $\Theta(n \log n)$ / $\Theta(n \log n)$
Память на массивах	$O(n)$ буфер $+O(\log n)$ стек
Память на связных списках	$O(1)$ вспомогательная память
Устойчивость (Stability)	**Да** (благодаря условию `<=`)
In-place	**Нет** (на массивах требует $O(n)$ памяти)

Как ощущения? Всё прозрачно? Если да — идём к **Теме 5: Быстрая сортировка (QuickSort, Хоар и Ломуто)**!

Тема 5. Быстрая сортировка (Quick Sort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «QuickSort — это алгоритм парадигмы «разделяй и властвуй», работающий на месте (in-place) и являющийся неустойчивым (unstable). Мы выбираем опорный элемент (pivot) и процедурой `Partition` делим массив на элементы меньше/равные pivot и больше/равные pivot, после чего рекурсивно сортируем обе части. В среднем и лучшем случае время работы — $\Theta(n \log n)$ с наименьшей константой среди всех сортировок сравнениями благодаря идеальной локальности кэша. В худшем случае — $\Theta(n^2)$. Память стека рекурсии при правильной элиминации хвостовой рекурсии составляет $O(\log n)$».

1. Сравнение схем разбиения: Ломуто против Хоара

На экзамене обязательно спросят: **«Какую схему разбиения вы написали и чем она отличается от альтернативной?»**

Давай сведем их в четкую сравнительную таблицу:

Критерий	Схема Ломуто (Lomuto)	Схема Хоара (Hoare)
Движение указателей	В одну сторону слева направо ($i, j \rightarrow$)	Навстречу друг другу ($i \rightarrow \dots \leftarrow j$)
Положение pivot в конце	Встаёт **строго на своё финальное место**	**Не обязательно** встаёт на индекс раздела
Рекурсивные границы	`[l, p - 1]` и `[p + 1, r]` (pivot исключается)	`[l, p]` и `[p + 1, r]` (pivot попадает в один из кусков)
Число обменов (swap)	Большое (делает своп почти на каждом элементе $\leq \text{pivot}$)	**В среднем в 3 раза меньше** (меняет только пары нарушителей)
Массив из одинаковых чисел	**Катастрофа:** делит на $n-1$ и 0, время $\Theta(n^2)$	**Идеально:** делит ровно пополам, время $\Theta(n \log n)$

Реализация схемы Хоара (золотой стандарт C++)

```cpp
int hoarePartition(std::vector<int>& a, int l, int r) {
    // Выбираем центральный элемент, чтобы не падать на отсортированных данных
    int pivot = a[l + (r - l) / 2];
    int i = l - 1;
    int j = r + 1;

    while (true) {
        do { ++i; } while (a[i] < pivot);
        do { --j; } while (a[j] > pivot);

        if (i >= j) {
            return j; // Точка раздела: левее j все <= pivot, правее j все >= pivot
        }
        std::swap(a[i], a[j]);
```

```
    }
}
```

> ⚠️ **Ловушка экзаменатора**
>
> **Опасная ошибка в рекурсии Хоара:** Студенты машинально пишут рекурсивный вызов как в Ломуто: `quickSort(a, l, p - 1)` и `quickSort(a, p + 1, r)`. Для схемы Хоара это приведёт к **пропуску элементов или бесконечному циклу**! В схеме Хоара pivot не изолируется. Правильный вызов:
>
> ```
> int p = hoarePartition(a, l, r);
> quickSort(a, l, p);       // Левая часть ВКЛЮЧАЕТ p
> quickSort(a, p + 1, r);   // Правая часть начинается с p + 1
> ```

---

2. Строгое доказательство среднего времени $\Theta(n \log n)$

На 1 и 2 потоке за это доказательство ставят высший балл. Оно опирается на **индикаторные случайные величины**.

Пусть $z_1 < z_2 < \ldots < z_n$ — элементы исходного массива в отсортированном порядке. Алгоритм сравнивает пары элементов. Обозначим индикатор:

$$X_{i,j} = \text{II}\big(z_i \text{ сравнивается с } z_j \text{ за всё время работы алгоритма}\big)$$

Тогда общее число сравнений C:

$$C = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{i,j}$$

По линейности математического ожидания:

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}\big[X_{i,j}\big] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{P}\big(z_i \text{ сравнивается с } z_j\big)$$

Когда элементы z_i и z_j вообще сравниваются?
Вспомним: элементы сравниваются **только с pivot**, друг с другом напрямую они не сравниваются. Рассмотрим множество чисел между ними:

$$Z_{i,j} = \{z_i, z_{i+1}, z_{i+2}, \ldots, z_j\}$$

Всего в этом множестве $j - i + 1$ элементов.

1. Если первым в качестве pivot выберется какой-то элемент из **середины** (между z_i и z_j), то он разделит z_i и z_j по разным подмассивам, и они **никогда не сравнятся**!
2. Элементы z_i и z_j сравнятся тогда и только тогда, когда **первым из множества $Z_{i,j}$ в качестве pivot будет выбран либо сам z_i, либо сам z_j**.
3. Так как любой из $j - i + 1$ элементов выбирается равновероятно:

$$\mathbb{P}\big(z_i \text{ сравнивается с } z_j\big) = \frac{2}{j - i + 1}$$

Считаем сумму:

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j - i + 1}$$

Сделаем замену переменной $k = j - i + 1$:

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} < 2 \sum_{i=1}^{n} \sum_{k=1}^{n} \frac{1}{k} = 2n \cdot H_n$$

где $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n} \approx \ln n + \gamma$ – гармоническое число. Получаем:

$$\mathbb{E}[C] < 2n \ln n = 2n \cdot \frac{\log_2 n}{\log_2 e} \approx 1.386 \cdot n \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Доказано строго и без оговорок!

---

3. Гарантия стека памяти $O(\log n)$ (Tail Call Optimization)

В наивной реализации стек рекурсии в худшем случае разворачивается на $O(n)$ уровней. При $n = 10^5$ это неминуемый `Stack Overflow`.

Как гарантировать, что глубина стека **никогда не превысит** $\lfloor \log_2 n \rfloor + 1$?

> **Правило:** Рекурсивный вызов делаем **строго в меньший подмассив**, а больший обрабатываем итеративно в цикле (устранение хвостовой рекурсии)!

```
void safeQuickSort(std::vector<int>& a, int l, int r) {
    while (l < r) {
        int p = hoarePartition(a, l, r);

        // Сравниваем размеры левой [l ... p] и правой [p+1 ... r] частей:
        if (p - l < r - (p + 1)) {
            // Левая часть МЕНЬШЕ -> идем в нее рекурсией
            safeQuickSort(a, l, p);
            // Большую правую часть сортируем в цикле, сдвигая левую границу:
            l = p + 1;
        } else {
            // Правая часть МЕНЬШЕ -> идем в нее рекурсией
            safeQuickSort(a, p + 1, r);
            // Большую левую часть сортируем в цикле, сдвигая правую границу:
            r = p;
        }
    }
}
```

Почему это гарантирует $O(\log n)$? Каждый раз, когда мы кладём фрейм в стек рекурсии, размер подзадачи уменьшается **как минимум в 2 раза** (ведь мы идём в меньшую половину, размер которой $\leq \frac{n}{2}$). Делить число на 2 можно не более $\log_2 n$ раз!

---

4. Трёхпутевое разбиение (3-Way Partition / Флаг Нидерландов)

Если в массиве много повторяющихся чисел (например, массив из нулей и единиц), классический QuickSort делает лишнюю работу. Эдсгер Дейкстра предложил разбивать массив не на две, а на **три зоны**:

```
[   < pivot   |   == pivot   |   не проверено   |   > pivot   ]
 l..........lt  lt........i     i..........gt   gt........r
```

- lt (less than): граница элементов, меньших pivot.
- gt (greater than): граница элементов, больших pivot.
- i: текущий проверяемый элемент.

```
void threeWayQuickSort(std::vector<int>& a, int l, int r) {
    if (l >= r) return;
    int pivot = a[l];
    int lt = l, gt = r, i = l + 1;

    while (i <= gt) {
        if (a[i] < pivot) {
            std::swap(a[lt++], a[i++]);
        } else if (a[i] > pivot) {
            std::swap(a[i], a[gt--]); // i не увеличиваем, так как новый a[i] еще не
проверен!
        } else {
            ++i;
        }
    }

    // Зона [lt ... gt] состоит из элементов, равных pivot.
    // ОНИ УЖЕ НА СВОИХ МЕСТАХ! Рекурсивно сортируем только хвосты:
    threeWayQuickSort(a, l, lt - 1);
    threeWayQuickSort(a, gt + 1, r);
}
```

- Если все элементы в массиве одинаковы, алгоритм расставит указатели за один проход и завершится за **строгие** $\Theta(n)$!

---

5. Доказательство неустойчивости

QuickSort **неустойчив (unstable)**. Минимальный контрпример, который нужно нарисовать на доске:

$$A = [2_a, 2_b, 1], \quad \text{pivot} = 1$$

1. Алгоритм разбиения (Хоара или Ломуто) видит, что 1 — самый маленький элемент, и меняет его местами с первым элементом 2_a.
2. Массив принимает вид: $[1, 2_b, 2_a]$.
3. Элемент 2_b оказался **левее** элемента 2_a, хотя изначально стоял правее! Относительный порядок нарушен.

Чек-лист характеристик QuickSort:

- **Время в лучшем случае:** $\Theta(n \log n)$ (сбалансированное деление пополам).
- **Время в среднем:** $\Theta(n \log n)$ (доказано через индикаторы).
- **Время в худшем случае:** $\Theta(n^2)$ (сильно несбалансированное деление 1 и $n-1$).
- **Память:** $O(\log n)$ (при элиминации хвостовой рекурсии).
- **In-place:** Да.
- **Устойчивость:** Нет.

Мы разобрали QuickSort от кишок разбиения до продвинутой оптимизации хвостовой рекурсии и 3-way partition. Готов двигаться к **Теме 6: QuickSelect (только для 1, 2 потока)**?

1. Код разбиения Ломуто наравне со схемой Хоара

Чтобы у тебя перед глазами было прямое сопоставление, вот канонический код обеих процедур:

```
// ========================================
// 1. СХЕМА ЛОМУТО (Lomuto Partition)
// ========================================
int lomutoPartition(std::vector<int>& a, int l, int r) {
```

```
    int pivot = a[r]; // Опорный — строго последний элемент
    int i = l - 1;    // Граница зоны чисел <= pivot

    for (int j = l; j < r; ++j) {
        if (a[j] <= pivot) {
            ++i;
            std::swap(a[i], a[j]); // Закидываем меньший элемент в левую зону
        }
    }
    // Ставим pivot ровно между зоной <= pivot и зоной > pivot:
    std::swap(a[i + 1], a[r]);
    return i + 1; // Возвращаем финальную позицию pivot!
}

// Рекурсивный вызов для Ломуто:
void quickSortLomuto(std::vector<int>& a, int l, int r) {
    if (l < r) {
        int p = lomutoPartition(a, l, r);
        quickSortLomuto(a, l, p - 1); // pivot исключен!
        quickSortLomuto(a, p + 1, r); // pivot исключен!
    }
}

// ========================================
// 2. СХЕМА ХОАРА (Hoare Partition)
// ========================================
int hoarePartition(std::vector<int>& a, int l, int r) {
    int pivot = a[l + (r - l) / 2]; // Опорный — из середины
    int i = l - 1;
    int j = r + 1;

    while (true) {
        do { ++i; } while (a[i] < pivot);
        do { --j; } while (a[j] > pivot);

        if (i >= j) return j; // Точка раздела (pivot не изолирован!)
        std::swap(a[i], a[j]);
    }
}

// Рекурсивный вызов для Хоара:
void quickSortHoare(std::vector<int>& a, int l, int r) {
    if (l < r) {
        int p = hoarePartition(a, l, r);
        quickSortHoare(a, l, p);     // p включен в левую часть!
        quickSortHoare(a, p + 1, r); // правая часть с p + 1
    }
}
```

2. Устойчивость QuickSort через массив индексов

По своей природе QuickSort **неустойчив**, так как элементы меняются через большие расстояния, перепрыгивая через равные себе ключи.

> **Вопрос экзаменатора:** «Можно ли сделать QuickSort устойчивым без переписывания логики самого алгоритма?» **Ответ:** Да! С помощью **стабилизации через первичные индексы** (Tag Sort / Decorate-Sort-Undecorate).

Идея «Тай-брейка по индексу»:
Если два элемента равны по значению ($A[i] == A[j]$), то в устойчивом массиве первым должен стоять тот, у которого **меньше исходный индекс**. Мы превращаем каждый элемент $A[i]$ в пару:

$$\text{Item}_i = (\text{значение } A[i], ; \text{исходный индекс } i)$$

Задаём строгий порядок сравнения для пар (лексикографический):

$$(\text{val}_1, \text{idx}_1) < (\text{val}_2, \text{idx}_2) \iff (\text{val}_1 < \text{val}_2) \vee (\text{val}_1 == \text{val}_2 \wedge \text{idx}_1 < \text{idx}_2)$$

Поскольку исходные индексы $0, 1, ..., n-1$ **все уникальны**, в массиве пар **вообще не существует одинаковых элементов**! А если нет равных элементов, то алгоритм физически не может нарушить их относительный порядок.

Реализация через массив индексов (на C++):

```cpp
void stableQuickSort(std::vector<int>& a) {
    int n = a.size();
    // 1. Создаем массив индексов: [0, 1, 2, ..., n-1]
    std::vector<int> p(n);
    for (int i = 0; i < n; ++i) p[i] = i;

    // 2. Компаратор с тай-брейком по исходному индексу
    auto cmp = [&](int i, int j) {
        if (a[i] != a[j]) return a[i] < a[j];
        return i < j; // При равенстве значений выигрывает тот, кто стоял левее!
    };

    // 3. Быстрая сортировка массива индексов p с использованием компаратора cmp
    // (любая стандартная реализация quicksort для массива p)
    quickSortIndices(p, 0, n - 1, cmp);

    // 4. Восстанавливаем элементы исходного массива по отсортированным индексам
    std::vector<int> sorted_a(n);
    for (int i = 0; i < n; ++i) {
        sorted_a[i] = a[p[i]];
    }
    a = sorted_a;
}
```

- **Цена устойчивости:** требуется $O(n)$ дополнительной памяти (под массив индексов p и копию). Зато исходный алгоритм QuickSort гарантирует строгую устойчивость!

Тема 6. Порядковые статистики и QuickSelect

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Задача выбора k-й порядковой статистики требует найти элемент, который стоял бы на позиции k после полной сортировки массива (например, медиану при $k = \lfloor \frac{n}{2} \rfloor$). Алгоритм **QuickSelect** использует идею `Partition` из быстрой сортировки, но спускается **только в один подмассив**, содержащий целевой индекс. В среднем QuickSelect работает за $\boldsymbol{\Theta(n)}$ и требует $O(1)$ памяти в итеративной версии. В худшем случае время составляет $\Theta(n^2)$. Для устранения худшего случая применяется алгоритм **BFPRT (медианы медиан)**, который гарантирует выбор опорного элемента за линейное время и обеспечивает сложность **строго $\boldsymbol{\Theta(n)}$ в худшем случае** благодаря рекуррентности $T(n) \le T(\frac{n}{5}) + T(7\frac{n}{10}) + O(n)$».

1. Постановка задачи

:::meaning **k-я порядковая статистика массива A размера n ($k \in [0, n-1]$):** Это элемент, который оказался бы в ячейке $A[k]$, если бы массив A был отсортирован по неубыванию.

- $k = 0$ — минимум массива (находится тривиально за $n-1$ сравнений).
- $k = n-1$ — максимум массива (за $n-1$ сравнений).
- $k = \lfloor \frac{n-1}{2} \rfloor$ — **медиана**.

:::

Наивный подход: отсортировать массив за $O(n \log n)$ (`std::sort`) и взять $A[k]$. **Вопрос:** можно ли найти медиану за линейное время $O(n)$ без полной сортировки? **Ответ:** да, алгоритмом QuickSelect.

2. Алгоритм Хоара: QuickSelect

Идея «срезания ветвей»

Вспомним QuickSort: мы запускаем `Partition(A, l, r)` , и опорный элемент встаёт на индекс p:

- Все элементы левее p меньше или равны ему.
- Все элементы правее p больше или равны ему.

В QuickSort мы рекурсивно сортировали **обе** половины: и левую, и правую. Но нам **не нужно сортировать весь массив**! Нам нужен только индекс k:

1. Если $k == p$ — мы мгновенно нашли ответ, это $A[p]$!
2. Если $k < p$ — целевой элемент гарантированно находится в левой половине. **Правую половину мы просто выбрасываем!**
3. Если $k > p$ — целевой элемент гарантированно в правой половине. **Левую половину выбрасываем!**

Итеративная реализация на C++ (память $O(1)$)

Вместо рекурсии пишется элементарный цикл `while` , который не тратит память на стек вызовов:

```
int quickSelect(std::vector<int>& a, int k) {
    int l = 0, r = a.size() - 1;

    while (l < r) {
```

```
        // Делаем partition (например, по схеме Ломуто или Хоара)
        int p = lomutoPartition(a, l, r);

        if (p == k) {
            return a[p]; // Нашли ровно k-й элемент
        } else if (k < p) {
            r = p - 1;   // Отбрасываем правую часть
        } else {
            l = p + 1;   // Отбрасываем левую часть
        }
    }
    return a[l];
}
```

3. Математика сложности QuickSelect

Среднее время: $\Theta(n)$
Предположим, что опорный элемент делит массив хотя бы приблизительно в постоянной пропорции (в среднем — пополам). Тогда на первом шаге мы просматриваем n элементов, на втором — $\frac{n}{2}$, затем $\frac{n}{4}$ и так далее:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \ldots = n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = n \cdot \frac{1}{1 - \frac{1}{2}} = \boldsymbol{2n} = \boldsymbol{\Theta(n)}.$$

Худшее время: $\Theta(n^2)$
Если каждый раз выбирать в качестве pivot минимальный (или максимальный) элемент, то подзадача уменьшается всего на 1:

$$T(n) = n + (n-1) + (n-2) + \ldots + 1 = \frac{n(n+1)}{2} = \boldsymbol{\Theta(n^2)}.$$

4. Детерминированный выбор за $O(n)$: Алгоритм BFPRT (Median of Medians)

Я переработал визуализацию: сделал **интерактивную цветовую карту** матрицы с кнопками переключения зон прямо в чате, а также добавил наглядную цветную схему в тексте.

🎨 Интерактивная карта матрицы $5 \times \left(\frac{n}{5}\right)$

В виджете ниже можно переключать режимы и смотреть, как геометрия отсекает элементы:

Цветная схема матрицы (разбор на пальцах)

Представь, что мы выстроили все элементы в таблицу из 5 строк и $\frac{n}{5}$ столбцов.

- Каждый столбец упорядочен **сверху вниз** (вверху маленькие, внизу большие). Медиана каждого столбца лежит **ровно в 3-й строке**.
- Столбцы отсортированы **слева направо** по значению своей медианы.
- Центральный элемент 3-й строки — это M (медиана среди всех медиан столбцов).

Вот как выглядит эта таблица с цветовым зонированием:

Строка	Столбцы левее M (≈ $\frac{n}{10}$ шт.)	Столбец M	Столбцы правее M (≈ $\frac{n}{10}$ шт.)

Стр. 1	🟥 $\leq M$	🟥 $\leq M$	⬜ **? (неизвестно)**
Стр. 2	🟥 $\leq M$	🟥 $\leq M$	⬜ **? (неизвестно)**
Стр. 3 (медианы)	🟥 $\leq M$ (медианы левых)	🟨 $[M]$ **(pivot)**	🟩 $\geq M$ (медианы правых)
Стр. 4	⬜ **? (неизвестно)**	🟩 $\geq M$	🟩 $\geq M$
Стр. 5	⬜ **? (неизвестно)**	🟩 $\geq M$	🟩 $\geq M$

1. Почему зелёный угол (справа снизу) гарантированно $\geq M$?

Посмотрим на правые столбцы:

1. В **3-й строке** стоят медианы этих столбцов. Так как они правее M, по определению M они $\geq M$.
2. В **4-й и 5-й строках** лежат элементы, которые находятся **ниже** своей медианы в том же столбце. Значит, они ещё больше своей медианы:

$$\text{Строка } 5 \geq \text{Строка } 4 \geq \text{Строка } 3 \text{ (медиана)} \geq M.$$

3. Значит, в каждом из этих столбцов **все 3 нижних элемента** гарантированно $\geq M$!
4. Сколько таких столбцов? Ровно половина от $\frac{n}{5}$, то есть $\frac{n}{10}$ столбцов.
5. Сколько всего зелёных элементов?

$$3 \text{ строки} \times \frac{n}{10} \text{ столбцов} = \mathbf{\frac{3n}{10}} \quad (\mathbf{30\%} \text{ массива}).$$

2. Почему красный угол (слева сверху) гарантированно $\leq M$?

Абсолютно симметрично:

1. В **3-й строке** левые медианы $\leq M$.
2. В **1-й и 2-й строках** лежат элементы, которые **выше** своей медианы, то есть ещё меньше неё.
3. Получаем 3 верхних элемента в каждом из $\frac{n}{10}$ левых столбцов:

$$3 \text{ строки} \times \frac{n}{10} \text{ столбцов} = \mathbf{\frac{3n}{10}} \quad (\mathbf{30\%} \text{ массива}).$$

3. Почему серые элементы (⬜) неизвестны и почему нам на них всё равно?

- **Справа сверху:** элемент меньше медианы своего столбца, но сама медиана $> M$. Число может оказаться как 1, так и 1000.
- **Слева снизу:** элемент больше медианы своего столбца, но медиана $< M$.
- **Но нам плевать на серые клетки!** Нам не нужно знать всё. Главное, что мы уже **железно отсекли 30%** с одной стороны и **30%** с другой стороны!

4. Строгое доказательство линейности $O(n)$ с точными константами

Преподаватель на коллоквиуме может спросить: **«А что, если n не делится нацело на 5 и есть неполная группа?»**

Учёт граничных условий:

1. Число групп: $\lceil \frac{n}{5} \rceil$.
2. Число групп с медианой $\geq M$: не менее $\frac{1}{2}\lceil \frac{n}{5} \rceil$.
3. Отбросим группу с самим M и последнюю группу (она может быть неполной, например из 1–2 элементов): Остаётся не менее $\left(\frac{1}{2}\lceil \frac{n}{5} \rceil - 2\right)$ полных групп.
4. В каждой полной группе гарантированно 3 элемента $\geq M$:

$$\text{Количество элементов} \geq M \geq 3 \cdot \left(\frac{1}{2} \cdot \frac{n}{5} - 2\right) = \mathbf{\frac{3n}{10} - 6}.$$

5. Значит, в худшем случае подзадача для рекурсивного спуска содержит не более:

$$n - \left(\frac{3n}{10} - 6\right) = \boldsymbol{\frac{7n}{10} + 6} \text{ элементов.}$$

Доказательство по индукции:
Получаем рекуррентность:

$$T(n) \le T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 6\right) + c \cdot n$$

Докажем гипотезу $\boldsymbol{T(n) \le C \cdot n}$:

1. Подставляем гипотезу:

$$T(n) \le C \cdot \frac{n}{5} + C \cdot \left(\frac{7n}{10} + 6\right) + c \cdot n$$

2. Складываем дроби с n:

$$T(n) \le C \cdot \left(\frac{2}{10} + \frac{7}{10}\right)n + 6C + \mathrm{cn} = \frac{9}{10}Cn + 6C + \mathrm{cn}$$

3. Выделим Cn:

$$T(n) \le Cn - \left(\frac{Cn}{10} - 6C - \mathrm{cn}\right)$$

4. Чтобы $T(n) \le \mathrm{Cn}$, скобка обязана быть неотрицательной:

$$\frac{Cn}{10} - 6C - \mathrm{cn} \ge 0 \iff n\left(\frac{C}{10} - c\right) \ge 6C$$

5. Выберем константу C так, чтобы $\frac{C}{10} - c \ge \frac{C}{20} \iff \boldsymbol{C \ge 20c}$. Тогда неравенство превращается в:

$$n \cdot \frac{C}{20} \ge 6C \iff n \ge 120.$$

6. **База индукции:** для всех $n < 120$ массив тривиально сортируется вставками за $O(1)$, поэтому константу C легко выбрать достаточно большой, чтобы покрыть базу.

Индукционный переход полностью сошёлся: $\boldsymbol{T(n) = \Theta(n)}$.

---

5. Строгое доказательство нелинейности для групп из 3 элементов

Почему деление на тройки **не даёт** $\boldsymbol{O(n)}$?

1. Разобьём массив на группы по **3 элемента** ($\frac{n}{3}$ групп).
2. Медиана в каждой тройке — центральный (2-й) элемент.
3. Половина групп ($\frac{1}{2} \cdot \frac{n}{3} = \frac{n}{6}$) имеет медиану $\ge M$.
4. В каждой такой группе только **2 элемента** (медиана и нижний) гарантированно $\ge M$.
5. Сколько элементов отсекается?

$$2 \text{ элемента} \times \frac{n}{6} \text{ групп} = \boldsymbol{\frac{n}{3}} \text{ элементов.}$$

6. Сколько элементов остаётся в худшем случае?

$$n - \frac{n}{3} = \boldsymbol{\frac{2n}{3}} \text{ элементов.}$$

7. Получаем рекуррентное уравнение:

$$T(n) \le T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + c \cdot n.$$

Анализ через дерево рекурсии:
Посмотрим на суммарную работу по ярусам дерева:

- **Ярус 0:** $c \cdot n$.
- **Ярус 1:** $c \cdot \frac{n}{3} + c \cdot \frac{2n}{3} = c \cdot \left(\frac{1}{3} + \frac{2}{3}\right)n = \boldsymbol{c \cdot n}$.
- **Ярус 2:** $c \cdot \left(\frac{n}{9} + \frac{2n}{9} + \frac{2n}{9} + \frac{4n}{9}\right) = c \cdot \left(\frac{1}{9} + \frac{4}{9} + \frac{4}{9}\right)n = \boldsymbol{c \cdot n}$.

Сумма долей $\frac{1}{3} + \frac{2}{3} = \mathbf{1.0}$. Это значит, что **на КАЖДОМ уровне дерева суммарная работа в точности равна $c \cdot n$!**
А глубина дерева рекурсии равна как минимум $\log_3 n$.

Суммируем работу по всем уровням:

$$T(n) \geq \sum_{j=0}^{\log_3 n} c \cdot n = c \cdot n \cdot \log_3 n = \boldsymbol{\Omega(n \log n)}.$$

Итоговый вердикт:

- **Для пятёрок:** $\frac{1}{5} + \frac{7}{10} = \mathbf{0.9} < \mathbf{1} \Longrightarrow$ геометрическая прогрессия убывает $\Longrightarrow \boldsymbol{O(n)}$.
- **Для троек:** $\frac{1}{3} + \frac{2}{3} = \mathbf{1.0} \Longrightarrow$ работа на уровнях не убывает $\Longrightarrow \boldsymbol{\Theta(n \log n)}$.

Стала ли теперь картинка и математика полностью прозрачной?

Псевдокод алгоритма BFPRT на C++

```cpp
int bfprt(std::vector<int>& a, int l, int r, int k) {
    if (r - l + 1 <= 5) {
        std::sort(a.begin() + l, a.begin() + r + 1);
        return a[l + k];
    }

    // 1. Разбиваем на пятерки и ищем медианы
    int num_medians = 0;
    for (int i = l; i <= r; i += 5) {
        int sub_r = std::min(i + 4, r);
        std::sort(a.begin() + i, a.begin() + sub_r + 1);
        int median_idx = i + (sub_r - i) / 2;
        std::swap(a[l + num_medians], a[median_idx]);
        num_medians++;
    }

    // 2. Рекурсивно находим медиану медиан
    int pivot = bfprt(a, l, l + num_medians - 1, num_medians / 2);

    // 3. 3-Way Partition относительно pivot
    int lt = l, gt = r, i = l;
    while (i <= gt) {
        if (a[i] < pivot) std::swap(a[lt++], a[i++]);
        else if (a[i] > pivot) std::swap(a[i], a[gt--]);
        else i++;
    }

    // 4. Рекурсивный спуск ровно в одну подзадачу
    int count_less = lt - l;
    int count_equal = gt - lt + 1;

    if (k < count_less) return bfprt(a, l, lt - 1, k);
    if (k < count_less + count_equal) return pivot;
    return bfprt(a, gt + 1, r, k - count_less - count_equal);
}
```

5. Что используется на практике: `IntroSelect` (`std::nth_element`)

В реальном коде C++ алгоритм `std::nth_element` устроен так:

- Запускается обычный быстрый **QuickSelect**.
- Алгоритм ведёт счётчик глубины рекурсии.
- Если глубина рекурсии превышает $2\log_2 n$ (то есть алгоритм заподозрил худший случай из-за плохих pivot), он переключается на **детерминированный алгоритм (HeapSelect или BFPRT)**.

Это даёт фантастическую скорость на средних данных и математическую защиту от худшего случая $O(n^2)$.

Сводка для коллоквиума:

Алгоритм	Лучшее время	Среднее время	Худшее время	Память
QuickSelect (Рандомизированный)	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$O(1)$ итеративно
BFPRT (Медиана медиан)	$\Theta(n)$	$\Theta(n)$	$\boldsymbol{\Theta(n)}$	$O(\log n)$ стек
IntroSelect (`std::nth_element`)	$\Theta(n)$	$\Theta(n)$	$\boldsymbol{\Theta(n)}$	$O(\log n)$ стек

Тема закрыта на максимум. Переходим к **Теме 7: Сортировка подсчётом (Counting Sort)**?

Тема 7. Сортировка подсчётом (Counting Sort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Сортировка подсчётом — это не сравнительный (non-comparison based) алгоритм, сортирующий целые числа из известного ограниченного диапазона $[0, k-1]$ за время $\Theta(n+k)$. Алгоритм подсчитывает частоты элементов, строит массив префиксных сумм (указывающих позиции последних вхождений каждого ключа) и раскладывает элементы в выходной массив. Алгоритм **строго устойчив (stable)** благодаря обходу исходного массива строго справа налево. Дополнительная память — $\Theta(n+k)$. Алгоритм не является in-place».

1. Как обойти теорему о $\Omega(n \log n)$?

> **Любимый вопрос экзаменатора:** «Теорема гласит, что сортировать быстрее $\Omega(n \log n)$ невозможно. Почему Counting Sort работает за линейное время? Теорема неверна?»

Ответ: Теорема абсолютно верна, но она сформулирована **только для сортировок сравнениями** (где мы можем делать только $A[i] < A[j]$). Counting Sort **вообще не сравнивает числа друг с другом**! Он использует само числовое значение ключа как **прямой адрес (индекс)** в оперативной памяти компьютера. Это принципиально другая модель вычислений.

Ограничение: Чтобы алгоритм работал эффективно, диапазон ключей k обязан быть сопоставим с размером массива ($k = O(n)$). Если на входе 10 чисел, но они лежат в диапазоне от 0 до 10^{12}, Counting Sort потребует терабайты памяти и зависнет.

2. Пошаговый алгоритм на пальцах

Алгоритм состоит из трёх чётких фаз:

Фаза 1. Подсчёт частот

Заводим массив счётчиков `count` размера k, заполненный нулями. Бежим по массиву A и для каждого числа x делаем:

```
count[x]++;
```

Теперь `count[x]` хранит, сколько раз число x встретилось во входе. **(Время: $\Theta(n)$, память: $\Theta(k)$).**

Фаза 2. Префиксные суммы (накопление)

Превращаем массив частот в массив префиксных сумм:

```
for (int i = 1; i < k; ++i) {
    count[i] += count[i - 1];
}
```

В чём физический смысл `count[x]` теперь? Теперь `count[x]` показывает, **сколько всего элементов массива $\leq x$**. Это означает, что последний элемент со значением x в отсортированном массиве должен занять позицию `count[x] - 1`!

Фаза 3. Раскладка в выходной массив (СТРОГО СПРАВА НАЛЕВО!)
Заводим выходной массив `out` размера n. Идём по исходному массиву A **с конца в начало** ($i = n-1 \ldots 0$):

```
int x = a[i];
out[--count[x]] = x;
```

Мы декрементируем позицию `--count[x]` и кладём x на освободившееся место.

---

3. Наглядный числовой пример трассировки

Пусть $n = 6$, диапазон чисел от 0 до 3 ($k = 4$):

$$A = [2_a, 1_a, 3, 1_b, 0, 2_b]$$

(Индексы a и b приписаны одинаковым числам, чтобы отследить устойчивость).

Шаг 1. Массив частот:
- `count[0] = 1` (один ноль)
- `count[1] = 2` (две единицы)
- `count[2] = 2` (две двойки)
- `count[3] = 1` (одна тройка)

$$\text{count} = [1, 2, 2, 1]$$

Шаг 2. Префиксные суммы:
- `count[0] = 1`
- `count[1] = 1 + 2 = 3` (чисел ≤ 1 ровно 3 штуки: они займут позиции 0, 1, 2)
- `count[2] = 3 + 2 = 5` (чисел ≤ 2 ровно 5 штук: позиции 0..4)
- `count[3] = 5 + 1 = 6`

$$\text{count} = [1, 3, 5, 6]$$

Шаг 3. Обратный проход по A:
- Смотрим последний элемент: $A[5] = 2_b$. `--count[2]` становится равным 4. Кладём $\text{out}[4] = 2_b$.
- Смотрим $A[4] = 0$. `--count[0]` становится равным 0. Кладём $\text{out}[0] = 0$.
- Смотрим $A[3] = 1_b$. `--count[1]` становится равным 2. Кладём $\text{out}[2] = 1_b$.
- Смотрим $A[2] = 3$. `--count[3]` становится равным 5. Кладём $\text{out}[5] = 3$.
- Смотрим $A[1] = 1_a$. `--count[1]` становится равным 1. Кладём $\text{out}[1] = 1_a$.
- Смотрим $A[0] = 2_a$. `--count[2]` становится равным 3. Кладём $\text{out}[3] = 2_a$.

Итоговый массив:

$$\text{out} = [0, 1_a, 1_b, 2_a, 2_b, 3]$$

Обрати внимание: 1_a оказалась левее 1_b, а 2_a левее 2_b! **Порядок сохранился идеально!**

---

4. Эталонный код на C++

```
void countingSort(std::vector<int>& a, int k) {
    int n = a.size();
    if (n <= 1) return;

    std::vector<int> count(k, 0);
    std::vector<int> out(n);

    // 1. Считаем частоты
    for (int x : a) {
        count[x]++;
```

```
    }

    // 2. Префиксные суммы
    for (int i = 1; i < k; ++i) {
        count[i] += count[i - 1];
    }

    // 3. Расставляем элементы: СТРОГО СПРАВА НАЛЕВО!
    for (int i = n - 1; i >= 0; --i) {
        int x = a[i];
        out[--count[x]] = x;
    }

    // Копируем отсортированный результат обратно
    a = std::move(out);
}
```

5. Главные ловушки экзаменаторов по Counting Sort

⚠️ Ловушка 1: Почему именно справа налево? Что будет, если идти слева направо?

- **Если идти слева направо:** алгоритм по-прежнему отсортирует массив! Все нули будут слева, все единицы посередине.
- **НО ОН ПОТЕРЯЕТ УСТОЙЧИВОСТЬ:** Элемент 1_a, стоявший левее, запишется в ячейку с бóльшим индексом, а 1_b — левее него. Относительный порядок дубликатов **инвертируется**.
- **Почему это критично:** в следующей теме (Radix Sort) потеря устойчивости ломает всё на корню.

⚠️ Ловушка 2: Упрощённый подсчёт (без массива префиксных сумм)

Иногда студенты пишут такой код:

```
// "Упрощенный" код без префиксных сумм:
int idx = 0;
for (int val = 0; val < k; ++val) {
    while (count[val] > 0) {
        a[idx++] = val;
        count[val]--;
    }
}
```

Экзаменатор спросит: **«Зачем городить префиксные суммы и выделять массив `out`, если можно просто переписать массив вот такими двумя циклами?»**

Ответ, который ждут на «отлично»: Этот упрощённый код работает, **только если массив состоит из голых чисел**! В реальных приложениях элементы — это сложные структуры (например, записи `Student { int grade; string name; }`). Если мы просто перезапишем оценки `5, 5, 4` числами, мы **сотрём имена студентов**! Полноценный алгоритм Counting Sort с префиксными суммами перемещает **сами исходные объекты целиком** со всеми их дополнительными полями (спутниковыми данными).

⚠️ Ловушка 3: Отрицательные числа

Если в массиве есть отрицательные числа (например, от -50 до $+50$), обращаться по отрицательному индексу `count[-5]` в C++ нельзя. **Решение:** находим минимум $\min$ за $O(n)$, размер диапазона делаем $k = \max - \min + 1$, а все обращения индексируем со сдвигом:

$$\text{индекс в } \mathrm{count} = x - \min\,.$$

Чек-лист свойств Counting Sort:

- **Время работы:** строго $\boldsymbol{\Theta(n+k)}$ во всех случаях (лучший, средний, худший).
- **Память:** $\boldsymbol{\Theta(n+k)}$ (массив частот k + выходной массив n).
- **In-place: Нет** (требует массив `out`).
- **Устойчивость: Да** (при обходе $i = n-1 \ldots 0$).
- **Где применяется:** внутри поразрядной сортировки (Radix Sort) и для сортировки символов строк (ASCII: $k = 256$).

Всё ли кристально понятно по подсчёту и префиксным суммам? Если да — делаем логический шаг к **Теме 8: Цифровая поразрядная сортировка (Radix Sort: LSD и MSD)**!

Тема 8. Цифровая сортировка (Radix Sort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Поразрядная сортировка упорядочивает многозначные числа или строки фиксированной длины, рассматривая их как последовательность из d цифр по основанию b. Существует два направления:
> 1. **LSD (Least Significant Digit):** от младших разрядов к старшим. Работает строго итеративно и критически опирается на **устойчивость (stability)** внутреннего сортировщика (Counting Sort).
> 2. **MSD (Most Significant Digit):** от старших к младшим. Работает рекурсивно по принципу Divide & Conquer и идеален для строк переменной длины.
>
> Время работы — $\Theta(d \cdot (n + b))$, дополнительная память — $\Theta(n + b)$. При выборе основания $b = 2^r \approx n$ алгоритм работает за **линейное время** $\Theta(n)$».

1. LSD Radix Sort (От младших разрядов к старшим)

Почему сортировка с конца вообще работает? (Главная интуиция)

Новичкам кажется диким: **«Как можно сначала отсортировать числа по единицам, потом по десяткам, потом по сотням — и получить правильный ответ? Разве сортировка по сотням не снесёт напрочь порядок единиц?»**

Вся фишка — в **устойчивости (Stability)** Counting Sort!

Давай посмотрим на трёхзначные числа:

$$A = [170, 045, 075, 090, 002, 024, 802, 066]$$

Шаг 1. Сортируем по младшему разряду (единицам):
Смотрим только на последние цифры:

- $17\mathbf{0}, 09\mathbf{0} \to$ кончаются на 0.
- $00\mathbf{2}, 80\mathbf{2} \to$ кончаются на 2 (заметь: 002 встанет перед 802, сохранив исходный порядок!).
- $02\mathbf{4} \to$ на 4.
- $04\mathbf{5}, 07\mathbf{5} \to$ на 5.
- $06\mathbf{6} \to$ на 6.

Массив после 1-го прохода:

$$[170, 090, 002, 802, 024, 045, 075, 066]$$

Шаг 2. Сортируем по десяткам (СТАБИЛЬНЫМ Counting Sort):
Смотрим на вторую цифру:

- $\mathbf{0}02, \mathbf{0}24 \to$ десятки 0 и 2: $[002, 802, 024]$
- $\mathbf{4}5 \to [045]$
- $\mathbf{6}6 \to [066]$
- $\mathbf{7}0, \mathbf{7}5 \to [170, 075]$ (заметь: 170 было раньше 075, и благодаря стабильности 170 остаётся раньше 075!).
- $\mathbf{9}0 \to [090]$

Массив после 2-го прохода:

$$[002, 802, 024, 045, 066, 170, 075, 090]$$

(Сейчас числа уже идеально упорядочены по двум младшим разрядам: $02 < 24 < 45 < 66 < 70 < 75 < 90$).

Шаг 3. Сортируем по старшему разряду (сотням):
- Сотни 0: $[002, 024, 045, 066, 075, 090]$
- Сотни 1: $[170]$
- Сотни 8: $[802]$

Массив стал **строго отсортирован**:

$$[002,; 024,; 045,; 066,; 075,; 090,; 170,; 802]$$

Строгое доказательство корректности LSD по индукции

На экзамене за это доказательство ставят высший балл:

> **База индукции:** После 1-го прохода массив отсортирован по разряду 0 (младшему). Это очевидно, так как мы запустили сортировку по этому разряду.
>
> **Индукционный переход:** Предположим, что массив уже отсортирован по суффиксу из первых i разрядов (от 0 до $i-1$). Запускаем устойчивую сортировку по разряду i. Рассмотрим два произвольных числа X и Y:
> 1. **Если цифры в разряде i различны ($X_i \neq Y_i$):** Их взаимный порядок будет определён разрядом i. То число, у которого цифра i меньше, встанет левее. Это корректно, так как разряд i старше всех разрядов $0 \ldots i-1$.
> 2. **Если цифры в разряде i одинаковы ($X_i == Y_i$):** Поскольку внутренняя сортировка **устойчива (stable)**, взаимный порядок чисел X и Y **не изменится**! Но по предположению индукции они уже стояли в правильном порядке по младшим разрядам $0 \ldots i-1$. Значит, их правильный относительный порядок сохранится!
>
> **Итог:** После прохода i массив корректно отсортирован по всем разрядам от 0 до i. По завершении d проходов весь массив отсортирован. $\square.\text{filled}$

---

2. Полный глубокий разбор MSD Radix Sort (Most Significant Digit)

Давай разберём **MSD (Most Significant Digit) Radix Sort** во всех деталях.

Если LSD интуитивно кажется «странным» (потому что мы сортируем с конца), то MSD, напротив, максимально близок к человеческой логике. Именно по принципу MSD устроен **любой алфавитный словарь, картотека в библиотеке или телефонный справочник**.

Поразрядная сортировка по старшим разрядам (MSD Radix Sort)

Главное концептуальное отличие от LSD:

- **В LSD** мы делали d **глобальных проходов по всему массиву** от младших разрядов к старшим. Массив никогда не дробился на части.
- **В MSD** мы смотрим на **самый старший разряд**, делим массив на независимые корзины (бакеты), а затем **рекурсивно** сортируем каждую корзину отдельно по следующему разряду. Это чистая парадигма **«Разделяй и властвуй» (Divide and Conquer)**.

```
Исходный массив
      |
[Разбиение по 1-му символу]
   /        |        \
Корзина 'a' Корзина 'b' Корзина 'c'
   |        |        |
[Рекурсия] [Рекурсия] [Рекурсия] (по 2-му символу внутри каждой корзины!)
```

---

1. Обработка строк переменной длины (Sentinel `-1`)

Главная сила MSD — сортировка **строк разной длины** (например, слов `["a", "cat", "car", "at"]`).

> **Проблема:** что делать, если в строке закончились символы, а мы проверяем разряд d? Например, сравниваем `"a"` и `"at"` на шаге $d = 1$. У слова `"at"` второй символ `'t'`, а у слова `"a"` второго символа вообще нет!

Решение: Символ конца строки (Sentinel)

Мы пишем вспомогательную функцию `charAt(string, d)`:

- Если индекс d меньше длины строки, возвращаем код символа (например, от 0 до 255).
- Если строка закончилась ($d \geq \text{length}$), возвращаем $\mathbf{-1}$.

```cpp
int charAt(const std::string& s, int d) {
    if (d < s.length()) {
        return static_cast<unsigned char>(s[d]);
    }
    return -1; // Признак того, что строка закончилась
}
```

Так как -1 меньше любого существующего символа, короткие строки (например, `"a"`) гарантированно упадут в **самую первую корзину** и встанут раньше более длинных слов (`"at"`, `"auto"`).

---

2. Пошаговая механика работы алгоритма

Пусть мы сортируем подмассив $A[l \ldots r]$ по символу с индексом d.

Базовые случаи рекурсии:

1. **Маленький подмассив ($r \leq l$):** если в корзине 0 или 1 элемент — сортировать нечего, сразу выходим.
2. **Отсечение по размеру ($r - l + 1 \leq \text{CUTOFF}$):** на маленьких корзинах (обычно ≤ 16 элементов) накладные расходы на создание массивов счётчиков огромны. Алгоритм переключается на **обычную сортировку вставками (Insertion Sort)**.

Основные 4 шага внутри рекурсии:

1. **Подсчёт частот:** Создаём массив `count` размера $R + 2$ (где R — размер алфавита, например 256, плюс ячейка под -1, плюс сдвиг для префиксных сумм). Считаем, сколько строк имеют каждую букву в позиции d.
2. **Префиксные суммы:** Превращаем частоты в границы диапазонов для каждой буквы.
3. **Раскладка в буфер:** Раскладываем строки во временный буфер по вычисленным корзинам и копируем обратно в $A[l...r]$.
4. **Рекурсивный спуск:**
   - Строки, которые закончились (буква -1), уже стоят на своих финальных местах! Их трогать не нужно.
   - Для каждой буквы c от 0 до $R - 1$ мы рекурсивно вызываем MSD для соответствующего диапазона корзины, но уже **для следующего символа** $d + 1$:

```
msdSort(a, bucket_left, bucket_right, d + 1, buf);
```

---

3. Подробная трассировка на живом примере

Пусть дан массив слов:

$$A = [\text{cat},; \text{car},; \text{cab},; \text{bar},; \text{bat},; \text{a}]$$

Глубина 0 ($d = 0$, смотрим на 1-ю букву):
Смотрим на первые буквы:

- На `'a'` : `["a"]`
- На `'b'` : `["bar", "bat"]`
- На `'c'` : `["cat", "car", "cab"]`

Массив разбился на три корзины:

$$[;\text{a};|;\text{bar},;\text{bat};|;\text{cat},;\text{car},;\text{cab};]$$

- Корзина `'a'` состоит из 1 элемента. Рекурсия для неё завершена.
- Теперь **независимо** запускаем рекурсию для корзины `'b'` и корзины `'c'`.

Глубина 1 ($d = 1$, смотрим на 2-ю букву):

Внутри корзины `'b'` ($A[1...2]$):

- `charAt("bar", 1) = 'a'`
- `charAt("bat", 1) = 'a'`

Оба слова имеют вторую букву `'a'`. Они попадают в одну корзину.

Внутри корзины `'c'` ($A[3...5]$):

- `charAt("cat", 1) = 'a'`
- `charAt("car", 1) = 'a'`
- `charAt("cab", 1) = 'a'`

Все три слова имеют вторую букву `'a'`. Попадают в одну корзину.

Глубина 2 ($d = 2$, смотрим на 3-ю букву):

Для корзины `'b'` :
Смотрим на 3-ю букву:

- `"bar"` → буква `'r'`
- `"bat"` → буква `'t'`

Так как `'r' < 't'`, корзина упорядочивается: `["bar", "bat"]`. Финал!

Для корзины `'c'`:
Смотрим на 3-ю букву:

- `"cat"` → `'t'`
- `"car"` → `'r'`
- `"cab"` → `'b'`

Буквы распределяются: `'b' < 'r' < 't'`. Корзина перестраивается: `["cab", "car", "cat"]`. Финал!

Итоговый массив:

$$[\text{a},; \text{bar},; \text{bat},; \text{cab},; \text{car},; \text{cat}]$$

---

4. Эталонный C++ код с оптимизацией Insertion Sort

```cpp
#include <vector>
#include <string>
#include <algorithm>

const int R = 256;       // Размер алфавита (ASCII)
const int CUTOFF = 15;   // Порог отсечения на сортировку вставками

// Безопасное извлечение символа
int charAt(const std::string& s, int d) {
    if (d < s.length()) return static_cast<unsigned char>(s[d]);
    return -1;
}

// Сортировка вставками для маленьких корзин
void insertionSortMSD(std::vector<std::string>& a, int l, int r, int d) {
    for (int i = l + 1; i <= r; ++i) {
        for (int j = i; j > l; --j) {
            // Сравниваем суффиксы строк, начиная с позиции d
            if (a[j].substr(d) < a[j - 1].substr(d)) {
                std::swap(a[j], a[j - 1]);
            } else {
                break;
            }
        }
    }
}

void msdSort(std::vector<std::string>& a, int l, int r, int d, std::vector<std::string>&
buf) {
    // 1. Базовый случай: маленькие корзины сортируем вставками
    if (r - l + 1 <= CUTOFF) {
        insertionSortMSD(a, l, r, d);
        return;
    }

    // count имеет размер R + 2:
    // count[0] — для символа -1 (конец строки)
    // count[1 ... R] — для символов 0 ... R - 1
    std::vector<int> count(R + 2, 0);

    // 2. Подсчет частот символов в позиции d
    for (int i = l; i <= r; ++i) {
        int c = charAt(a[i], d);
        count[c + 2]++;
```

```
    }

    // 3. Преобразование частот в префиксные суммы (смещение корзин)
    for (int r_idx = 0; r_idx < R + 1; ++r_idx) {
        count[r_idx + 1] += count[r_idx];
    }

    // 4. Распределение строк по корзинам в буфер
    for (int i = l; i <= r; ++i) {
        int c = charAt(a[i], d);
        buf[l + count[c + 1]++] = a[i];
    }

    // 5. Копируем обратно в исходный массив
    for (int i = l; i <= r; ++i) {
        a[i] = buf[i];
    }

    // 6. Рекурсивные вызовы для каждой корзины по следующему символу (d + 1)
    // Корзину 0 (где c == -1, т.е. строки завершились) не трогаем, они уже отсортированы!
    for (int r_idx = 0; r_idx < R; ++r_idx) {
        int bucket_l = l + count[r_idx];
        int bucket_r = l + count[r_idx + 1] - 1;
        if (bucket_l < bucket_r) {
            msdSort(a, bucket_l, bucket_r, d + 1, buf);
        }
    }
}

void radixSortMSD(std::vector<std::string>& a) {
    int n = a.size();
    if (n <= 1) return;
    std::vector<std::string> buf(n);
    msdSort(a, 0, n - 1, 0, buf);
}
```

5. Глубокий теоретический анализ (Ловушки экзаменаторов)

⚠️ Ловушка 1: В чём главная уязвимость «наивного» MSD без CUTOFF?

Экзаменатор спросит: **«Что произойдёт, если убрать переключение на Insertion Sort и рекурсивно доходить до подмассивов размера 1?»**

Ответ: Произойдёт **катастрофическая деградация по памяти и времени**. Представь: у тебя 100 строк. На глубине 2 массив разбился на сотни маленьких корзин размера 1 или 2. На каждом рекурсивном вызове алгоритм создаёт массив `count` размера 258 (около 1 КБ памяти). Создание и зануление тысяч пустых массивов `count` по 258 элементов займёт времени в десятки раз больше, чем сама сортировка! **Отсечение (CUTOFF) при $N \leq 16$ жизненно необходимо для MSD.**

⚠️ Ловушка 2: Почему время работы MSD может быть ЛУЧШЕ, чем у LSD?

- **В LSD** мы **всегда** делаем все d проходов для каждого числа, даже если числа уже можно было различить по первому разряду!
- **В MSD** алгоритм просматривает символ **только тогда, когда это необходимо для разрешения коллизии**. Если слова начинаются на разные буквы (`"apple"` , `"banana"` , `"zebra"`), MSD отсортирует их **за один проход по первому символу** и вообще не будет смотреть на остальные буквы!

- **Среднее время MSD:** $\Theta(n \cdot \log_R n)$, то есть алгоритм смотрит только на минимальный различающий префикс.

⚠️ Ловушка 3: MSD и бор (Trie / Префиксное дерево)

> **Вопрос:** «Какая существует связь между MSD Radix Sort и структурой данных Trie (бор)?»

Ответ: Дерево рекурсивных вызовов MSD Radix Sort **в точности изоморфно структуре дерева Trie (бора)**, построенного на этих строках! Каждый вызов `msdSort` для корзины символа c соответствует переходу в дочерний узел дерева Trie по ребру с символом c. По сути, MSD Radix Sort — это **построение и обход бора прямо в массиве без явного выделения узлов**!

⚠️ Продвинутый уровень: Трёхпутевая быстрая сортировка строк (3-Way String QuickSort)

Если алфавит огромен (например, Unicode / UTF-16, где $R = 65536$), создавать массив `count` размера 65536 накладно даже один раз. Тогда используют гибрид **QuickSort + MSD** (алгоритм Бентли — Седжвика): Массив делят по текущему символу на 3 части: `[ символ < pivot | символ == pivot | символ > pivot ]`. Для первых двух частей рекурсия идёт по тому же символу d, а для средней части (`== pivot`) символ совпал, и мы переходим к $d + 1$! Память падает до $O(\log n)$.

Теперь тема MSD раскрыта на исчерпывающем уровне: от обработки коротких строк через `-1` до изоморфизма с деревом Trie.

Двигаемся дальше — к **Теме 9: Пирамида (Двоичная куча и 2 способа построения)**?

---

3. Математика сложности и выбор оптимального основания b

Пусть мы сортируем n чисел, каждое из которых имеет размер w **бит** (например, 32-битные `int`, $w = 32$).

Мы можем сгруппировать биты в «цифры» размером по r **бит**.

- Тогда основание системы счисления: $b = 2^r$.
- Число разрядов: $d = \lceil \frac{w}{r} \rceil$.
- Время работы Counting Sort на каждом шаге: $\Theta(n + b) = \Theta(n + 2^r)$.
- Общее время:

$$T(n) = \Theta\left(\frac{w}{r} \cdot (n + 2^r)\right)$$

Как выбрать r, чтобы минимизировать время?

- Если взять $r = 1$ ($b = 2$ — двоичная система): $d = 32$ прохода. Число операций: $32 \cdot (n + 2) \approx 32n$. Это много проходов!
- Если взять слишком большое r, например $r = 20$ ($b = 2^{20} \approx 10^6$): Проходов мало ($d = 2$), но слагаемое 2^r в памяти и времени сожрёт все ресурсы.
- **Оптимум:** выбираем $2^r \approx n$, то есть $r \approx \lfloor \log_2 n \rfloor$.

Главный теоретический вывод:

Если диапазон чисел не превосходит полинома от n (то есть $w \leq c \log_2 n$):

$$T(n) = \Theta\left(\frac{c \log_2 n}{\log_2 n} \cdot (n + n)\right) = \Theta(n).$$

Radix Sort сортирует такие числа за **гарантированное линейное время**!

---

4. Эталонный C++ код для 32-битных чисел (Байтовая сортировка)

В продакшене никто не делит на 10 (`% 10` и `/ 10` — очень медленные операции для процессора). 32-битное число делят на **4 байта** по 8 бит ($r = 8, b = 2^8 = 256, d = 4$). Извлечение разряда делается побитовым сдвигом `(x >> (8 * byte)) & 0xFF` за **1 такт процессора**!

```cpp
void radixSortLSD(std::vector<uint32_t>& a) {
    int n = a.size();
    if (n <= 1) return;

    std::vector<uint32_t> out(n);
    const int BITS_PER_BYTE = 8;
    const int RADIX = 256; // 2^8
    const int MASK = 255;  // 0xFF

    // 4 прохода: для байтов 0, 1, 2, 3
    for (int byte = 0; byte < 4; ++byte) {
        int shift = byte * BITS_PER_BYTE;
        std::vector<int> count(RADIX, 0);

        // 1. Считаем частоты текущего байта
        for (int i = 0; i < n; ++i) {
            int digit = (a[i] >> shift) & MASK;
            count[digit]++;
        }

        // 2. Префиксные суммы
        for (int i = 1; i < RADIX; ++i) {
            count[i] += count[i - 1];
        }

        // 3. Раскладка справа налево для сохранения устойчивости
        for (int i = n - 1; i >= 0; --i) {
            int digit = (a[i] >> shift) & MASK;
            out[--count[digit]] = a[i];
        }

        a = out; // Переходим к следующему байту
    }
}
```

Этот код сортирует 10^7 чисел в разы быстрее, чем `std::sort`!

---

5. Любимые ловушки экзаменаторов

⚠️ Ловушка 1: Что будет, если в LSD заменить Counting Sort на QuickSort?

- **Ответ:** Алгоритм **полностью сломается** и выдаст мусор. Шаг индукции опирается на то, что числа с одинаковыми текущими цифрами сохраняют свой порядок, выстроенный на предыдущих шагах. Неустойчивый QuickSort этот порядок разрушит.

⚠️ Ловушка 2: Как отсортировать отрицательные числа в Radix Sort?

В знаковом формате `int32_t` старший бит (31-й) отвечает за знак: у отрицательных чисел он равен `1`, а у положительных — `0`. Если сортировать числа как беззнаковые, отрицательные числа окажутся **в конце** массива (так как их старший бит `1` больше `0`).

- **Решение:** На последнем (4-м) проходе инвертировать старший бит числа `x ^ 0x80000000` (это перенесёт отрицательные числа в начало в правильном порядке) либо прибавить `INT_MIN`.

Сводка свойств Radix Sort:

- **Время:** $\Theta(d \cdot (n + b))$ во всех случаях.
- **Память:** $\Theta(n + b)$ (массив вывода + массив счётчиков).
- **Устойчивость: Да** (для LSD).
- **In-place: Нет**.

Мы закрыли сортировки сравнениями и линейные сортировки! Готов переходить к следующему блоку — **Теме 9: Пирамида (Двоичная куча и 2 способа построения)**?

Тема 9. Пирамида (Двоичная куча)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Двоичная куча (Binary Heap) — это массив, логически представляющий почти полное бинарное дерево, удовлетворяющее двум инвариантам:
> 1. **Форма (Shape property):** все уровни полностью заполнены, кроме последнего, который заполняется слева направо без пропусков.
> 2. **Порядок (Heap property):** для max-кучи значение в любом узле не меньше значений его детей ($A[\text{parent}] \geq A[\text{child}]$).
>
> Существует **два способа построения кучи**:
> - **Инкрементальный (через `siftUp`):** добавляем элементы по одному в конец массива и просеиваем вверх. Время: $\Theta(n \log n)$.
> - **Алгоритм Флойда (через `siftDown`):** запускаем просеивание вниз от середины массива к корню. Время: **строго** $\Theta(n)$, так как большинство вершин находятся на малой высоте, и ряд $\sum \frac{h}{2^h} = 2$ сходится к константе».

1. Адресация в массиве без указателей

Благодаря инварианту формы кучу не нужно хранить в виде связанных узлов с указателями. Она идеально ложится в обычный непрерывный массив (при 0-индексации):

```
Дерево:                 Массив индексов:
        [0]             Индекс:   0   1   2   3   4   5
       /   \            Узел:    [0,  1,  2,  3,  4,  5]
     [1]   [2]
    /   \   /
  [3]  [4] [5]
```

Формулы переходов:

- **Корень дерева:** индекс `0`.
- **Левый ребёнок:** $\text{left}(i) = 2i + 1$.
- **Правый ребёнок:** $\text{right}(i) = 2i + 2$.
- **Родитель (предок):** $\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$ (в C++ целочисленное деление `(i - 1) / 2` автоматически округляет вниз).

> ⚠️ **Ловушка экзаменатора**
>
> **Где начинаются листья?** В куче из n элементов все вершины с индексами от $\lfloor \frac{n}{2} \rfloor$ **до** $n - 1$ являются **листьями** (у них физически нет детей, так как $2 \cdot (\frac{n}{2}) + 1 \geq n$). Последний узел, у которого есть хотя бы один ребёнок, находится по индексу $\frac{n}{2} - 1$.

2. Две базовые операции: `siftUp` и `siftDown`

1. `siftUp` (Просеивание вверх)

Используется, когда элемент стал **слишком большим** для своей позиции (например, при вставке нового элемента в конец кучи):

- Сравниваем элемент с родителем.
- Если элемент больше родителя — меняем их местами и шагаем в позицию родителя.
- Повторяем, пока не дойдём до корня или пока родитель не станет $\geq$ элемента.

```
void siftUp(std::vector<int>& a, int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (a[i] > a[p]) {
            std::swap(a[i], a[p]);
            i = p;
        } else {
            break; // Инвариант кучи восстановлен
        }
    }
}
```

- **Сложность:** пропорциональна глубине узла от корня $\Longrightarrow O(\log n)$.

---

2. `siftDown` (Просеивание вниз)

Используется, когда элемент в узле стал **слишком маленьким** (например, после удаления максимума из корня туда поместили элемент из хвоста):

- Сравниваем элемент с его левым и правым ребёнком.
- Находим **НАИБОЛЬШЕГО из двух детей**.
- Если этот ребёнок больше текущего элемента — меняем их местами и спускаемся в позицию этого ребёнка.
- Повторяем, пока узел не станет листом или не станет больше обоих детей.

```
void siftDown(std::vector<int>& a, int n, int i) {
    while (true) {
        int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;

        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;

        if (largest == i) {
            break; // Элемент больше обоих детей, все в порядке
        }
        std::swap(a[i], a[largest]);
        i = largest;
    }
}
```

- **Сложность:** пропорциональна высоте поддерева под узлом $\Longrightarrow O(\log n)$.

> ⚠️ **Ловушка экзаменатора**
>
> **Любимый вопрос экзаменатора:** «Почему в `siftDown` мы обязаны менять элемент именно с **БОЛЬШИМ** из детей?»

> **Ответ:** Представь узел со значением 2, а его дети — 10 и 5. Если мы по ошибке поменяем 2 с **меньшим** ребёнком (5), то на место родителя встанет 5. Но второй ребёнок равен 10! И получится, что у родителя 5 ребёнок равен $10 > 5$, что **немедленно нарушает свойство кучи**! Замена строго с максимумом гарантирует, что новый родитель будет больше обоих сыновей.

---

3. Два вида построения кучи (Build Heap)

Пусть на входе дан произвольный неотсортированный массив из n чисел. Мы хотим сделать из него max-кучу.

Способ 1: Наивный (Инкрементальный, через `siftUp`)

Мы стартуем с пустой кучи и последовательно добавляем в неё элементы массива от 0 до $n-1$, вызывая `siftUp`:

```
void buildHeapNaive(std::vector<int>& a) {
    for (int i = 1; i < a.size(); ++i) {
        siftUp(a, i);
    }
}
```

Сложность: $\Theta(n \log n)$
Почему здесь получается $n \log n$?

- Посмотрим, где сосредоточена работа:
  - Примерно половина элементов дерева ($\frac{n}{2}$) — это **листья на самом нижнем уровне**.
  - Глубина каждого листа равна $\approx \log_2 n$.
  - Каждый из этих $\frac{n}{2}$ листьев потенциально может просеяться вверх **на всю высоту дерева**!
- Считаем сумму работы:

$$T(n) \approx \frac{n}{2} \cdot \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Этот способ тратит максимум усилий на самые многочисленные узлы (листья).

Способ 2: Метод Роберта Флойда (Bottom-Up, через `siftDown`)

Роберт Флойд (1964) предложил гениальную идею: идти **снизу вверх**!

1. Листья (элементы от $\frac{n}{2}$ до $n-1$) просеивать вниз не нужно — у них нет детей, они сами по себе уже являются корректными кучами из одного элемента.
2. Мы начинаем с **последнего нелистового узла** (индекс $\frac{n}{2}-1$) и идём назад к корню (`i = 0`), вызывая для каждого узла `siftDown`:

```
void buildHeapFloyd(std::vector<int>& a) {
    int n = a.size();
    for (int i = n / 2 - 1; i >= 0; --i) {
        siftDown(a, n, i);
    }
}
```

---

4. Строгое доказательство сложности $\Theta(n)$ для алгоритма Флойда

Это центральное математическое доказательство билета.

В чём интуиция?
В отличие от первого способа, в алгоритме Флойда работа узла определяется не глубиной, а **высотой h** (расстоянием до самого нижнего листа):
- $\frac{n}{2}$ листьев имеют высоту $h = 0$ (делают 0 шагов).
- $\frac{n}{4}$ вершин имеют высоту $h = 1$ (делают не более 1 шага).
- $\frac{n}{8}$ вершин имеют высоту $h = 2$ (делают не более 2 шагов).
- ...
- И только **одна** вершина (корень) имеет максимальную высоту $\log_2 n$!

> **Главный вывод:** Большинство вершин делают минимальное количество шагов, а максимальную работу делает лишь горстка вершин наверху!

Математический вывод:
1. Количество вершин на высоте h в почти полном бинарном дереве не превосходит:

$$\text{leftceil}\left(\frac{n}{2^{h+1}}\right)$$

2. Вершина на высоте h при вызове `siftDown` совершает не более $c \cdot h$ операций.
3. Суммарное время работы:

$$T(n) \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} \text{leftceil}\left(\frac{n}{2^{h+1}}\right) \cdot c \cdot h \leq c \cdot n \sum_{h=1}^{\infty} \frac{h}{2^h}.$$

Вычисление суммы бесконечного ряда $S = \sum_{h=1}^{\infty} \frac{h}{2^h}$:
Выпишем ряд:

$$S = \frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \dots$$

Умножим обе части на $\frac{1}{2}$:

$$\frac{1}{2}S = \frac{1}{2^2} + \frac{2}{2^3} + \frac{3}{2^4} + \dots$$

Вычтем второе уравнение из первого:

$$S - \frac{1}{2}S = \frac{1}{2} + \left(\frac{2}{4} - \frac{1}{4}\right) + \left(\frac{3}{8} - \frac{2}{8}\right) + \left(\frac{4}{16} - \frac{3}{16}\right) + \dots$$

Получаем обычную бесконечную геометрическую прогрессию:

$$\frac{1}{2}S = \sum_{k=1}^{\infty} \frac{1}{2^k} = \frac{\frac{1}{2}}{1 - \frac{1}{2}} = 1.$$

Отсюда:

$$\frac{1}{2}S = 1 \Longrightarrow \boldsymbol{S = 2}.$$

Подставляем $S = 2$ в оценку сложности:

$$T(n) \leq c \cdot n \cdot S = 2c \cdot n = \boldsymbol{\Theta(n)}.$$

Доказано абсолютно строго!

---

5. Важные вопросы на понимание

1. Является ли пирамида двоичным деревом поиска (BST)?

НЕТ!

- В дереве поиска (BST) есть горизонтальный порядок: левый ребёнок $<$ корень $<$ правый ребёнок.
- В куче есть **только вертикальный порядок**: предок $\geq$ потомок. Между левым и правым сыновьями **никакого порядка нет** (левый может быть как больше, так и меньше правого).

2. За сколько можно найти произвольный элемент в куче?

За $\boldsymbol{\Theta(n)}$ (полным обходом массива). Так как между братьями и поддеревьями нет горизонтального порядка, бинарный поиск в куче невозможен. Куча оптимизирована исключительно под одну операцию — мгновенный доступ к экстремуму (максимуму или минимуму) за $O(1)$.

Сравнительная таблица двух способов построения:

Параметр	Способ 1 (`siftUp`)	Способ 2 (`siftDown` , Флойд)
Направление обхода	Слева направо ($1 \dots n-1$)	Справа налево ($\frac{n}{2}-1 \dots 0$)
Базовая операция	`siftUp(i)`	`siftDown(i)`
Время работы	$\Theta(n \log n)$	$\boldsymbol{\Theta(n)}$
Память	$O(1)$ на месте	$O(1)$ на месте

Теперь пирамида и магия Флойда у тебя под полным контролем. Переходим к **Теме 10: Пирамидальная сортировка (HeapSort)**?

Тема 10. Пирамидальная сортировка (HeapSort)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Пирамидальная сортировка — это неустойчивый (unstable) алгоритм сортировки сравнениями, работающий строго на месте (in-place, доп. память $O(1)$). Идея заключается в **оптимизации сортировки выбором (Selection Sort)**: вместо линейного поиска максимума за $O(n)$ мы строим max-кучу, извлекаем максимум за $O(\log n)$ и ставим его в конец массива. Алгоритм состоит из двух фаз:
> 1. Построение max-кучи алгоритмом Флойда за $\Theta(n)$.
> 2. Цикл из $n-1$ шагов: обмен корня с последним элементом и просеивание вниз `siftDown` за $O(\log n)$.
>
> Время работы — **гарантированные $\Theta(n \log n)$ во всех случаях** (лучший, средний, худший)».

---

1. Концепция: как превратить Selection Sort в ракету

Вспомним наивную сортировку выбором (Selection Sort):

- Мы ищем максимум среди неотсортированных элементов за линейное сканирование $\Theta(n)$ и меняем его с последним элементом.
- Делаем так n раз $\Longrightarrow$ время $\Theta(n^2)$.

В чём идея Д. Уильямса (1964) в HeapSort? Зачем каждый раз искать максимум полным перебором, если можно организовать данные в **max-кучу**, где максимум **всегда лежит на нулевом индексе** $A[0]$? Тогда взятие максимума стоит $O(1)$, а поддержание структуры после его удаления — всего $O(\log n)$!

---

2. Две фазы алгоритма на пальцах

Алгоритм сортирует массив по возрастанию с помощью **max-кучи** (не min-кучи, а именно max!).

Фаза 1: Построение max-кучи (Build Heap)

Превращаем входной массив $A[0 \ldots n-1]$ в max-кучу алгоритмом Флойда (идём от $\frac{n}{2}-1$ к 0):

```
for (int i = n / 2 - 1; i >= 0; --i) {
    siftDown(a, n, i);
}
```

- Теперь в ячейке $A[0]$ гарантированно лежит **абсолютный максимум** всего массива.
- Затраты времени: строго $\Theta(n)$.

Фаза 2: Поочерёдное извлечение максимумов

Массив логически делится на две части: `[ Живая куча | Отсортированный суффикс ]`

- Слева (от 0 до i): элементы кучи.
- Справа (от $i+1$ до $n-1$): уже отсортированные максимальные элементы.

Мы крутим цикл для $i = n-1$ назад до 1:

1. Меняем местами корень $A[0]$ (текущий максимум кучи) с последним элементом кучи $A[i]$. Теперь этот максимум **встал на своё идеальное финальное место в массиве**!
2. Уменьшаем размер активной кучи на единицу (теперь куча имеет размер i).

---

3. В корне $A[0]$ оказался случайный элемент из хвоста. Вызываем `siftDown(a, i, 0)` на отрезке размера i, чтобы вернуть свойство кучи.

```
Шаг 1: [ MAX  .  .  .  .  x ]  ->  swap(A[0], A[n-1])  ->  [ x  .  .  .  . | MAX ]
Шаг 2: siftDown(A, n-1, 0)                              ->  [ MAX2 .  .  .  . | MAX ]
Шаг 3: swap(A[0], A[n-2])                               ->  [ y  .  .  . | MAX2  MAX ]
... и так до конца!
```

3. Эталонный код на C++ (строго in-place)

```cpp
void siftDown(std::vector<int>& a, int heap_size, int i) {
    while (true) {
        int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;

        if (l < heap_size && a[l] > a[largest]) largest = l;
        if (r < heap_size && a[r] > a[largest]) largest = r;

        if (largest == i) break;
        std::swap(a[i], a[largest]);
        i = largest;
    }
}

void heapSort(std::vector<int>& a) {
    int n = a.size();
    if (n <= 1) return;

    // Фаза 1: Построение max-кучи за Theta(n)
    for (int i = n / 2 - 1; i >= 0; --i) {
        siftDown(a, n, i);
    }

    // Фаза 2: Извлечение максимумов за Theta(n log n)
    for (int i = n - 1; i > 0; --i) {
        // Корень (максимум) отправляем в конец неотсортированной части
        std::swap(a[0], a[i]);
        // Восстанавливаем кучу на уменьшенном префиксе размера i
        siftDown(a, i, 0);
    }
}
```

4. Математический анализ сложности

1. Время работы: $\Theta(n \log n)$

- **Фаза 1 (Build Heap):** $\Theta(n)$.
- **Фаза 2 (Экстракция):** Мы делаем $n-1$ просеивание вниз. На шаге i размер кучи равен i, а высота дерева — $\lfloor \log_2 i \rfloor$. Суммарное число операций:

$$T(n) = \Theta(n) + \sum_{i=1}^{n-1} O(\log i) = \Theta(n) + O(\log(n!)).$$

По формуле Стирлинга $\log(n!) = \Theta(n \log n)$.

- **Во всех случаях:**

- В худшем: $\Theta(n \log n)$.
- В среднем: $\Theta(n \log n)$.
- В лучшем (даже если все элементы одинаковы): корень всё равно спускается на дно $\Longrightarrow \Theta(n \log n)$.

2. Память: строго $O(1)$

- Алгоритм не использует рекурсию (`siftDown` написан через `while`).
- Нет стека вызовов, нет буферов слияния. Всё происходит в исходном массиве.

---

5. Доказательство неустойчивости (Контрпример)

HeapSort **не сохраняет** относительный порядок равных элементов.

Нарисуй на доске минимальный контрпример из трёх чисел:

$$A = [2_a, 2_b, 1]$$

1. **Фаза 1 (Build Heap):**
   - Узел 0 равен 2_a, его дети — 2_b и 1.
   - Так как $2_a \geq 2_b$ и $2_a \geq 1$, свойство кучи выполнено изначально.
   - Массив не меняется: $[2_a, 2_b, 1]$.
2. **Фаза 2 (Шаг 1):**
   - Меняем корень $A[0]$ (2_a) с последним элементом $A[2]$ (1):

$$[1, 2_b, \mathbf{2_a}]$$

- Вызываем `siftDown` на префиксе $[1, 2_b]$ размера 2: 1 меняется с бóльшим ребёнком 2_b:

$$[\mathbf{2_b}, 1, \mathbf{2_a}]$$

3. **Фаза 2 (Шаг 2):**
   - Меняем корень $A[0]$ (2_b) с последним элементом префикса $A[1]$ (1):

$$[1, \mathbf{2_b}, \mathbf{2_a}]$$

Итог: Элемент 2_b оказался **раньше** 2_a, хотя в исходном массиве стоял позже! HeapSort **неустойчив**.

---

6. Главный практический парадокс: почему QuickSort быстрее HeapSort?

> **Любимый каверзный вопрос:** «И QuickSort, и HeapSort работают на месте за $O(1)$ памяти. В худшем случае QuickSort деградирует до $O(n^2)$, а HeapSort всегда гарантирует $O(n \log n)$. Почему тогда все стандартные библиотеки используют QuickSort, а не HeapSort?»

Ответ (Кэш процессора / Locality of Reference):

- **QuickSort обожает кэш L1/L2:** указатели бегут по памяти **последовательно** слева направо и справа налево. Процессор загружает в кэш целые кэш-линии (по 64 байта) за один раз. Промахов мимо кэша (Cache Misses) почти нет.
- **HeapSort уничтожает кэш процессора:** в операции `siftDown` переходы по индексам $i \to 2i + 1 \to 4i + 3 \to \ldots$ удваивают индекс на каждом шаге! Алгоритм прыгает по оперативной памяти гигантскими скачками, вызывая кэш-промах почти на каждом сравнении.

На реальном процессоре QuickSort обгоняет HeapSort в **2–3 раза**!

Где же HeapSort реально применяется?

В алгоритме **Introsort (`std::sort` в C++)**: `std::sort` запускает быстрый QuickSort, но если глубина рекурсии превышает $2 \log_2 n$ (то есть QuickSort попал на худший случай), управление немедленно передаётся **HeapSort**!

Это даёт идеальную скорость QuickSort в 99.9% случаев и железную защиту HeapSort от зависаний в худшем случае.

Чек-лист характеристик HeapSort:

Характеристика	Значение
Время (лучшее / среднее / худшее)	$\Theta(n \log n)$ во всех случаях
Дополнительная память	$\boldsymbol{O(1)}$ (строго in-place)
Устойчивость (Stability)	**Нет** (контрпример: $[2_a, 2_b, 1]$)
Связь с Selection Sort	Оптимизация выбора максимума с $O(n)$ до $O(\log n)$

Тема 10 закрыта! Переходим к **Теме 11: Очередь с приоритетом на пирамиде (повышение / понижение приоритета)**?

Тема 11. Очередь с приоритетом на пирамиде

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «**Очередь** — это абстрактный интерфейс. Обычная очередь реализует дисциплину **FIFO** («первым пришёл — первым ушёл»). **Очередь с приоритетом (Priority Queue)** извлекает элемент не по времени прихода, а по **наивысшему приоритету**. Её можно реализовать на любой базовой структуре (массиве, списке, BST), но **двоичная куча (пирамида)** является оптимальным выбором: она даёт вставку и извлечение за $O(\log n)$ без оверхеда на указатели и балансировку. При изменении приоритета (`increaseKey` / `decreaseKey`) для быстрого нахождения элемента за $O(1)$ используется **позиционный массив обратных индексов** `pos` , что позволяет выполнить операцию за $O(\log n)$ и является фундаментом алгоритма Дейкстры за $O((V + E) \log V)$».

1. Что такое «Очередь» вообще? (АТД vs Структура данных)

Нужно чётко разделять два понятия:

1. **Абстрактный Тип Данных (АТД):** Это «контракт», чёрный ящик. Он говорит, **ЧТО** структура умеет делать (набор операций), но умалчивает, **КАК** это устроено внутри памяти.
2. **Структура данных:** Это конкретная физическая организация байтов в памяти компьютера (массив, связный список и т.д.).

Обычная очередь (FIFO Queue)

Интерфейс обычной очереди: `push(x)` (встать в конец), `pop()` (выйти из начала), `front()` (посмотреть первого). Дисциплина: **FIFO (First In, First Out)** — кто раньше встал, того и обслужили (как очередь за кофе).

> **Почему очередь можно реализовать на ЧЁМ УГОДНО?** Потому что АТД не привязан к «железу». Мы можем реализовать очередь:
> - На **кольцевом массиве:** два индекса `head` и `tail` , двигающиеся по кругу через `% capacity` (память $O(1)$ на узел, быстро).
> - На **односвязном списке:** указатель на `head` для снятия и на `tail` для добавления.
> - На **двух стеках:** входной стек S_{in} и выходной S_{out} (при опустошении S_{out} переливаем в него элементы из S_{in}, инвертируя порядок).

2. Что такое Очередь с приоритетом (Priority Queue)?

В реальном мире принцип FIFO часто несправедлив или опасен.

> **Пример из жизни:** В приёмный покой больницы поступают пациенты. Один пришёл в 10:00 с ушибом пальца, а второй в 10:05 с инфарктом. Если обслуживать по FIFO, второй пациент погибнет. Нужна **очередь с приоритетом**: врач всегда первым забирает того, у кого наивысший приоритет (наиболее тяжёлое состояние), независимо от времени прихода!

Интерфейс очереди с приоритетом:

- `insert(x, priority)` — добавить элемент x с приоритетом p.
- `extractMax()` (или `extractMin()`) — извлечь элемент с наивысшим приоритетом.

- `getMax()` — посмотреть на элемент с наивысшим приоритетом без удаления за $O(1)$.
- `changePriority(id, new_priority)` — изменить приоритет уже лежащего в очереди элемента.

---

3. Баттл структур данных: почему именно Куча (Пирамида)?

Почему нельзя просто взять массив или список? Давай сравним все варианты:

Физическая реализация	Вставка `insert`	Извлечение максимума `extractMax`	Поиск максимума `getMax`	Расход памяти на узел
Неотсортированный массив	$\boldsymbol{O(1)}$ (в конец)	$O(n)$ (надо найти максимум перебором)	$O(n)$	Минимальный (0 байт оверхеда)
Отсортированный массив	$O(n)$ (нужно сдвигать элементы)	$\boldsymbol{O(1)}$ (снять с конца)	$\boldsymbol{O(1)}$	Минимальный
Связный список	$O(1)$ или $O(n)$	$O(n)$ или $O(1)$	$O(n)$ или $O(1)$	Огромный ($8-16$ байт на указатели)
Сбалансированное дерево поиска (AVL / Red-Black)	$O(\log n)$	$O(\log n)$	$O(\log n)$ (или $O(1)$)	Большой ($24-32$ байта: указатели `left`, `right`, `parent`, бит цвета)
Двоичная куча (Пирамида)	$\boldsymbol{O(\log n)}$	$\boldsymbol{O(\log n)}$	$\boldsymbol{O(1)}$	**Абсолютно нулевой оверхед!**

Почему куча — безоговорочный победитель?

1. **Золотая середина по времени:** И вставка, и удаление работают за быстрый логарифм $\boldsymbol{O(\log n)}$, а просмотр максимума — за $\boldsymbol{O(1)}$.
2. **Память и процессор:** Куча не хранит указателей на детей/родителей (всё вычисляется по индексам `2i+1` и `(i-1)/2`). Она упакована в плотный массив, который не фрагментирует память и не требует балансировок, как деревья поиска.

---

4. Изменение приоритета: `increaseKey` и `decreaseKey`

Рассмотрим **max-кучу** (корень — максимум).

Операция 1: Повышение приоритета (`increaseKey`)

Приоритет элемента **увеличился** (стал больше, чем был).

- Элемент теперь может стать больше своего родителя!
- Свойство кучи нарушено **снизу вверх**.
- **Действие:** вызываем `siftUp(i)` за $O(\log n)$.

Операция 2: Понижение приоритета (`decreaseKey`)

Приоритет элемента **уменьшился** (стал меньше, чем был).

- Элемент теперь может стать меньше своих детей!
- Свойство кучи нарушено **сверху вниз**.
- **Действие:** вызываем `siftDown(i)` за $O(\log n)$.

> 💡 **Важное замечание**
>
> **Осторожно, терминология в алгоритме Дейкстры!** В алгоритме Дейкстры мы ищем **кратчайший** путь, поэтому используется **min-куча** (корень — минимум). Когда мы находим более короткий путь до вершины, её рассто-

> яние **уменьшается** (это называется `decreaseKey`). Но в min-куче уменьшение расстояния делает вершину более «приоритетной», поэтому она должна всплывать вверх через **`siftUp`**!

---

5. Главная проблема и супер-решение: Позиционный массив (Indirect Heap)

А вот здесь кроется **главная ловушка темы**, на которой валят на экзамене.

> **Экзаменатор:** «Отлично, `increaseKey` делает `siftUp` за $O(\log n)$. Но как вы узнаете, на каком индексе i в массиве кучи СЕЙЧАС лежит элемент с идентификатором id?»

Вспомним Тему 9: **в куче нет поиска за $O(\log n)$**! Чтобы найти, где лежит вершина с номером `ID = 5`, придётся сканировать весь массив за $O(n)$. Если поиск занимает $O(n)$, то грош цена нашему `siftUp` за $O(\log n)$ — вся операция займёт $O(n)$!

Архитектурное решение: Массив обратных индексов `pos`

Мы заводим два синхронных массива:

1. `heap[i] = id` — хранит идентификатор элемента, лежащего в i-й ячейке кучи.
2. `pos[id] = i` — хранит индекс в массиве кучи, где **в данный момент** находится элемент id.

Инвариант взаимной однозначности:

$$\mathbf{pos}[\mathbf{heap}[i]] == i \quad \text{и} \quad \mathbf{heap}[\mathbf{pos}[\mathbf{id}]] == \mathbf{id}.$$

```
ID элемента:     0    1    2    3    (например, номера вершин в графе)
pos[ID]:       [ 2,   0,   3,   1 ]  <-- говорит: вершина 1 лежит в корне (индекс 0)!

Куча (индекс):   0    1    2    3
heap[i]:       [ 1,   3,   0,   2 ]  <-- в ячейке 0 лежит вершина 1
```

Как поддерживать инвариант при обменах?

Когда в операциях `siftUp` или `siftDown` два узла кучи i и j меняются местами, мы **синхронно обновляем их позиции в массиве `pos`**:

```
void swapNodes(int i, int j) {
    // 1. Меняем местами элементы в куче
    std::swap(heap[i], heap[j]);

    // 2. Мгновенно обновляем их обратные адреса!
    pos[heap[i]] = i;
    pos[heap[j]] = j;
}
```

Теперь изменение приоритета работает за истинные $O(\log n)$:

```
void changePriority(int id, int new_priority) {
    int i = pos[id]; // 1. Мгновенно находим ячейку в куче за O(1)!
    int old_priority = priority[id];
    priority[id] = new_priority;

    if (new_priority > old_priority) {
        siftUp(i);   // 2. Всплываем за O(log n)
    } else {
        siftDown(i); // 2. Погружаемся за O(log n)
```

```
    }
}
```

6. Зачем это нужно? Алгоритм Дейкстры

Именно благодаря очереди с приоритетом и массиву `pos` алгоритм Дейкстры работает за канонические:

$$\boldsymbol{O((V+E)\log V)}$$

- Каждая вершина извлекается из кучи 1 раз: $V \times \text{extractMin} = O(V \log V)$.
- Для каждого ребра может произойти релаксация (улучшение расстояния): $E \times \text{decreaseKey} = O(E \log V)$.
- Без массива `pos` алгоритм Дейкстры работал бы либо за $O(V^2 + E)$, либо требовал бы засорения кучи дубликатами за $O(E \log E)$ памяти и времени.

Чек-лист характеристик очереди с приоритетом на куче:

- **`insert`:** $O(\log n)$ (в конец и `siftUp`).
- **`extractMax` / `extractMin`:** $O(\log n)$ (корень меняем с хвостом и `siftDown`).
- **`getMax` / `getMin`:** $O(1)$ (просто прочитать ячейку `heap[0]`).
- **`increaseKey` / `decreaseKey`:** $O(\log n)$ (поиск за $O(1)$ через `pos` + просеивание).
- **Память:** строго $O(n)$ (два линейных массива `heap` и `pos`).

Мы полностью закрыли блок структур данных и сортировок! Теперь мы переходим к тому, чего ты больше всего ждал — **Блоку Динамического Программирования**!

Готов начать с **Темы 12: ДП (определения, виды, принцип Беллмана, пример на Фибоначчи)**?

Тема 12. Динамическое программирование: основы и архитектура

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Динамическое программирование (ДП) — это метод решения задач путём их декомпозиции на **перекрывающиеся подзадачи**, где результат каждой подзадачи вычисляется один раз и сохраняется в памяти. Для применимости ДП необходимы два свойства: **оптимальная подструктура** (принцип Беллмана: оптимум всей задачи складывается из оптимумов подзадач) и **перекрывающиеся подзадачи**. Реализуется двумя подходами: **Top-Down** (нисходящий с мемоизацией на рекурсии) и **Bottom-Up** (восходящий с табуляцией в циклах). Любое ДП эквивалентно поиску пути или подсчёту путей в **ориентированном ациклическом графе (DAG)** состояний».

1. Откуда такое странное название?

Слово «программирование» здесь не имеет отношения к написанию кода на C++ или Python. Создатель метода, выдающийся математик Ричард Беллман, придумал термин в 1950-х годах, работая в корпорации RAND на военных. В то время слово **«programming»** означало **составление оптимального плана или расписания** (как «линейное программирование»), а слово **«dynamic»** Беллман добавил, чтобы проект звучал солидно для сенаторов и под него щедро выделили финансирование.

> **Суть ДП одной фразой: «Помни своё прошлое, чтобы не вычислять его заново»** (Remember your past, or you're condemned to repeat it).

2. Два обязательных признака: когда ДП РАБОТАЕТ, а когда НЕТ?

На экзамене это спросят обязательно. ДП применимо **только** тогда, когда задача обладает двумя свойствами:

Свойство 1: Оптимальная подструктура (Принцип Беллмана)

> Оптимальное решение глобальной задачи состоит из оптимальных решений входящих в неё подзадач.

- **Где РАБОТАЕТ:** Поиск кратчайшего пути на карте из Москвы во Владивосток через Новосибирск. Если кратчайший путь идёт через Новосибирск, то его кусок «Москва → Новосибирск» **обязан** быть кратчайшим путём между Москвой и Новосибирском! Нельзя срезать путь до Владивостока, сделав хуже дорогу до Новосибирска.
- **Где НЕ РАБОТАЕТ (Контрпример!):** Поиск **самого длинного простого пути** в графе. Если мы склеим два самых длинных простых пути, они могут пересечься по вершинам и перестанут быть «простым» путём. Здесь оптимумы подзадач не склеиваются в глобальный оптимум.

Свойство 2: Перекрывающиеся подзадачи (Overlapping Subproblems)

В процессе работы алгоритм натыкается на **одни и те же вопросы снова и снова**.

В чём разница с «Разделяй и властвуй» (MergeSort):

- **В MergeSort:** левая и правая половины массива **независимы и не пересекаются**. Левая половина никогда не спросит: «Эй, а что там получилось в правой?» Перекрытия нет.
- **В ДП:** разные ветки вычислений обращаются к **одинаковым ячейкам памяти**. И вместо того, чтобы считать их заново миллион раз, мы считаем их **один раз и сохраняем**.

3. Четыре шага проектирования ЛЮБОГО ДП (Чек-лист Беллмана)

Когда тебе на экзамене дадут незнакомую задачу на ДП, не пытайся сразу писать код. Пройди по этим 4 шагам вслух:

```
1. Определение состояния (State)  --->  "Что означает ячейка dp[i]?"
2. База динамики (Base Cases)     --->  "Что мы знаем сразу, без вычислений?"
3. Переход динамики (Transition)  --->  "Как выразить dp[i] через предыдущие?"
4. Порядок обхода (Order)         --->  "В каком направлении крутить цикл?"
```

4. Разбор на числах Фибоначчи: эволюция от ужаса до идеала

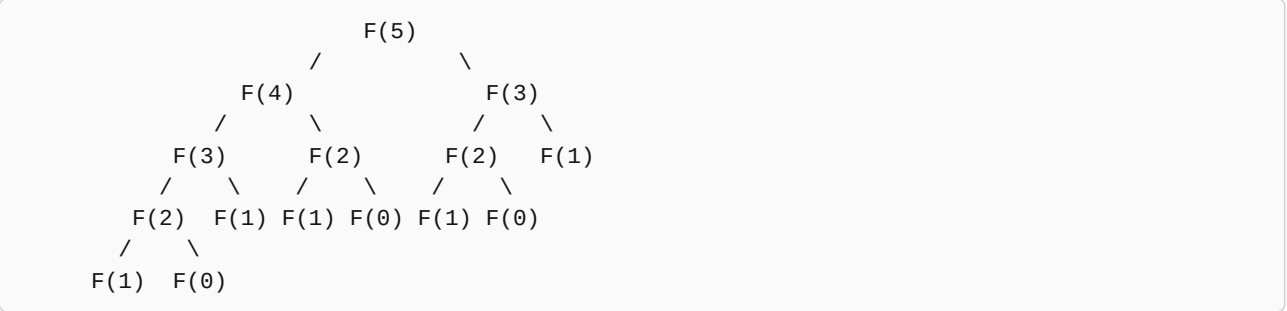
Последовательность Фибоначчи:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

Этап 0: Наивная рекурсия (Школьный код)

```
int fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}
```

Почему это катастрофа?
Посмотрим на дерево вызовов для $F(5)$:



Взгляни на этот ужас:

- $F(3)$ вычисляется **2 раза**.
- $F(2)$ вычисляется **3 раза**.
- $F(1)$ вычисляется **5 раз**!

Количество листьев дерева растёт как золотое сечение в степени n:

$$T(n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) \approx \mathbf{\Theta(1.618^n)}.$$

При $n = 50$ компьютер зависнет навсегда (сделает 10^{15} операций)!

Этап 1: Top-Down ДП (Нисходящее с мемоизацией)

Мы оставляем ту же самую рекурсию, но добавляем «блокнот» (кэш `memo`), куда записываем уже посчитанные значения:

```cpp
std::vector<int> memo(n + 1, -1); // -1 означает "еще не вычислено"

int fibTopDown(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n]; // Уже знаем ответ! Мгновенный возврат за O(1)

    memo[n] = fibTopDown(n - 1) + fibTopDown(n - 2); // Считаем и запоминаем
    return memo[n];
}
```

Что произошло с деревом?
Как только $F(3)$ посчитан один раз левой веткой, правая ветка просто берёт готовый ответ из памяти за $O(1)$. Дерево рекурсии мгновенно **схлопнулось в цепочку из n узлов**!

- **Время работы:** упало с 1.618^n до **строгих $\Theta(n)$**!
- **Память:** $O(n)$ под массив `memo` $+O(n)$ стек рекурсии.

Этап 2: Bottom-Up ДП (Восходящее с табуляцией)

А зачем вообще звать рекурсию и тратить память на стек вызовов? Давайте пойдём снизу вверх: от базы (0 и 1) к цели (n), заполняя массив `dp` в обычном цикле:

```cpp
int fibBottomUp(int n) {
    if (n <= 1) return n;
    std::vector<int> dp(n + 1);

    // 1. База
    dp[0] = 0;
    dp[1] = 1;

    // 2. Цикл (переход)
    for (int i = 2; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

- **Время работы:** $\Theta(n)$ без какого-либо оверхеда на вызовы функций.
- **Память:** $\Theta(n)$ под массив `dp`.

Этап 3: Оптимизация памяти до $O(1)$ (Скользящее окно)

Посмотрим внимательно на формулу:

$$\mathrm{dp}[i] = \mathrm{dp}[i-1] + \mathrm{dp}[i-2].$$

Чтобы узнать $\mathrm{dp}[i]$, нам нужны **только два предыдущих числа**! Нам абсолютно не нужно помнить, чему равнялся $\mathrm{dp}[0]$ или $\mathrm{dp}[5]$, когда мы вычисляем $\mathrm{dp}[100]$. Вся история до $i-2$ уже бесполезна!

Заменим массив размером n на **всего три переменные**:

```cpp
int fibOptimized(int n) {
    if (n <= 1) return n;

    int prev2 = 0; // F(i-2)
    int prev1 = 1; // F(i-1)
    int current = 0;
```

```
    for (int i = 2; i <= n; ++i) {
        current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }

    return current;
}
```

- **Время работы:** $\Theta(n)$.
- **Память:** $O(1)$! Мы сэкономили всю оперативную память.

5. Сравнение Top-Down и Bottom-Up (Любимый вопрос на защите)

Критерий	Top-Down (Мемоизация)	Bottom-Up (Табуляция)
Принцип	Сверху вниз (от цели к базе)	Снизу вверх (от базы к цели)
Механизм	Рекурсия + кэширование в хэш-таблице/массиве	Итеративные циклы `for` по таблице
Достижимость состояний	**Плюс:** Вычисляет **только те** состояния, которые реально нужны для ответа	**Минус:** Заполняет всю таблицу подряд, даже ненужные клетки
Скорость и кэш	Медленнее (накладные расходы на вызовы функций)	**Быстрее** (процессор легко векторизует простой цикл `for`)
Память	Риск `Stack Overflow` при глубине $> 10^5$	**Легко сжимается** (как мы сжали память с $O(n)$ до $O(1)$)

6. Граф состояний (DAG): глубокий взгляд для 1 и 2 потока

Любая задача динамического программирования — это на самом деле **ориентированный ациклический граф (DAG — Directed Acyclic Graph)**:

- **Вершины графа:** Это состояния dp. В Фибоначчи — это числа $0, 1, 2, \ldots, n$.
- **Рёбра графа:** Это переходы между ними. Из вершины $i - 1$ и $i - 2$ идут стрелочки в вершину i.
- **Значения на рёбрах:** Веса или коэффициенты.

```
(0) ---> (2) ---> (3) ---> (4) ---> (5)
  \     ^  \     ^  \     ^
   \---/    \---/    \---/
(1) ------/
```

> **Почему граф ОБЯЗАН быть ациклическим (без циклов)?** Если бы в графе был цикл (например, состояние A зависит от B, а B зависит от A), мы получили бы замкнутый круг («курица или яйцо»). Никакое ДП не сможет вычислить значение! Поэтому правильный порядок заполнения массива в Bottom-Up — это не что иное, как **топологическая сортировка графа состояний**!

7. Бонус для 1 потока: Фибоначчи за $O(\log n)$ (Матричная экспоненциация)

Экзаменатор может хитро спросить: **«А можно ли посчитать n-е число Фибоначчи быстрее, чем за $O(n)$?»**

Ответ: Да, за $O(\log n)$ через возведение матриц в степень! Запишем переход в матричном виде:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$$

Применяя это соотношение n раз, получаем:

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Матрицу размера 2×2 можно возвести в степень n алгоритмом **быстрого бинарного возведения в степень (BinPow)** всего за $\log_2 n$ матричных умножений! Сложность: $O(\log n)$ арифметических операций.

Фундамент ДП заложен!
Теперь у тебя в руках есть шаблон мышления:
1. Понять, что такое состояние.
2. Записать базу.
3. Вывести рекуррентную формулу.
4. Понять, как сжать память.

Готов применить это супероружие к классическим задачам и перейти к **Теме 13: ДП Кузнечик (с оптимизацией деком и матрицами)**?

Тема 13. ДП: Задача о кузнечике

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Задача о кузнечике — классическая одномерная динамика на прямой. Кузнечик стартует в клетке 0 и хочет попасть в клетку $n-1$, совершая прыжки длиной от 1 до k клеток.
> - Если задача **комбинаторная** (число путей): $\mathrm{dp}[i] = \sum_{j=1}^{k} \mathrm{dp}[i-j]$.
> - Если задача **оптимизационная** (минимальная стоимость с ценами $c[i]$): $\mathrm{dp}[i] = c[i] + \min_{1 \le j \le k} \mathrm{dp}[i-j]$.
>
> Базовое решение работает за время $O(\mathbf{nk})$ и память $O(n)$. Память сжимается до $O(k)$ циклическим буфером. Время поиска минимума ускоряется с $O(\mathrm{nk})$ до $O(n)$ с помощью **монотонного дека (скользящего минимума)**, а число путей при $n \le 10^{18}$ находится за $O(k^3 \log n)$ через быстрое возведение матрицы в степень».

1. Постановка задачи с монетами/стоимостями

Кузнечик находится на числовой прямой в клетке 0. Ему нужно добраться до клетки $n-1$.
- Из клетки i он может прыгнуть в любую из клеток $i+1, i+2, \ldots, i+k$.
- За посещение клетки i нужно заплатить стоимость $c[i]$ (или забрать монеты, если максимизируем).
- **Цель:** дойти до клетки $n-1$ с **минимальными** суммарными затратами.

Применим наш **Чек-лист 4 шагов**:

1. **Состояние:** $\mathrm{dp}[i]$ — минимальная стоимость добраться из старта в клетку i.
2. **База:** $\mathrm{dp}[0] = c[0]$ (в стартовой клетке мы уже стоим и платим её цену). Для остальных $\mathrm{dp}[i] = +\infty$.
3. **Переход:** В клетку i можно было прийти только одним прыжком из клеток $i-1, i-2, \ldots, i-k$. Мы выбираем самую выгодную:

$$\mathrm{dp}[i] = c[i] + \min_{1 \le j \le k, i-j \ge 0} \mathrm{dp}[i-j]$$

4. **Порядок обхода:** Строго слева направо: $i = 1, 2, \ldots, n-1$.

2. Базовая реализация и восстановление пути

Очень часто на экзамене говорят: **«Отлично, минимальную стоимость вы нашли. А теперь выведите САМ ПУТЬ — номера клеток, по которым прыгал кузнечик!»**

Как восстанавливать ответ? (Массив предков `prev`)

Каждый раз, когда мы выбираем оптимального предшественника для клетки i, мы записываем его индекс в массив `prev[i]`. В конце мы просто «раскручиваем» путь с конца: от $n-1$ назад до 0 по стрелочкам!

```
#include <vector>
#include <algorithm>
#include <iostream>

void solveGrasshopper(int n, int k, const std::vector<int>& c) {
    const int INF = 1e9;
    std::vector<int> dp(n, INF);
    std::vector<int> prev(n, -1); // Массив предков для восстановления пути
```

```
    // 1. База
    dp[0] = c[0];

    // 2. Заполнение таблицы
    for (int i = 1; i < n; ++i) {
        for (int j = 1; j <= k; ++j) {
            if (i - j >= 0) {
                if (dp[i - j] + c[i] < dp[i]) {
                    dp[i] = dp[i - j] + c[i];
                    prev[i] = i - j; // Запомнили, откуда пришли!
                }
            }
        }
    }

    std::cout << "Минимальная стоимость: " << dp[n - 1] << "\n";

    // 3. Восстановление пути с конца в начало
    std::vector<int> path;
    for (int curr = n - 1; curr != -1; curr = prev[curr]) {
        path.push_back(curr);
    }
    std::reverse(path.begin(), path.end()); // Разворачиваем: от 0 к n-1

    std::cout << "Оптимальный путь: ";
    for (int node : path) std::cout << node << " ";
    std::cout << "\n";
}
```

- **Время работы:** $\Theta(n \cdot k)$ (вложенный цикл k раз на каждом шаге).
- **Память:** $\Theta(n)$ под массивы `dp` и `prev`.

---

3. Оптимизация памяти: Сжатие до $O(k)$

Если восстанавливать путь не нужно, а требуется только итоговая стоимость: Посмотрим на формулу: $\mathrm{dp}[i]$ зависит только от k **последних значений**:

$$\mathrm{dp}[i-1],; \mathrm{dp}[i-2],; \ldots,; \mathrm{dp}[i-k]$$

Хранить весь массив размера n расточительно! Мы можем завести массив размера k и обращаться к ячейкам по кругу через **циклический буфер**:

$$\text{индекс} = i \bmod k.$$

Память сокращается с $O(n)$ до **строгих $O(k)$**!

---

4. Оптимизация времени: от $O(\mathrm{nk})$ к $O(n)$ через Монотонный Дек

А вот это — топ-уровень для 1 и 2 потока.

> **Экзаменатор:** «Представьте, что $n = 10^5$ и длина прыжка $k = 10^5$. Ваше решение сделает $n \cdot k = 10^{10}$ операций и упадёт по Time Limit (TL). Как решить задачу за **строго $O(n)$**?»

Интуиция
Посмотрим внимательно на то, что мы ищем во внутреннем цикле:

$$\min(\mathrm{dp}[i-1],; \mathrm{dp}[i-2],; \ldots,; \mathrm{dp}[i-k])$$

Это же классическая задача **поиска минимума в скользящем окне длины k (Sliding Window Minimum)**!

Как работает монотонный дек (`std::deque`)?

Мы заводим двустороннюю очередь (дек), в которой храним **индексы** элементов. Мы поддерживаем железный инвариант:

> **Инвариант монотонности дека:** Значения dp по индексам в деке **строго возрастают** от начала к концу:

$$\mathrm{dp}[q[0]] < \mathrm{dp}[q[1]] < \mathrm{dp}[q[2]] < \ldots$$

1. **Взятие минимума за $O(1)$:** Самый маленький элемент в текущем окне **всегда лежит в голове дека** (`q.front()`)!
2. **Очистка устаревших:** Перед шагом i проверяем голову: если `q.front() < i - k` , этот элемент уже выпал из окна прыжка — делаем `q.pop_front()` .
3. **Добавление нового элемента $\mathrm{dp}[i]$:** Мы хотим положить i в хвост дека. Но если сзади лежат элементы, у которых значение $\mathrm{dp} \geq \mathrm{dp}[i]$, они **никогда в жизни больше не станут минимумом** (ведь новый элемент $\mathrm{dp}[i]$ и меньше их, и проживёт в окне дольше них!). Мы безжалостно выталкиваем их с конца `q.pop_back()` , пока хвост не станет $< \mathrm{dp}[i]$.

```cpp
#include <deque>

int grasshopperFast(int n, int k, const std::vector<int>& c) {
    std::vector<int> dp(n);
    std::deque<int> q; // Хранит индексы в порядке возрастания dp

    dp[0] = c[0];
    q.push_back(0);

    for (int i = 1; i < n; ++i) {
        // 1. Убираем из дека индексы, вылетевшие из окна длины k:
        while (!q.empty() && q.front() < i - k) {
            q.pop_front();
        }

        // 2. Минимум окна лежит в голове дека за O(1)!
        dp[i] = c[i] + dp[q.front()];

        // 3. Поддерживаем монотонность: выкидываем с конца все элементы >= dp[i]
        while (!q.empty() && dp[q.back()] >= dp[i]) {
            q.pop_back();
        }

        // 4. Добавляем текущий индекс в дек
        q.push_back(i);
    }

    return dp[n - 1];
}
```

Амортизационный анализ: почему это $O(n)$?

В коде два вложенных цикла `while` . Почему это не $O(n^2)$? Используем **амортизационный анализ**:

- Каждый индекс от 0 до $n-1$ добавляется в дек ровно **один раз** (`push_back`).
- Каждый индекс удаляется из дека не более **одного раза** (`pop_front` или `pop_back`).
- Суммарное число операций со всеми деками за всё время работы цикла не превышает $2n$!
- **Итог:** время работы — **строго $\Theta(n)$**, независимо от того, чему равно k!

5. Комбинаторный кузнечик: $O(k^3 \log n)$ матрицами

Если задача звучит так: **«Сколькими способами кузнечик может дойти до клетки n, прыгая на расстояния от 1 до k? Но $n = 10^{18}$!»**

Здесь переход линеен:

$$\mathrm{dp}[i] = \mathrm{dp}[i-1] + \mathrm{dp}[i-2] + ... + \mathrm{dp}[i-k]$$

Мы упаковываем последние k значений в вектор-столбец:

$$\begin{pmatrix} \mathrm{dp}[i] \\ \mathrm{dp}[i-1] \\ \vdots \\ \mathrm{dp}[i-k+1] \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & ... & 1 \\ 1 & 0 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ \vdots & \vdots & ....\text{down} & ... & \vdots \\ 0 & 0 & ... & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \mathrm{dp}[i-1] \\ \mathrm{dp}[i-2] \\ \vdots \\ \mathrm{dp}[i-k] \end{pmatrix}$$

Матрица размера $k \times k$ умножается на себя через быстрое бинарное возведение в степень (`BinPow`) за $\log_2 n$ шагов. Сложность: $\boldsymbol{O(k^3 \log n)}$!

Чек-лист эволюции задачи о кузнечике:

Метод	Время	Память	Идея
Наивный перебор	$O(k^n)$	$O(n)$	Рекурсивное дерево
Базовое ДП	$O(n \cdot k)$	$O(n)$	Вложенный цикл по прыжкам j
Сжатие памяти	$O(n \cdot k)$	$\boldsymbol{O(k)}$	Циклический буфер $i \bmod k$
Монотонный дек	$\boldsymbol{O(n)}$	$O(n)$ или $O(k)$	Скользящий минимум в окне длины k
Матричный BinPow	$\boldsymbol{O(k^3 \log n)}$	$O(k^2)$	Для $n \le 10^{18}$ и подсчёта путей

Видишь, как на одной простой задаче раскрывается вся мощь алгоритмической мысли? Готов переходить к следующему колоссу — **Теме 14: ДП Рюкзак (0-1, неограниченный, по стоимости и Meet-in-the-middle)**?

Тема 14. Задача о рюкзаке (Knapsack Problem)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Задача о рюкзаке: дано n предметов с весами w_i и стоимостями c_i, вместимость рюкзака — W. Требуется выбрать набор предметов максимальной суммарной стоимости с суммарным весом $\leq W$. Задача является **NP-полной**. Классическое ДП решается за **псевдополиномиальное время** $\Theta(nW)$:
> - В **0-1 рюкзаке** (предмет берётся ≤ 1 раза) одномерный массив $\mathrm{dp}[w]$ обходится **строго справа налево** ($w = W \ldots w_i$), чтобы не взять предмет дважды.
> - В **неограниченном рюкзаке** (предмет берётся сколько угодно раз) цикл идёт **слева направо** ($w = w_i \ldots W$).
> - Если W огромно (10^9), но мала сумма стоимостей, задача решается через **ДП по стоимости** за $\Theta(n \sum c_i)$.
> - Если $n \leq 40$, а числа произвольно велики, применяется метод **Meet-in-the-middle** за $\Theta\left(2^{\frac{n}{2}} \cdot n\right)$».

1. Постановка задачи и почему Жадный алгоритм не работает

Дано:
- Вместимость рюкзака: целое число W.
- n предметов. У каждого предмета i:
  - Вес: $w_i > 0$.
  - Стоимость (ценность): $c_i > 0$.

Почему нельзя просто брать самые «выгодные» предметы (Жадный подход)?
Естественная жадная эвристика — посчитать **удельную стоимость** («плотность ценности») каждого предмета:

$$\rho_i = \frac{c_i}{w_i} \quad (\text{рублей за килограмм})$$

и жадно набивать рюкзак предметами с максимальной плотностью.

> ⚠️ **Ловушка экзаменатора**
>
> **Убийственный контрпример для экзаменатора:** Пусть вместимость рюкзака $W = 50$.
> - Предмет 1: вес $w_1 = 10$, цена $c_1 = 60 \Longrightarrow \rho_1 = 6.0$ руб/кг.
> - Предмет 2: вес $w_2 = 20$, цена $c_2 = 100 \Longrightarrow \rho_2 = 5.0$ руб/кг.
> - Предмет 3: вес $w_3 = 30$, цена $c_3 = 120 \Longrightarrow \rho_3 = 4.0$ руб/кг.
>
> **Жадный алгоритм:** сначала возьмёт предмет 1 (плотность 6.0, осталось места 40), затем возьмёт предмет 2 (плотность 5.0, осталось места 20). Предмет 3 уже не влезает! Итог жадности: Предметы $1 + 2$, вес 30, **стоимость 160**.
>
> **Оптимальный ответ:** взять предметы $2 + 3$! Вес $20 + 30 = 50$, **стоимость $100 + 120 = 220$**! Жадный алгоритм с треском провалился, потому что в дискретном рюкзаке остаются «пустоты», которые невозможно заполнить кусками предметов.

2. Вариант 1: Классический 0-1 рюкзак (Каждый предмет берётся ≤ 1 раза)

Как мы приходим к состоянию ДП?

Мы перебираем предметы по очереди: $1, 2, \ldots, n$. Когда мы держим в руках i-й предмет, у нас есть ровно **два варианта действия**:

1. **НЕ БРАТЬ предмет i:** тогда максимальная стоимость равна лучшему результату для первых $i-1$ предметов при той же вместимости w.
2. **ВЗЯТЬ предмет i:** это возможно, только если $w \geq w_i$. Мы кладём предмет i в рюкзак (получая $+c_i$ к цене), а остаток рюкзака вместимости $w - w_i$ должны оптимально заполнить предметами из первых $i-1$.

Отсюда рождается двумерное состояние:

$$\mathbf{dp}[\boldsymbol{i}][\boldsymbol{w}]$$

> **Смысл состояния:** максимальная стоимость, которую можно унести в рюкзаке вместимости w, выбирая только среди первых i предметов.

Формула перехода (Recurrence):

$$\mathrm{dp}[i][w] = \begin{cases} \mathrm{dp}[i-1][w] & \text{если } w < w_i \text{ (предмет не влезает)} \\ \max(\mathrm{dp}[i-1][w],; \mathrm{dp}[i-1][w-w_i] + c_i) & \text{если } w \geq w_i. \end{cases}$$

База динамики:

- $\mathrm{dp}[0][w] = 0$ для всех w (из 0 предметов можно унести только 0 стоимости).
- $\mathrm{dp}[i][0] = 0$ для всех i (в рюкзак вместимости 0 ничего не влезет).

Трассировка на маленьком примере

Пусть $W = 4$. Предметы:

1. $w_1 = 2, c_1 = 3$
2. $w_2 = 1, c_2 = 2$
3. $w_3 = 3, c_3 = 4$

Строим таблицу $\mathrm{dp}[i][w]$:

i / w	**0**	**1**	**2**	**3**	**4**
0 (нет предм.)	0	0	0	0	0
1 (предм. 1: w=2, c=3)	0	0	3	3	3
2 (предм. 2: w=1, c=2)	0	2	3	**5**	5
3 (предм. 3: w=3, c=4)	0	2	3	5	**7**

Посмотрим на ячейку $\mathrm{dp}[2][3]$ (значение 5): Мы выбирали: либо не брать предмет 2 (значение сверху $\mathrm{dp}[1][3] = 3$), либо взять предмет 2 ($c_2 = 2$) плюс $\mathrm{dp}[1][3-1] = \mathrm{dp}[1][2] = 3$. Сумма $2 + 3 = 5$. Максимум — 5!

- **Время работы:** $\Theta(n \cdot W)$ (таблица размера $n \times W$).
- **Память:** $\Theta(n \cdot W)$.

---

3. Сжатие памяти до $O(W)$: Механизм обратного прохода

Таблица $n \times W$ при $n = 1000$ и $W = 10^6$ займёт $1000 \times 10^6 \times 4$ байт $\approx \mathbf{4}$ ГБ оперативной памяти! Это недопустимо.

Посмотрим на формулу:

$$\mathrm{dp}[i][w] = \max(\mathrm{dp}[\boldsymbol{i-1}][w],; \mathrm{dp}[\boldsymbol{i-1}][w - w_i] + c_i)$$

Чтобы посчитать строку i, нам нужна **только предыдущая строка $\boldsymbol{i-1}$**! Вся история до $i-2$ уже не нужна.

Казалось бы, можно завести одномерный массив `dp[w]` размера $W+1$.

> ⚠️ **Ловушка экзаменатора**
>
> **ГЛАВНЫЙ ВОПРОС КОЛЛОКВИУМА:** «В каком направлении должен крутиться цикл по весу w: от 0 до W или от W до 0? И почему?»

Давай разберём оба варианта и поймём физику процесса!

Вариант А: Идём слева направо ($w = w_i \ldots W$) — ОШИБКА ДЛЯ 0-1!

Пусть у нас один предмет: $w_1 = 2, c_1 = 10$. Массив `dp` изначально заполнен нулями: `[0, 0, 0, 0, 0, 0]`. Запускаем цикл слева направо:

- $w = 2 : \mathrm{dp}[2] = \max(\mathrm{dp}[2], \mathrm{dp}[0] + 10) = 10$. Массив: `[0, 0, 10, 0, 0, 0]`.
- $w = 3 : \mathrm{dp}[3] = 10$.
- $w = 4 : \mathrm{dp}[4] = \max(\mathrm{dp}[4], \mathbf{dp}[\mathbf{2}] + 10) = 10 + 10 = \mathbf{20}$!
- $w = 6 : \mathrm{dp}[6] = \max(\mathrm{dp}[6], \mathbf{dp}[\mathbf{4}] + 10) = 20 + 10 = \mathbf{30}$!

Что произошло?! Когда мы считали $\mathrm{dp}[4]$, мы обратились к уже обновлённой ячейке $\mathrm{dp}[2]$ и взяли предмет №1 **второй раз**! Когда считали $\mathrm{dp}[6]$, взяли его **в третий раз**! Проход слева направо решает задачу, где предметы **можно брать бесконечно много раз**!

Вариант Б: Идём строго СПРАВА НАЛЕВО ($w = W \ldots w_i$) — ИДЕАЛЬНО!

Тот же предмет: $w_1 = 2, c_1 = 10$. Исходный массив: `[0, 0, 0, 0, 0, 0]`. Запускаем цикл справа налево:

- $w = 6 : \mathrm{dp}[6] = \max(\mathrm{dp}[6], \mathbf{dp}[\mathbf{4}] + 10) = 0 + 10 = 10$. (Ячейка $\mathrm{dp}[4]$ ещё не тронута, там старый ноль!).
- $w = 5 : \mathrm{dp}[5] = \max(\mathrm{dp}[5], \mathbf{dp}[\mathbf{3}] + 10) = 0 + 10 = 10$.
- $w = 4 : \mathrm{dp}[4] = \max(\mathrm{dp}[4], \mathbf{dp}[\mathbf{2}] + 10) = 0 + 10 = 10$. (Ячейка $\mathrm{dp}[2]$ ещё старая!).
- $w = 2 : \mathrm{dp}[2] = \max(\mathrm{dp}[2], \mathbf{dp}[\mathbf{0}] + 10) = 0 + 10 = 10$.

Итог: предмет взят **строго один раз** во все подходящие вместимости!

Код 0-1 рюкзака с памятью $O(W)$:

```cpp
int knapsack01(int W, const std::vector<int>& weights, const std::vector<int>& costs) {
    int n = weights.size();
    std::vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        int w_i = weights[i];
        int c_i = costs[i];
        // СТРОГО СПРАВА НАЛЕВО ДО w_i:
        for (int w = W; w >= w_i; --w) {
            dp[w] = std::max(dp[w], dp[w - w_i] + c_i);
        }
    }
    return dp[W];
}
```

---

4. Восстановление ответа в 0-1 рюкзаке (Какие предметы взяли?)

> **Компромисс памяти и восстановления:** Если мы сжали память до одномерного массива $O(W)$, **восстановить состав предметов невозможно** (мы затерли промежуточные шаги). Чтобы восстановить сам набор предметов, **необходима полная 2D-таблица** $\mathrm{dp}[n][W]$.

Алгоритм восстановления (Backtracking):
Мы встаём в финальную ячейку $i = n, w = W$ и идём назад к $i = 0$:

- Если $\mathrm{dp}[i][w] == \mathrm{dp}[i-1][w]$: значит, предмет i **не брали** (значение пришло сверху). Просто уменьшаем $i \to i-1$.
- Иначе: предмет i **точно брали**! Добавляем его номер в ответ, вычитаем его вес из вместимости $w \to w - w_i$, и переходим к $i \to i-1$.

```
std::vector<int> taken_items;
int curr_w = W;
for (int i = n; i > 0; --i) {
    if (dp[i][curr_w] != dp[i - 1][curr_w]) {
        taken_items.push_back(i - 1); // Взяли предмет (i-1 в 0-индексации)
        curr_w -= weights[i - 1];
    }
}
```

---

5. Вариант 2: Неограниченный рюкзак (Unbounded Knapsack)

Каждый предмет можно брать **любое количество раз** (хоть 100 штук одного типа).

Из нашего предыдущего анализа всё уже очевидно: Мы берём тот же самый одномерный массив `dp[w]`, но запускаем цикл **слева направо**!

```
int unboundedKnapsack(int W, const std::vector<int>& weights, const std::vector<int>&
costs) {
    int n = weights.size();
    std::vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        // СЛЕВА НАПРАВО: позволяет брать предмет повторно!
        for (int w = weights[i]; w <= W; ++w) {
            dp[w] = std::max(dp[w], dp[w - weights[i]] + costs[i]);
        }
    }
    return dp[W];
}
```

Время: $\Theta(\mathrm{nW})$, память: $\Theta(W)$.

---

6. Вариант 3: Рюкзак по стоимости (Когда $W = 10^9$, но цены маленькие)

Это любимая олимпиадная задача на разворот пространства состояний.

> **Условие:** Вместимость рюкзака $W = 10^9$ (миллиард!). Предметов $n = 100$. Стоимость каждого предмета $c_i \leq 1000$. Если запустить обычный алгоритм, массив размера 10^9 потребует 4 гигабайта памяти и 10^{11} операций — мы гарантированно получим Memory Limit (ML) и Time Limit (TL).

Революция в мышлении: меняем местами параметр и значение!
Максимальная суммарная стоимость всех предметов:

$$C_{\max} = \sum_{i=1}^{n} c_i \le 100 \times 1000 = \mathbf{10^5} \quad (\text{очень маленькое число!}).$$

Давай определим состояние наоборот:

> **Новое состояние $\mathrm{dp}[c]$: Минимальный суммарный вес**, необходимый для того, чтобы набрать суммарную стоимость **ровно** c.

- **База:** $\mathrm{dp}[0] = 0$ (стоимость 0 весит 0 кг), все остальные $\mathrm{dp}[c] = +\infty$.
- **Переход:**

$$\mathrm{dp}[c] = \min(\mathrm{dp}[c],; \mathrm{dp}[c - c_i] + w_i), \quad \text{цикл } c = C_{\max}...c_i \ (\text{справа налево!})$$

- **Как получить ответ?** Пробегаем по массиву $\mathrm{dp}[c]$ от максимальной стоимости $C_{\max}$ вниз к 0. Первое найденное c, для которого вес влезает в рюкзак ($\mathrm{dp}[c] \le W$), и есть наш ответ!

```
int knapsackByCost(int W, const std::vector<int>& weights, const std::vector<int>& costs)
{
    int n = weights.size();
    int max_cost = 0;
    for (int c : costs) max_cost += c;

    const long long INF = 1e18;
    std::vector<long long> dp(max_cost + 1, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        for (int c = max_cost; c >= costs[i]; --c) {
            if (dp[c - costs[i]] != INF) {
                dp[c] = std::min(dp[c], dp[c - costs[i]] + weights[i]);
            }
        }
    }

    for (int c = max_cost; c >= 0; --c) {
        if (dp[c] <= W) return c; // Максимальная стоимость, влезшая в рюкзак W!
    }
    return 0;
}
```

- **Сложность:** Время $\Theta(n \sum c_i)$, Память $\Theta(\sum c_i)$.
- При $n = 100, \sum c_i = 10^5$ это всего 10^7 операций (0.01 секунды)!

7. Вариант 4: Метод «Встреча посередине» (Meet-in-the-Middle)

> **Условие:** $W = 10^{14}$, стоимости $c_i = 10^{14}$, но предметов мало: $n \le 40$.

Здесь бессильны оба ДП (и по весу, и по стоимости). Наивный полный перебор ($2^{40} \approx 1.1 \times 10^{12}$) тоже не успеет за 1 секунду.

Решение: Разделяй пополам ($\frac{n}{2} \le 20$)
Величина $2^{20} \approx 10^6$ — это крошечное число, перебирается за миллисекунды!

1. Делим n предметов на две группы по $\frac{n}{2} \le 20$ штук.
2. Для **первой группы** генерируем все 2^{20} подмножеств: получаем список пар `(вес, стоимость)`.

3. Для **второй группы** аналогично генерируем все 2^{20} пар `(вес, стоимость)`.
4. Сортируем вторую группу по весу. Оставляем в ней только точки Парето (если вес больше, а стоимость меньше — выкидываем).
5. Для каждой пары (w_1, c_1) из первой группы мы ищем во второй группе лучшую пару с весом $w_2 \le W - w_1$ с помощью **бинарного поиска (`std::upper_bound`)** за $O\big(\log\big(2^{\frac{n}{2}}\big)\big) = O(\frac{n}{2})$!

Сложность:

$$\boldsymbol{O\big(2^{\frac{n}{2}} \cdot n\big)}$$

При $n = 40$ это около $10^6 \times 20 \approx 2 \times 10^7$ операций. Решается мгновенно!

---

8. Почему задача о рюкзаке NP-полная и что такое «псевдополиномиальное время»?

Это любимый теоретический вопрос на экзамене:

> **Экзаменатор:** «Сложность 0-1 рюкзака — $\Theta(nW)$. Но ведь $n \cdot W$ — это полином! Почему тогда рюкзак называют NP-полной задачей, для которой нет полиномиального решения?»

Ответ, который производит фурор: Сложность в теории алгоритмов измеряется **от длины входа в битах**! Число W на вход подаётся не в виде строки из W палочек, а в виде двоичного числа, которое занимает всего:

$$L = \lceil \log_2 W \rceil \text{ бит.}$$

Выразим время работы через размер входа L:

$$T = \Theta(n \cdot W) = \Theta(n \cdot 2^L).$$

Время работы зависит **экспоненциально** от длины записи числа W в битах! Такие алгоритмы называют **псевдополиномиальными**: они выглядят как полином от численного значения W, но на самом деле экспоненциальны относительно размера входа.

Сводная шпаргалка по всем рюкзакам:

Задача	Ограничения	Метод	Время	Память
0-1 Рюкзак	$W \le 10^6$	ДП справа налево ($w = W \dots w_i$)	$\Theta(nW)$	$O(W)$
Неограниченный	$W \le 10^6$	ДП слева направо ($w = w_i \dots W$)	$\Theta(nW)$	$O(W)$
Рюкзак по стоимости	W огромное, $\sum c_i \le 10^5$	ДП с инверсией состояния $\mathrm{dp}[c]$	$\Theta(n \sum c_i)$	$O(\sum c_i)$
Meet-in-the-middle	W, C огромные, $n \le 40$	Перебор двух половин по $\frac{n}{2}$ + бинпоиск	$\Theta\big(2^{\frac{n}{2}} \cdot n\big)$	$O\big(2^{\frac{n}{2}}\big)$

Теперь семейство рюкзаков у тебя разобрано до атомов. Переходим к **Теме 15: ДП НВП (Наибольшая возрастающая подпоследовательность: $O(n^2)$ и $O(n \log n)$ с бинпоиском)**?

Тема 15. Наибольшая возрастающая подпоследовательность (НВП)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Задача об НВП: в числовом массиве A найти самую длинную подпоследовательность элементов, строго возрастающую слева направо ($A[i_1] < A[i_2] < ... < A[i_k]$ при $i_1 < i_2 < ... < i_k$). Базовое ДП решает задачу за $\Theta(n^2)$, где состояние $\mathrm{dp}[i]$ — длина НВП, оканчивающейся на $A[i]$. Быстрый алгоритм работает за $\Theta(n \log n)$ с помощью массива $\mathrm{tails}[\mathrm{len}]$, хранящего минимальный конец возрастающей подпоследовательности длины len. Массив tails строго монотонен, что позволяет находить позицию для каждого нового элемента **бинарным поиском** (`std::lower_bound` для строгой НВП и `std::upper_bound` для нестрогой). Память алгоритма — $O(n)$».

1. Подпоследовательность против Подмассива

Первая вещь, на которой ловят первокурсников:

- **Подмассив (Subarray):** Элементы обязаны идти **строго подряд** в исходном массиве без пропусков (непрерывный кусок).
- **Подпоследовательность (Subsequence):** Элементы **НЕ обязаны идти подряд**! Мы можем выкинуть любые элементы, главное — сохранить исходный порядок слева направо ($i_1 < i_2 < ... < i_k$).

Пример: Для массива $[10, \mathbf{2}, 5, \mathbf{3}, 7, \mathbf{101}, 18]$: Подпоследовательность $[2, 3, 7, 101]$ является возрастающей длины 4. А самая длинная — $[2, 3, 7, 18]$ (длина 4) или $[2, 5, 7, 18]$ (длина 4).

2. Базовое квадратичное решение за $\Theta(n^2)$

Чек-лист Беллмана:

1. Определение состояния (Самый важный трюк!):

> $\mathrm{dp}[i]$ — длина наибольшей возрастающей подпоследовательности, которая заканчивается **СТРОГО на элементе $A[i]$**.

Почему нельзя просто определить $\mathrm{dp}[i]$ как «длину НВП на префиксе $0...i$»? Потому что если мы не знаем, **какое именно число** стоит в конце подпоследовательности, мы не сможем понять, имеем ли мы право пристыковать к ней следующее число $A[i+1]$! Чтобы продолжить подпоследовательность числом X, нам жизненно необходимо знать её последний элемент Y и проверить $Y < X$.

2. База динамики:
$\mathrm{dp}[i] = 1$ для всех i (каждый элемент сам по себе является возрастающей подпоследовательностью длины 1).

3. Переход динамики:
Чтобы найти $\mathrm{dp}[i]$, мы смотрим на всех возможных предшественников $j < i$. Если $A[j] < A[i]$, мы можем пристыковать $A[i]$ в конец подпоследовательности, завершавшейся на $A[j]$:

$$\mathrm{dp}[i] = 1 + \max_{0 \le j < i \setminus A[j] < A[i]} \mathrm{dp}[j]$$

Если подходящих j нет, то $\mathrm{dp}[i] = 1$.

4. Итоговый ответ:
Так как НВП может заканчиваться на любом элементе массива:

$$\text{Длина НВП} = \max_{0 \le i < n} \text{dp}[i].$$

Код с восстановлением ответа за $\Theta(n^2)$:

```
void solveLIS_quadratic(const std::vector<int>& a) {
    int n = a.size();
    if (n == 0) return;

    std::vector<int> dp(n, 1);
    std::vector<int> parent(n, -1); // Для восстановления ответа

    int max_len = 1;
    int best_end = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j; // Запоминаем, что перед a[i] стоял a[j]
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
            best_end = i;
        }
    }

    // Восстанавливаем саму подпоследовательность по предкам:
    std::vector<int> lis;
    for (int curr = best_end; curr != -1; curr = parent[curr]) {
        lis.push_back(a[curr]);
    }
    std::reverse(lis.begin(), lis.end());

    std::cout << "Длина НВП: " << max_len << "\n";
    std::cout << "Сама НВП: ";
    for (int x : lis) std::cout << x << " ";
    std::cout << "\n";
}
```

- **Сложность:** Время $\Theta(n^2)$ (два вложенных цикла), Память $\Theta(n)$.

3. Быстрый алгоритм за $\Theta(n \log n)$ (Массив `tails` и бинпоиск)

Квадратичное решение делает слишком много лишней работы: на каждом шаге i мы перебираем все $j < i$. Можно ли ускорить поиск лучшего предшественника с $O(n)$ до $O(\log n)$? **Да!**

Жадная интуиция:
Представь, что мы уже нашли две разные возрастающие подпоследовательности одинаковой длины 3:

- Первая заканчивается на число $\mathbf{10}$ (например, $[2, 5, 10]$).
- Вторая заканчивается на число $\mathbf{6}$ (например, $[1, 4, 6]$).

Какая из них выгоднее для нас в будущем? **Конечно, вторая!** Потому что чем **меньше** последний элемент, тем выше шанс, что следующее пришедшее число x окажется больше него и сможет продолжить эту подпоследовательность (число 8 сможет продолжить вторую, но не сможет первую!).

Массив минимальных концов `tails`:
Заведём массив:

$$\mathbf{tails}[\mathbf{len}]$$

> **Смысл:** Минимальное значение последнего элемента среди ВСЕХ найденных возрастающих подпоследовательностей длины len.

Строгое доказательство монотонности массива `tails`:
Это главный вопрос экзаменатора: **«Почему вы имеете право применять бинарный поиск к массиву tails? Докажите, что он строго отсортирован!»**

> **Теорема:** В любой момент работы алгоритма массив tails строго возрастает:

$$\mathrm{tails}[1] < \mathrm{tails}[2] < \ldots < \mathrm{tails}[L].$$

Доказательство (от противного): Предположим, что это не так, и нашлись такие длины, что $\mathrm{tails}[\mathrm{len}] \geq \mathrm{tails}[\mathrm{len}+1]$.

1. По определению, $\mathrm{tails}[\mathrm{len}+1]$ — это конец некоторой подпоследовательности длины $\mathrm{len}+1$.
2. Выбросим из этой подпоследовательности её последний элемент.
3. Оставшийся префикс имеет длину len.
4. Так как подпоследовательность строго возрастала, предпоследний элемент y был **строго меньше** последнего элемента:

$$y < \mathrm{tails}[\mathrm{len}+1].$$

5. Но этот префикс — это подпоследовательность длины len, оканчивающаяся на y!
6. По определению $\mathrm{tails}[\mathrm{len}]$ — это **минимально возможный** конец среди всех подпоследовательностей длины len. Значит:

$$\mathrm{tails}[\mathrm{len}] \leq y < \mathrm{tails}[\mathrm{len}+1] \Longrightarrow \mathbf{tails}[\mathbf{len}] < \mathbf{tails}[\mathbf{len}+\mathbf{1}].$$

Получили строгое противоречие! Значит, предположение неверно, и массив tails **всегда строго возрастает**. $\square$.filled

---

Как работает алгоритм с `std::lower_bound`:
Мы обрабатываем элементы массива A по одному слева направо. Для каждого числа $x = A[i]$:

1. Так как массив tails строго возрастает, мы можем за $O(\log n)$ с помощью бинарного поиска (`std::lower_bound`) найти **первый элемент в $\mathbf{tails}$, который $\geq x$**:

```
auto it = std::lower_bound(tails.begin(), tails.end(), x);
```

2. **Случай А (элемент не найден, x больше всех хвостов):** Это значит, что x может продолжить самую длинную существующую подпоследовательность! Мы просто дописываем x в конец: `tails.push_back(x)`. Длина НВП выросла на 1!
3. **Случай Б (найден элемент `*it >= x` на позиции `len`):** Это значит, что мы нашли способ закончить подпоследовательность длины `len` числом x, которое **меньше или равно** текущего хвоста `*it`! Мы обновляем хвост: `*it = x`. Мы сделали подпоследовательность длины `len` более перспективной для будущего!

Наглядная трассировка: `A = [10, 9, 2, 5, 3, 7, 101, 18]`

- Пришло `10`: $\mathrm{tails} = [\mathbf{10}]$. (Длина 1).
- Пришло `9`: `9 < 10`, улучшаем длину 1: $\mathrm{tails} = [\mathbf{9}]$.
- Пришло `2`: `2 < 9`, улучшаем длину 1: $\mathrm{tails} = [\mathbf{2}]$.

- Пришло `5` : `5 > 2` , создаём длину 2: $\text{tails} = [2, ; \mathbf{5}]$.
- Пришло `3` : `3 < 5` , улучшаем длину 2: $\text{tails} = [2, ; \mathbf{3}]$.
- Пришло `7` : `7 > 3` , создаём длину 3: $\text{tails} = [2, 3, ; \mathbf{7}]$.
- Пришло `101` : `101 > 7` , создаём длину 4: $\text{tails} = [2, 3, 7, ; \mathbf{101}]$.
- Пришло `18` : `18 < 101` , улучшаем длину 4: $\text{tails} = [2, 3, 7, ; \mathbf{18}]$.

Финал: длина массива tails равна **4**! Это и есть длина наибольшей возрастающей подпоследовательности. Сложность: n чисел $\times$ бинпоиск $O(\log n) = \boldsymbol{\Theta(n \log n)}$!

---

4. Главная ловушка: Хранит ли массив `tails` саму НВП?

> ⚠️ **Ловушка экзаменатора**
>
> **САМЫЙ КОВАРНЫЙ ВОПРОС КОЛЛОКВИУМА:** «Посмотрите на получившийся массив $\text{tails} = [2, 3, 7, 18]$. Является ли он сам по себе искомой подпоследовательностью?»

Ответ: НЕТ, НИ В КОЕМ СЛУЧАЕ! В нашем примере числа случайно совпали, но в общем случае в массиве tails лежат элементы из **совершенно разных, несовместимых подпоследовательностей**!

Контрпример:
Пусть массив $A = [3, 4, 1]$.
1. Пришло `3` : $\text{tails} = [3]$.
2. Пришло `4` : $\text{tails} = [3, 4]$. (НВП = $[3, 4]$, длина 2).
3. Пришло `1` : `1 < 3` , обновляем ячейку 0: $\text{tails} = [1, 4]$.

Длина массива tails равна 2 (найдена верно!). Но если мы выведем сам массив tails, мы получим $[1, 4]$. А ведь подпоследовательности $[1, 4]$ **вообще не существует в массиве** A (число 1 шло ПОСЛЕ числа 4)!

Как восстановить саму НВП за $\Theta(n \log n)$?

Чтобы восстановить ответ, нужно сохранять историю:
1. `pos[len]` — индекс в исходном массиве A, который сейчас является хвостом длины len.
2. `parent[i]` — индекс элемента, который стоял перед $A[i]$ в момент, когда $A[i]$ стал частью подпоследовательности.

```cpp
std::vector<int> findLIS(const std::vector<int>& a) {
    int n = a.size();
    if (n == 0) return {};

    std::vector<int> tails;     // Значения минимальных хвостов
    std::vector<int> tails_idx; // Индексы этих хвостов в массиве a
    std::vector<int> parent(n, -1);

    for (int i = 0; i < n; ++i) {
        int x = a[i];
        auto it = std::lower_bound(tails.begin(), tails.end(), x);
        int len = it - tails.begin();

        if (it == tails.end()) {
            tails.push_back(x);
            tails_idx.push_back(i);
        } else {
            *it = x;
            tails_idx[len] = i;
        }
```

```
        // Предок текущего элемента — это хвост длины len - 1
        if (len > 0) {
            parent[i] = tails_idx[len - 1];
        }
    }

    // Восстанавливаем ответ от последнего элемента хвоста
    std::vector<int> lis;
    for (int curr = tails_idx.back(); curr != -1; curr = parent[curr]) {
        lis.push_back(a[curr]);
    }
    std::reverse(lis.begin(), lis.end());
    return lis;
}
```

5. Вариация: Неубывающая подпоследовательность (LNDS)

Что делать, если в условии задачи сказано: **«найдите наибольшую НЕУБЫВАЮЩУЮ подпоследовательность ($A[i_1] \le A[i_2] \le ... \le A[i_k]$)»**?

В коде меняется **ровно одна функция**: Вместо `std::lower_bound` мы пишем **`std::upper_bound`**!

- `lower_bound` ищет первый элемент $\ge x$. Если приходит число x, равное текущему хвосту, `lower_bound` заменит этот хвост на x (длина не вырастет, строгое возрастание).
- `upper_bound` ищет первый элемент **строго** $> x$. Если приходит число, равное текущему хвосту, `upper_bound` не тронет его, а пойдёт дальше и позволит **увеличить длину на равном элементе**!

6. Прорыв для 1 и 2 потока: Связь с Таблицами Юнга и RSK

Если ты расскажешь это у доски, экзаменатор поставит максимальный балл без дополнительных вопросов:

> **Теорема Шенстеда (1961):** Длина наибольшей возрастающей подпоследовательности массива A равна в точности **длине первой строки таблицы Юнга (Young Tableau)**, полученной с помощью алгоритма вставки Робинсона — Шенстеда — Кнута (RSK bumping algorithm)!
>
> Более того, **длина первой колонки** той же таблицы Юнга равна в точности длине **наибольшей убывающей подпоследовательности**! А по **теореме Грина (Greene's Theorem)** сумма длин первых k строк таблицы Юнга равна максимальной суммарной длине объединения k непересекающихся возрастающих подпоследовательностей.

Сводная таблица по НВП:

Метод	Время	Память	Особенность
Базовое ДП	$\Theta(n^2)$	$\Theta(n)$	Просто пишется, легко восстанавливать путь
Бинпоиск (`tails`)	$\boldsymbol{\Theta(n \log n)}$	$\Theta(n)$	Идеальная скорость, требует `parent` для пути
Строгая НВП	$\Theta(n \log n)$	$\Theta(n)$	Используется `std::lower_bound`
Неубывающая (LNDS)	$\Theta(n \log n)$	$\Theta(n)$	Используется `std::upper_bound`

Вся тема НВП теперь выстроена в чёткую логическую линию: от инварианта $\text{tails}[\text{len}]$ до алгоритма RSK.

Готов перейти к предпоследней теме — **Теме 16: Динамика на отрезках и по профилю (только для 1, 2 потока)**?

Тема 16. Динамика на отрезках и по профилю

БЛОК 1. Динамика на отрезках (Interval DP)

1. В чём главная идея «отрезков»?

Во всех предыдущих задачах (кузнечик, рюкзак, НВП) подзадача определялась **одним префиксом**: «решить задачу для первых i элементов». Мы шли слева направо: $i = 1, 2, 3 \ldots$

Но в некоторых задачах префиксы бессильны! Представь длинную деревянную рейку. Чтобы её распилить или склеить, нам нужно знать ответ для **любого непрерывного куска от i до j** (для любого подотрезка $[i \ldots j]$).

> **Определение состояния в динамике на отрезках:** $\mathrm{dp}[i][j]$ — это оптимальный ответ для подотрезка данных **с i-го по j-й элемент включительно**.

---

2. Классическая задача: Оптимальное перемножение цепочки матриц

Пусть нам нужно перемножить цепочку из нескольких матриц:

$$A_1 \times A_2 \times A_3 \times A_4$$

Из школьной линейной алгебры мы знаем:
1. Матрицы умножаются ассоциативно: $(A \times B) \times C = A \times (B \times C)$. Ответ от расстановки скобок **не меняется**.
2. Но **количество элементарных скалярных умножений чисел** зависит от порядка скобок катастрофически!

Наглядный шок-пример:
Умножаем матрицу A размера 10×100 на B размера 100×5 и на C размера 5×50. **(Напоминание: умножение матрицы $p \times q$ на $q \times r$ требует $p \cdot q \cdot r$ операций).**

- **Вариант 1:** $(A \times B) \times C$
  1. $A \times B$: результат 10×5. Операций: $10 \cdot 100 \cdot 5 = \mathbf{5,000}$.
  2. $(\mathrm{AB}) \times C$: результат 10×50. Операций: $10 \cdot 5 \cdot 50 = \mathbf{2,500}$.
  - **Всего операций: $\mathbf{5,000 + 2,500 = 7,500}$!**
- **Вариант 2:** $A \times (B \times C)$
  1. $B \times C$: результат 100×50. Операций: $100 \cdot 5 \cdot 50 = \mathbf{25,000}$.
  2. $A \times (\mathrm{BC})$: результат 10×50. Операций: $10 \cdot 100 \cdot 50 = \mathbf{50,000}$.
  - **Всего операций: $\mathbf{25,000 + 50,000 = 75,000}$!**

Разница в **10 раз** на пустом месте! Мы хотим найти расстановку скобок, минимизирующую число операций.

---

3. Решение через ДП на отрезках

Пусть матрица A_i имеет размерность $p_{i-1} \times p_i$.

Шаг 1. Состояние

> $\mathbf{dp}[i][j]$ — минимальное число скалярных умножений, чтобы перемножить подцепочку матриц $A_i \times A_{i+1} \times \ldots \times A_j$.

Шаг 2. База динамики

Если отрезок состоит из одной матрицы ($i == j$), её ни с чем умножать не надо:

$$\mathrm{dp}[i][i] = 0.$$

Шаг 3. Переход (Где поставить последнюю скобку?)

В любом произведении $A_i \dots A_j$ на самом последнем шаге перемножаются две матрицы: левый кусок $(A_i \dots A_k)$ и правый кусок $(A_{k+1} \dots A_j)$. Где находится эта граница k? Мы не знаем! Поэтому мы просто **перебираем все возможные точки разреза k от i до $j-1$**:

```
Отрезок [i ... j]:
[ A_i  ...  A_k ]   x   [ A_{k+1}  ...  A_j ]
   левый кусок               правый кусок
     dp[i][k]                  dp[k+1][j]
```

При склейке левого куска (размера $p_{i-1} \times p_k$) и правого (размера $p_k \times p_j$) потребуется ещё $p_{i-1} \cdot p_k \cdot p_j$ операций. Получаем формулу:

$$\mathbf{dp}[i][j] = \min_{i \le k < j} \left(\mathbf{dp}[i][k] + \mathbf{dp}[k+1][j] + p_{i-1} \cdot p_k \cdot p_j \right)$$

Шаг 4. В КАКОМ ПОРЯДКЕ КРУТИТЬ ЦИКЛЫ? (Ловушка!)

> ⚠️ **Ловушка экзаменатора**
>
> Если написать стандартные циклы `for i = 1 to n` и `for j = 1 to n`, код **выдаст мусор**! Почему? Чтобы посчитать $\mathrm{dp}[1][4]$, нам нужно знать $\mathrm{dp}[2][4]$. Но если мы идём по i от 1, то $\mathrm{dp}[2][4]$ ещё не посчитан!

Золотое правило динамики на отрезках: Циклы крутятся **ПО ДЛИНЕ ОТРЕЗКА ($\mathrm{len} = 2, 3, \dots, n$)**!

- Сначала считаем все отрезки длины 2 ($[1..2], [2..3], [3..4]$).
- Затем длины 3 ($[1..3], [2..4]$) — ведь они опираются на уже готовые отрезки меньшей длины!
- В самом конце — один отрезок максимальной длины n ($[1..n]$).

```cpp
// Порядок обхода:
for (int len = 2; len <= n; ++len) { // Длина отрезка
    for (int i = 1; i <= n - len + 1; ++i) { // Левая граница
        int j = i + len - 1; // Правая граница
        dp[i][j] = INF;
        for (int k = i; k < j; ++k) { // Перебор точки раздела
            int cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
            dp[i][j] = std::min(dp[i][j], cost);
        }
    }
}
```

Сложность:

- 3 вложенных цикла: длина (n), левая граница (n), точка раздела (n).
- Время: ровно $\Theta(n^3)$.
- Память: таблица $n \times n \Longrightarrow \Theta(n^2)$.

(Для 1 потока на заметку: если целевая функция удовлетворяет неравенству четырёхугольника, то применима оптимизация Кнута — Яо**, которая сужает перебор k и ускоряет время с $O(n^3)$ до $O(n^2)$).**

БЛОК 2. Динамика по профилю (Profile DP)

Теперь переходим к тому, что пугает тебя больше всего: **домино и профили**. Давай снимем этот страх.

1. Задача о замощении сетки доминошками 2×1

Дана прямоугольная таблица размера $N \times M$ (например, пол в комнате). У нас есть бесконечный запас плиток домино размера 2×1 (каждая плитка закрывает 2 соседние клетки либо горизонтально, либо вертикально). **Вопрос:** сколькими способами можно полностью покрыть сетку $N \times M$ плитками без наложений и дырок?

> **Главный нюанс ограничений:** В задачах на ДП по профилю одна из сторон сетки **ОЧЕНЬ МАЛЕНЬКАЯ** (обычно $N \leq 16$, а часто $N = 3, 4, 5$), а вторая сторона M может быть большой. Если сетка 100×100, ДП по профилю неприменимо!

---

2. Что такое «Профиль»? (Образ кирпичной стены)

Представь, что мы замащиваем пол **столбец за столбцом слева направо**: Столбец 1, затем столбец 2, затем 3...

Когда мы замостили первые $i - 1$ столбцов, некоторые плитки, положенные горизонтально, могут **«выглядывать» (торчать) в следующий i-й столбец**!

```
Столбец (i-1)      Столбец i
   [ X ]  ----->    [ 1 ]   (плитка вылезла сюда из прошлого столбца)
   [ . ]            [ 0 ]   (клетка свободна)
   [ X ]  ----->    [ 1 ]   (плитка вылезла сюда из прошлого столбца)
```

Взгляни на столбец i: В нём какие-то клетки **уже заняты** горизонтальными плитками из прошлого столбца, а какие-то **свободны**! Этот срез на границе между обработанной и необработанной частью сетки и называется **ПРОФИЛЕМ**!

Профиль — это просто БИТОВАЯ МАСКА!

В столбце всего N клеток (по вертикали). Каждая клетка может находиться в одном из двух состояний:

- `1` — клетка уже занята (в неё торчит доминошка).
- `0` — клетка свободна.

Следовательно, границу между столбцами можно закодировать **одним целым числом — битовой маской от 0 до $2^N - 1$**! Если $N = 3$, то масок всего $2^3 = 8$ штук (от `000` до `111`).

---

3. Базовый постолбцовый профиль: $\mathrm{dp}[i][\mathrm{mask}]$

Шаг 1. Состояние

> $\mathbf{dp}[i][\mathbf{mask}]$ — количество способов замостить первые $i - 1$ столбцов так, чтобы в i-й столбец торчал профиль mask.

Шаг 2. База

- Перед 1-м столбцом ничего не торчит: $\mathrm{dp}[1][0] = 1$ (все остальные маски для 1-го столбца равны 0).

Шаг 3. Переход

Чтобы перейти от столбца i к столбцу $i + 1$: Мы смотрим на маску mask_1 в i-м столбце и думаем: какими плитками можно заполнить оставшиеся нули в i-м столбце, и какая маска mask_2 при этом вылезет в $(i + 1)$-й столбец?

Две маски mask_1 и mask_2 называются **совместимыми**, если можно положить горизонтальные плитки (они превратят 0 в mask_1 в 1 в mask_2) и вертикальные плитки (закроют соседние нули внутри самого i-го столбца).

$$\text{dp}[i+1][\text{mask}_2] = \sum_{\text{mask}_1 \text{ совместима с } \text{mask}_2} \text{dp}[i][\text{mask}_1]$$

Сложность постолбцового подхода:
- Столбцов: M.
- Масок в текущем столбце: 2^N.
- Масок в следующем столбце: 2^N.
- Время: $M \times 2^N \times 2^N = \boldsymbol{O(M \cdot 2^{2N}) = O(M \cdot 4^N)}$.

Если $N = 10$, то $4^{10} \approx 10^6$ — уже тяжеловато.

И тут советские математики придумали гениальный трюк — **изломанный профиль**!

---

4. Изломанный профиль (Broken Profile): Шедевр за $O(N \cdot M \cdot 2^N)$

> **В чём интуиция?** Зачем шагать сразу целыми столбцами и мучиться с 4^N переходов? Давайте класть плитки **по одной единственной клеточке за шаг**!

Мы обходим сетку как при чтении книги: слева направо, сверху вниз: клетка $(0,0) \to (1,0) \to \ldots \to (N-1,0) \to (0,1) \ldots$

Как выглядит «изломанный» профиль?
В любой момент времени граница между замощенной частью сетки и пустой имеет форму **«уголка»** вокруг текущей клетки (r, c):

```
           столбец c     столбец c+1
строка 0:    [X]            [.]
строка 1:    [X]            [.]
строка r:    (r, c) <-- СЕЙЧАС СМОТРИМ СЮДА!
строка r+1:  [.]            [.]
```

Нам нужно помнить состояние всего $\boldsymbol{N}$ **пограничных клеток** вокруг текущей позиции (это опять маска из N бит!).

Всего 2 варианта для текущей клетки (r, c):
Мы смотрим на бит текущей клетки в маске:

1. **Если бит равен 1 (клетка УЖЕ занята):** Мы ничего не можем сюда положить! Мы просто шагаем в следующую клетку, сбрасывая этот бит в 0.
2. **Если бит равен 0 (клетка СВОБОДНА):** У нас есть ровно два варианта положить плитку:
   - **Положить горизонтально:** плитка займёт (r, c) и вылезет в $(r, c+1)$. В новой маске бит станет 1.
   - **Положить вертикально:** (если $r + 1 < N$ и клетка снизу свободна) плитка займёт (r, c) и $(r+1, c)$.

Почему изломанный профиль побеждает?
На каждом шаге мы рассматриваем **всего одну клетку** и делаем не более **2 переходов**!
- Всего клеток: $N \times M$.
- Состояний профиля: 2^N.
- Переходов из каждого: $O(1)$!
- **Суммарное время:**

$$\boldsymbol{O(N \cdot M \cdot 2^N)}$$

Сравни: $M \cdot 4^N$ против $N \cdot M \cdot 2^N$. При $N = 10$: $4^{10} \approx 1,048,576$, а $10 \cdot 2^{10} \approx 10,240$! **Ускорение в 100 раз!**

---

5. Для 1 потока: Матричная экспоненциация $O(8^N \log M)$

А что, если на экзамене скажут:

> **«Сетка узкая ($N = 3$), но длина поля астрономическая: $M = 10^{18}$! Ваше $O(M \cdot 2^N)$ сделает 10^{18} шагов и зависнет. Как решить?»**

Посмотрим на постолбцовый переход:

$$\mathrm{dp}[i+1][\mathrm{mask}_2] = \sum_{\mathrm{mask}_1} \mathrm{dp}[i][\mathrm{mask}_1] \cdot \mathrm{compatible}(\mathrm{mask}_1, \mathrm{mask}_2)$$

Матрица совместимости T имеет фиксированный размер $2^N \times 2^N$. Она **не зависит от номера столбца i**! Переход от столбца i к $i+1$ — это просто умножение вектора состояний на матрицу T:

$$V_{i+1} = T \cdot V_i \Longrightarrow V_M = T^{M-1} \cdot V_1.$$

Мы возводим матрицу размера $2^N \times 2^N$ в степень M за $O(\log M)$ с помощью бинарного возведения в степень (`BinPow`)!

- Размер матрицы: $K = 2^N$.
- Умножение двух матриц: $K^3 = (2^N)^3 = \mathbf{8^N}$.
- Общее время: $\boldsymbol{O(8^N \cdot \log M)}$!

При $N = 3$: $8^3 = 512$ операций на один шаг умножения матриц. Задача для $M = 10^{18}$ решается за долю миллисекунды!

Твоя шпаргалка на случай ответа у доски:

1. **Динамика на отрезках ($\mathbf{dp}[i][j]$):**
   - Пример: цепочка матриц $A_i \dots A_j$.
   - Циклы крутятся **по длине отрезка** $\mathbf{len}$ от 2 до n.
   - Сложность $\Theta(n^3)$ по времени, $\Theta(n^2)$ по памяти.
2. **Динамика по профилю ($\mathbf{dp}[i][\mathbf{mask}]$):**
   - Применяется, когда одна сторона сетки маленькая ($N \le 16$).
   - Профиль — это битовая маска границы между замощенной и пустой частью.
   - Постолбцовый профиль: время $O(M \cdot 4^N)$.
   - Поклеточный изломанный профиль: время $O(N \cdot M \cdot 2^N)$.
   - При $M \le 10^{18}$: быстрое возведение матрицы переходов в степень за $O(8^N \log M)$.

Как сейчас ощущения? Развеялся ли туман вокруг этих страшных названий?

Тема 17. Редакционное расстояние (Алгоритм Вагнера — Фишера)

Блиц-ответ на 30 секунд у доски:

> ⚡ **30-секундный ответ у доски**
>
> «Редакционное расстояние (расстояние Левенштейна) между двумя строками S и T — это **минимальное количество элементарных операций**, необходимых для превращения строки S в строку T. Разрешены три операции: **вставка** символа, **удаление** символа и **замена** одного символа на другой. Задача решается динамическим программированием (алгоритм Вагнера — Фишера) за время $\Theta(\mathbf{nm})$ и память $\Theta(\mathbf{nm})$. Память легко сжимается до $O(\min(n, m))$ двумя строками, а восстановить само оптимальное выравнивание за линейную память $O(n + m)$ позволяет **алгоритм Хиршберга**. Расстояние Левенштейна образует строгое **метрическое пространство** (удовлетворяет аксиомам тождества, симметрии и неравенству треугольника)».

1. Три элементарные операции Левенштейна

Пусть советский математик Владимир Левенштейн (1965 г.) предложил измерять разницу между строками тремя операциями:

Операция	Что происходит	Пример: превращаем «КОТ» в «СКАТ»	Стоимость
1. Вставка (Insert)	Вставляем новый символ в любое место	Вставляем `'С'` в начало: `"КОТ"` → `"СКОТ"`	1
2. Удаление (Delete)	Стираем один символ	Удаляем `'О'`: `"СКОТ"` → `"СКТ"`	1
3. Замена (Replace)	Меняем один символ на другой	Меняем `'О'` на `'А'`: `"КОТ"` → `"КАТ"`	1 (если символы разные)0 (если одинаковые!)

> **Пример:** Сколько правок нужно, чтобы превратить **«КОТ»** в **«СКАТ»**?
> 1. Вставить `'С'` в начало → «СКОТ» (1 правка).
> 2. Заменить `'О'` на `'А'` → «СКАТ» (ещё 1 правка).
>
> Итого расстояние Левенштейна равно **2**!

2. Переход к ДП: Алгоритм Вагнера — Фишера (1974)

Пусть дана строка S длины n и строка T длины m. Мы хотим превратить $S \to T$.

Применим наш классический **Чек-лист ДП**:

Шаг 1. Определение состояния:

> $\mathbf{dp}[i][j]$ — минимальное количество операций, чтобы превратить префикс $S[1...i]$ длины i в префикс $T[1...j]$ длины j.

Итоговый ответ всей задачи будет лежать в правом нижнем углу таблицы: $\mathbf{dp}[n][m]$.

Шаг 2. База динамики (Пустые строки):
Что делать, если одна из строк пустая?
1. $\mathrm{dp}[0][0] = 0$ (пустая строка в пустую превращается за 0 правок).
2. $\mathrm{dp}[i][0] = i$ (чтобы превратить строку длины i в пустую строку, нужно сделать i **удалений**).
3. $\mathrm{dp}[0][j] = j$ (чтобы из пустой строки получить строку длины j, нужно сделать j **вставок**).

Первая строка таблицы будет: `[0, 1, 2, 3, ...]`, первый столбец: `[0, 1, 2, 3, ...]`.

Шаг 3. Рекуррентный переход (Выбор из трёх вариантов):
Представь, что мы стоим в ячейке (i, j) и смотрим на последние символы префиксов: $S[i]$ и $T[j]$. Как мы могли получить префикс $T[1...j]$ из $S[1...i]$? Есть ровно 3 пути:

```
             dp[i-1][j] (Удаление)
                 |
                 v
dp[i][j-1] ---> dp[i][j] <--- dp[i-1][j-1] (Замена или совпадение)
(Вставка)
```

1. **Путь сверху (Удаление):** Мы стёрли символ $S[i]$. Значит, префикс $S[1...i-1]$ уже должен был превратиться в $T[1...j]$. Стоимость: $\mathrm{dp}[i-1][j] + 1$.
2. **Путь слева (Вставка):** Мы вставили в конец символ $T[j]$. Значит, $S[1...i]$ уже был превращён в $T[1...j-1]$. Стоимость: $\mathrm{dp}[i][j-1] + 1$.
3. **Путь по диагонали (Замена или Совпадение):** Мы сопоставили символ $S[i]$ с символом $T[j]$:
   - Если символы **совпадают** ($S[i] == T[j]$): править ничего не нужно! Стоимость: $\mathrm{dp}[i-1][j-1] + 0$.
   - Если символы **различаются** ($S[i] \neq T[j]$): мы заменили $S[i]$ на $T[j]$. Стоимость: $\mathrm{dp}[i-1][j-1] + 1$.

Берём самый дешёвый вариант из трёх:

$$\mathbf{dp}[i][j] = \mathbf{min}\begin{cases} \mathbf{dp}[i-1][j] + 1 & (\text{удаление}) \\ \mathbf{dp}[i][j-1] + 1 & (\text{вставка}) \\ \mathbf{dp}[i-1][j-1] + [S[i] \neq T[j]] & (\text{замена / совпадение}). \end{cases}$$

---

3. Живая трассировка на таблице: «МАРТ» → «КАРТА»

Пусть $S = \text{МАРТ}$ ($n = 4$), $T = \text{КАРТА}$ ($m = 5$). Заполняем таблицу по строкам слева направо:

S / T	∅	К	А	Р	Т	А
∅	**0**	1	2	3	4	5
М	1	**1**	2	3	4	5
А	2	2	**1**	2	3	4
Р	3	3	2	**1**	2	3
Т	4	4	3	2	**1**	**2**

Посмотрим, как получилось финальное число **2** в правом нижнем углу:
1. Заменили „М“ на „К“ $\Longrightarrow$ «КАРТ» (1 замена).
2. Символы „А“, „Р“, „Т“ совпали (0 правок!).
3. Вставили „А“ в конец $\Longrightarrow$ «КАРТА» (1 вставка).

Итого: ровно 2 операции!

- **Время работы:** $\Theta(n \cdot m)$ (таблица $n \times m$, на каждую ячейку $O(1)$).

- **Память:** $\Theta(n \cdot m)$.

---

4. Сжатие памяти до $O(\min(n, m))$

Посмотрим на переходы: ячейка $\mathrm{dp}[i][j]$ зависит только от:
- Своей же строки ($\mathrm{dp}[i][j-1]$).
- Предыдущей строки ($\mathrm{dp}[i-1][j]$ и $\mathrm{dp}[i-1][j-1]$).

Вся таблица нам не нужна! Достаточно хранить **всего две строки**: `prev` и `curr`. Память падает с $O(\mathrm{nm})$ до $\boldsymbol{O(\min(n, m))}$ (мы можем пустить более короткую строку по столбцам).

---

5. Высший пилотаж для 1 и 2 потока: Алгоритм Хиршберга (Память $O(n + m)$)

На экзамене обязательно спросят:

> **Экзаменатор:** «Отлично, двумя строками вы нашли само расстояние за $O(m)$ памяти. А теперь выведите **саму цепочку правок (выравнивание строк)**! Но всю таблицу $n \times m$ в память сохранять запрещено!»

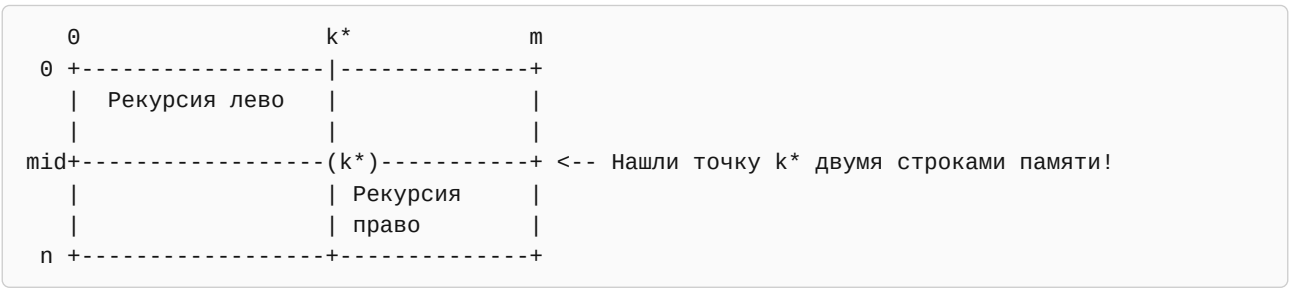
Казалось бы, тупик: без таблицы нельзя пойти назад стрелочками! В 1975 году Дэниел Хиршберг нашёл фантастическое решение, объединив **ДП и парадигму «Разделяй и властвуй»**.

Идея Хиршберга:

1. Разрежем строку S ровно пополам: в позиции $\mathrm{mid} = \frac{n}{2}$.
2. Мы знаем, что оптимальный путь в таблице обязан пересечь эту горизонтальную черту mid в какой-то точке (mid, k) по столбцу $k \in [0, m]$!
3. Чтобы найти эту точку раздела k:
   - Запускаем прямое ДП от начала S до середины mid (двумя строками памяти). Получаем вектор расстояний $\mathrm{dp}_{\text{left}}[k]$.
   - Запускаем обратное ДП от конца S до середины mid (тоже двумя строками памяти!). Получаем вектор $\mathrm{dp}_{\text{right}}[m-k]$.
   - Ищем индекс k, минимизирующий сумму:

$$k^* = \mathrm{argmin}_k \big(\mathrm{dp}_{\text{left}}[k] + \mathrm{dp}_{\text{right}}[m-k]\big).$$

4. Точка (mid, k^*) гарантированно лежит на оптимальном пути!
5. Мы рекурсивно запускаем алгоритм для двух меньших подпрямоугольников:
   - Левый верхний: от $(0, 0)$ до (mid, k^*).
   - Правый нижний: от (mid, k^*) до (n, m).



Сложность Хиршберга:

- **Память:** $O(n + m)$ — хранятся только векторы среза!
- **Время работы:**

$$T = \mathrm{nm} + \frac{\mathrm{nm}}{2} + \frac{\mathrm{nm}}{4} + \ldots \leq 2\mathrm{nm} = \boldsymbol{\Theta(\mathrm{nm})}.$$

Время осталось тем же $O(\mathrm{nm})$, а память стала **строго линейной** $\boldsymbol{O(n + m)}$!

---

6. Почему расстояние Левенштейна — это настоящая Метрика?

Экзаменатор-математик может спросить: **«Докажите, что расстояние Левенштейна $d(S,T)$ задаёт метрическое пространство»**.

Нужно проверить **4 аксиомы метрики**:

1. **Неотрицательность:** $d(S,T) \geq 0$ (число операций не может быть отрицательным).
2. **Аксиома тождества:** $d(S,T) = 0 \iff S == T$ (0 операций нужно только тогда, когда строки уже идентичны).
3. **Симметричность:** $d(S,T) = d(T,S)$ (любая операция обратима: удаление символа из S эквивалентно вставке символа в T).
4. **Неравенство треугольника:**

$$d(A,C) \leq d(A,B) + d(B,C)$$

Доказательство: Пусть мы умеем превратить $A \to B$ за $d(A,B)$ шагов, а $B \to C$ за $d(B,C)$ шагов. Если мы просто применим сначала первую цепочку правок, а затем вторую, мы гарантированно превратим A в C за $d(A,B) + d(B,C)$ шагов. Минимальное же расстояние $d(A,C)$ не может быть хуже, чем эта конкретная последовательность. Значит, $d(A,C) \leq d(A,B) + d(B,C)$. $\square$.filled

7. Модификация: Расстояние Дамерау — Левенштейна

В реальной жизни люди часто делают опечатку типа **транспозиции** (перестановка двух соседних букв местами: написали «**првиет**» вместо «**привет**»). В обычном Левенштейне это стоило бы 2 операции (заменить „в“ на „и“, а „и“ на „в“). Фредерик Дамерау добавил **4-ю операцию — перестановку соседних символов стоимостью 1**:

Если $S[i] == T[j-1]$ и $S[i-1] == T[j]$, добавляется четвёртая ветка в $\min$:

$$\mathrm{dp}[i-2][j-2] + 1 \quad (\text{транспозиция}).$$

Твоя шпаргалка по Редакционному расстоянию:

- **Базовый алгоритм:** Вагнер — Фишер за $\Theta(\mathrm{nm})$ времени и памяти.
- **3 операции:** вставка ($\mathrm{dp}[i][j-1]+1$), удаление ($\mathrm{dp}[i-1][j]+1$), замена/совпадение ($\mathrm{dp}[i-1][j-1] + \mathrm{cost}$).
- **Оптимизация памяти:** 2 строки $\Longrightarrow O(\min(n,m))$.
- **Восстановление ответа в малом объёме памяти:** Алгоритм Хиршберга за $O(n+m)$ памяти и $O(\mathrm{nm})$ времени.
- **Математика:** является строгой метрикой (выполняется неравенство треугольника).

Финал нашей программы!

Мы прошли **все 17 тем** твоей программы: от RAM-модели до алгоритма Хиршберга! Как ты себя чувствуешь? Есть ли какая-то тема или отдельный вопрос из пройденного, по которому хочется ещё раз пройтись или устроить мини-репетицию устного ответа?